STATUTORY SOFTWARE COPYRIGHT FILING DOSSIER

Autonomous Multi-Agent UAV Swarm Fault Detection, Isolation (FDI), and Two-Layer Guardrail Control Architecture

Document Type	Official Statutory Software Copyright Deposit & Technical Description
Subject Matter	Computer Software Work / Literary Work under Copyright Act
Project Title	Multi-Agent UAV Swarm FDI with Agentic AI & Formal CBF Guardrails
Filing Date	October 26, 2026
Target Jurisdiction	National Copyright Offices (Form XIV / Form TX compliant)
Total Deposit Source Files	4 Registered Source Modules (Complete Verbatim Code)
Core Languages / Stack	Python 3, LangGraph StateGraph, NumPy, SciPy, Tornado, Three.js, Canvas

CONFIDENTIALITY NOTICE & COPYRIGHT STATEMENT:
This document contains proprietary source code, algorithmic formulations, and design specifications. Copyright © 2026. All rights reserved. Unlicensed reproduction, distribution, or decompilation of this software without explicit written authorization is strictly prohibited by law.

SECTION 1: STATUTORY DESCRIPTIONS FOR COPYRIGHT FILING

When filing an application for registration of copyright of computer software with government intellectual property offices, precise, formal technical descriptions are mandatory. The following subsections contain the ready-to-copy fields required for statutory submission.

1.1 Registration Package A: Core Swarm Autonomy & Safety Engine

Field Name	Statutory Entry Content
Title of Work	Autonomous Multi-Agent UAV Swarm Fault Detection, Isolation, and Dual-Layer Safety Guardrail Control En
Class of Work	Computer Software / Literary Work
Author(s)	UAV FDI Research Team / Development Lead
Date of Creation	October 2026
Primary Files	step80_multiagent_swarm.py, step76_control_barrier_function.py
Brief Statement of Functionality	Executes a decentralized 10-node state-graph agentic processing cycle on multiple independent unmanned

1.2 Registration Package B: Unified Ground Control & Mission Dashboard

Field Name	Statutory Entry Content
Title of Work	Unified Web Ground Control and 3D/2D Multi-Agent Swarm Mission Command Interface (Mission Control v1
Class of Work	Computer Software / Graphic & Literary Work
Author(s)	UAV FDI Research Team / Development Lead
Date of Creation	October 2026
Primary Files	mission_control.py, web_gui/index.html
Brief Statement of Functionality	A full-stack asynchronous ground control interface combining a Python Tornado REST/WebSocket server wi

SECTION 2: PROPRIETARY ALGORITHMIC INVENTIONS

This section details the exact algorithmic logic and technical formulations invented and embodied in the registered source code. These four algorithms constitute the primary intellectual contribution of this project.

2.1 Algorithm 1: Two-Layer Guardrail Architecture (Semantic L1 + Kinematic L2 CBF-QP)

Problem Solved: Traditional autonomous agents either rely purely on discrete AI rules (which cannot guarantee physical safety against collisions) or purely on low-level control theory (which cannot reason about mission goals, battery life, or sensor degradation).
Our Innovation: A sequential, two-step verification mechanism where every flight decision passes through two independent filters:

- **Layer 1 (Semantic Policy Guardrail):** Validates regulatory constraints including absolute altitude ceilings (2m to 120m), maximum allowed nominal velocity (5 m/s), low-battery triggers (<15%), and enforces mission overrides such as Return-To-Home (RTL).
- **Layer 2 (Control Barrier Function QP Filter):** Takes the Layer 1-approved velocity command $\mathbf{u}_{\text{nom}} = [v_x, v_y, v_z]$ and projects it onto a forward-invariant safe set $\mathcal{C}$ by solving a Quadratic Program in real time:
  $$\min_{\mathbf{u}} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 \quad \text{s.t.} \quad A_{\text{cbf}} \mathbf{u} \le \mathbf{b}_{\text{cbf}}$$
  Simultaneously enforcing ground clearance (h_{alt}), kinetic speed envelope (h_{speed}), minimum swarm separation ($h_{\text{swarm}} \ge 2.5\text{m}$), and repulsive geofence barriers (h_{nfz}).

2.2 Algorithm 2: Dynamic Latching Tangent Bypass for Proactive NFZ Avoidance

Problem Solved: When drones approach an obstacle boundary, standard potential field or barrier algorithms often experience 'limit-cycle chattering' (rapidly oscillating back and forth along the boundary surface) or fail to detect intersections between waypoints.
Our Innovation: A predictive line-segment intersection detector coupled with a latched orthogonal tangent bypass generator:

- **Line-Segment Projection:** Projects vector segment from current UAV position to the assigned waypoint against all No-Fly Zone circles. Detects obstruction if the minimum distance to the center c is less than $(R_{\text{core}} + 10\text{m})$.
- **Orthogonal Tangent Calculation:** Determines the normal unit vector $\hat{n}$ outward from the NFZ center, derives the orthogonal tangent $\hat{t}$, and picks the tangent pointing toward the destination waypoint ($(\hat{t} \cdot \mathbf{d}_{\text{target}}) > 0$).
- **State Latching:** Places a bypass waypoint at $c + \hat{n}(R + 14\text{m}) + 10\hat{t}$ and **latches** this coordinate. The UAV tracks the latched point until it comes within a 3.0-meter radius, eliminating jitter and oscillation before smoothly resuming the primary mission path.

2.3 Algorithm 3: Fault-Disqualified Decentralized Leader Election Protocol

Problem Solved: In multi-agent swarms, if the leader experiences an in-flight sensor failure (such as GPS drift or compass degradation), it will propagate erroneous navigational cues to all follower drones, risking whole-swarm catastrophe.
Our Innovation: A decentralized consensus election protocol in `SwarmBus` with strict health pre-filtering:

- **Strict Health Exclusion:** Before computing fitness scores, any UAV with *any* detected sensor anomaly (`fault_label != 'No Failure'`) or critical battery is immediately and unconditionally excluded from candidacy.
- **Composite Fitness Evaluation:** Eligible healthy candidates are scored according to:
  $$\text{Fitness} = 0.40 \cdot \text{Batt} + 0.30 \cdot (1.0 - \text{Risk}) + 0.20 \cdot \text{GPS}_{\text{valid}} + 0.10 \cdot \text{Sensor}_{\text{health}}$$
  The highest-scoring healthy agent is democratically elected as Swarm Leader (■), guaranteeing that a degraded

drone can never command peers.

2.4 Algorithm 4: In-Flight Trajectory Splicing & Dynamic Task Handoff Protocol

Problem Solved: When an individual drone suffers a hardware failure, conventional systems simply abort its mission, leaving search grids or inspection routes incomplete.
Our Innovation: An autonomous, seamless peer-to-peer workload handoff mechanism:

- **Dynamic Handoff Request:** Upon detecting critical battery or sensor failure, the degraded UAV extracts all unvisited remaining route coordinates (`remaining_route = route_coords[current_wp_idx:]`) and broadcasts a handover request across the `SwarmBus`.
- **Optimal Peer Bidding:** Healthy peers calculate a fitness bid considering battery capacity and Euclidean proximity to the abandoned task.
- **In-Flight Trajectory Splicing:** The winning peer dynamically splices the transferred waypoints directly into its active flight route immediately after its current target (`peer.route_coords.insert(insert_pos, wp)`), expanding its task queue while the degraded UAV safely descends into Beacon Retrieval Mode.

SECTION 3: VERBATIM SOURCE CODE DEPOSIT INDEX

The following sections contain the complete, verbatim source code of the four primary project files designated for copyright deposit. Line numbers, formatting, and implementation logic are preserved for forensic intellectual property verification.

Deposit Part	Target File Path	Language	Lines	Key Modules / Classes Embodied
Part 3.1	step80_multiagent_swarm.py	Python 3	2,539	SwarmBus, SwarmAgentState, 10-Node Pipeline, Handoff
Part 3.2	step76_control_barrier_function.py	Python 3	218	UAVControlBarrierFunction, QP Solver, Multi-Barrier Form
Part 3.3	mission_control.py	Python 3	295	Tornado Web Server, REST APIs, Telemetry WebSockets
Part 3.4	web_gui/index.html	HTML5/JS/CSS	1,754	Three.js 3D Scene, 2D Canvas Radar, Zoom/Pan Engine,
Total	Complete System Core	Multi	4,806	Full Autonomous Swarm & Mission Control Architecture

Part 3.1: step80_multiagent_swarm.py (Core Swarm Autonomy & Pipeline)

Source File Path: `/home/samiksha/uav_fdi_data/step80_multiagent_swarm.py`
Character Encoding: UTF-8 | **Preservation:** Verbatim line-numbered software deposit

```
"""step80_multiagent_swarm.py  – Multi-Agent UAV Swarm with Autonomous LangGraph Agents

Real Multi-Agent Agentic AI Architecture
========================================
1. Swarm of 4 Autonomous UAV Agents (UAV-1 Alpha, UAV-2 Bravo, UAV-3 Charlie, UAV-4 Delta).
2. EACH UAV executes an independent 10-node LangGraph state graph every tick:
     [Telemetry] → [Energy Monitor] → [FDI Inference] → [Risk Assessment] →
     [Agentic Reasoning] → [Swarm Coordination] → [Guardrail L1] →
     [Guardrail L2 CBF-QP] → [Execute Action] → [Finalize]
3. SwarmBus Coordination Protocol:
     - Dynamic Heartbeat exchange.
     - Task delegation & handoff protocol: Degraded UAV broadcasts HandoffRequest;
       peers evaluate battery, health, and distance; winning peer accepts the mission.
     - Inter-agent collision avoidance via Control Barrier Functions (CBF).
4. ROS 2 Integration:
     - Publishes live UAVStateReport msgs to /uav_swarm/uav_{id}/state_report.
     - If live PX4 SITL is running, UAV-1 can bind to live PX4 Micro-XRCE-DDS topics.
5. Live Multi-Agent ANSI Dashboard:
     - Side-by-side 4-agent status cards + swarm overview + inter-agent communication log.
6. Dynamic Fault Injection:
     - CLI flags to inject battery drop, sensor failure, or swarm degradation at any tick.
"""

from __future__ import annotations

import argparse
import asyncio
import csv
import json
import os
import sys
import threading
import time
from collections import deque
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple
import numpy as np

# ── LangGraph ──────────────────────────────────────────────────────────────
try:
    from langgraph.graph import StateGraph, END, START
except ImportError:
    print("[ERROR] langgraph not installed. Run: pip install langgraph")
    sys.exit(1)

# ── Control Barrier Function (step76) ──────────────────────────────────────
try:
    from step76_control_barrier_function import UAVControlBarrierFunction
    _CBF_AVAILABLE = True
except ImportError:
    _CBF_AVAILABLE = False

# ── Sensor FDI Model (step70) ──────────────────────────────────────────────
try:
    import joblib
    _MODEL_PATH = os.path.join(os.path.dirname(__file__), "step70_clean_fdi_model.joblib")
    _FEAT_PATH  = os.path.join(os.path.dirname(__file__), "step70_feature_names.joblib")
    _FDI_MODEL     = joblib.load(_MODEL_PATH)
    _FEATURE_NAMES = joblib.load(_FEAT_PATH)
    _FDI_AVAILABLE = True
except Exception:
    _FDI_AVAILABLE = False

# ── ROS 2 Support ──────────────────────────────────────────────────────────
_ROS_AVAILABLE = False
try:
    import rclpy
    from rclpy.node import Node
    from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
    from uav_swarm_msgs.msg import UAVStateReport
    _ROS_AVAILABLE = True
except ImportError:
    _ROS_AVAILABLE = False

# ── MAVLink Multi-Vehicle Support (for QGroundControl live map) ────────────
_MAVLINK_AVAILABLE = False
try:
    from pymavlink import mavutil
    _MAVLINK_AVAILABLE = True
except ImportError:
    _MAVLINK_AVAILABLE = False


def is_px4_sitl_running() -> bool:
    """Detect whether PX4 SITL (x500 instance 0) binary is running."""
    try:
        import subprocess
        # Must match exact executable name 'px4', not substring matches like 'python3 ...'
        out = subprocess.check_output(["pgrep", "-x", "px4"], stderr=subprocess.DEVNULL)
        return len(out.strip()) > 0
    except Exception:
        return False
```

```
 93 |
 94 |
 95 | # ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 96 | # Swarm Communication Bus (Inter-Agent Messaging)
 97 | # ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 98 | @dataclass
 99 | class SwarmMessage:
100 |     msg_type:  str       # HEARTBEAT | HANDOFF_REQUEST | HANDOFF_BID | HANDOFF_ACCEPT
101 |     sender_id: str
102 |     data:      Dict[str, Any]
103 |     timestamp: float = field(default_factory=time.time)
104 |
105 | # Global No-Fly Zones (NFZ): Center (x, y), radius r, and clearance alt
106 | NFZ_ZONES = [
107 |     {"name": "NFZ-Alpha (Hospital)", "x": 50.0, "y": 60.0, "radius": 18.0, "alt_ceil": 25.0},
108 |     {"name": "NFZ-Bravo (Airport)", "x": -15.0, "y": 90.0, "radius": 22.0, "alt_ceil": 30.0},
109 | ]
110 |
111 | class SwarmBus:
112 |     """Decentralized message broker for dynamic peer-to-peer UAV swarm communications."""
113 |
114 |     def __init__(self):
115 |         self.lock = threading.RLock()
116 |         self.peer_states: Dict[str, Dict[str, Any]] = {}
117 |         self.event_log: deque = deque(maxlen=10)
118 |         self.active_faults: Dict[str, str] = {}  # Injected via QGC/CLI: { "UAV-1": "GPS Failure" }
119 |         self._ros_node = None
120 |         self._publishers: Dict[str, Any] = {}
121 |         self._mav_conns: Dict[str, Any] = {}
122 |         self._px4_onboard_conn = None
123 |         self.latest_px4_telemetry: Dict[str, Any] = {}
124 |         self._last_px4_msg_time: float = 0.0
125 |         self._qgc_listener_sock = None
126 |         self.nfz_zones = list(NFZ_ZONES)
127 |         # Task-completion events queued during a tick for immediate master reporting
128 |         self._pending_task_completions: List[Dict[str, Any]] = []
129 |         # Routes from degraded UAVs waiting to be picked up by assigned peers
130 |         self._pending_handoff_routes: Dict[str, Dict[str, Any]] = {}
131 |
132 |     def notify_task_completed(self, uav_id: str, wp_idx: int, tasks_done: int, total_tasks: int):
133 |         """Called by node_collect_telemetry when a UAV finishes a waypoint.
134 |         Queues an event so the main loop emits an immediate master report."""
135 |         with self.lock:
136 |             self._pending_task_completions.append({
137 |                 "uav_id": uav_id,
138 |                 "wp_idx": wp_idx,
139 |                 "tasks_done": tasks_done,
140 |                 "total_tasks": total_tasks,
141 |             })
142 |
143 |     def register_uav(self, uav_id: str):
144 |         """Dynamically registers a new UAV agent for horizontal scaling."""
145 |         with self.lock:
146 |             if uav_id not in self.peer_states:
147 |                 self.peer_states[uav_id] = {
148 |                     "pos": [0.0, 0.0, 10.0],
149 |                     "vel": [0.0, 0.0, 0.0],
150 |                     "battery_pct": 90.0,
151 |                     "fault": "No Failure",
152 |                     "available": True,
153 |                     "task": "Dynamic Enroute",
154 |                     "action": "NOMINAL_CONTINUE",
155 |                 }
156 |                 # Initialize MAVLink stream for QGC if available
157 |                 if _MAVLINK_AVAILABLE and uav_id not in self._mav_conns:
158 |                     try:
159 |                         sys_id = int(uav_id.split("-")[-1])
160 |                         mav = mavutil.mavlink_connection("udpout:127.0.0.1:14550", source_system=sys_id, source_com
    |     ■ ponent=1)
161 |                         self._mav_conns[uav_id] = mav
162 |                         self._send_params(mav, sys_id)
163 |                     except Exception:
164 |                         pass
165 |                 # Initialize ROS2 publisher if ROS2 is active
166 |                 if self._ros_node and uav_id not in self._publishers and _ROS_AVAILABLE:
167 |                     try:
168 |                         topic = f"/uav_swarm/{uav_id.lower().replace('-', '_')}/state_report"
169 |                         qos = QoSProfile(reliability=ReliabilityPolicy.BEST_EFFORT, depth=10)
170 |                         self._publishers[uav_id] = self._ros_node.create_publisher(UAVStateReport, topic, qos)
171 |                     except Exception:
172 |                         pass
173 |                 self.log_event(f"■ [HORIZONTAL SCALING] {uav_id} registered into active swarm mesh")
174 |
175 |     def init_ros(self):
176 |         if not _ROS_AVAILABLE:
177 |             return
178 |         try:
179 |             if not rclpy.ok():
180 |                 rclpy.init()
181 |             self._ros_node = Node("step80_swarm_bus")
182 |             qos = QoSProfile(reliability=ReliabilityPolicy.BEST_EFFORT, depth=10)
183 |             for uid in list(self.peer_states.keys()) or ("UAV-1", "UAV-2", "UAV-3", "UAV-4"):
184 |                 topic = f"/uav_swarm/{uid.lower().replace('-', '_')}/state_report"
185 |                 self._publishers[uid] = self._ros_node.create_publisher(UAVStateReport, topic, qos)
186 |         except Exception as e:
187 |             self.log_event(f"ROS2 publisher init error: {e}")
188 |
189 |     def _send_params(self, mav, sys_id: int):
190 |         """Send required MAVLink parameter responses so QGC does not timeout."""
191 |         params = [
192 |             (b"SYS_AUTOSTART\x00\x00\x00", 4001.0),
193 |             (b"MAV_TYPE\x00\x00\x00\x00\x00\x00\x00\x00", 2.0),
194 |             (b"COM_RC_IN_MODE\x00\x00", 1.0),
195 |             (b"NAV_RCL_ACT\x00\x00\x00\x00\x00", 2.0),
196 |             (b"SYS_ID\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00", float(sys_id)),
197 |         ]
```

```
        for idx, (p_id, val) in enumerate(params):
            try:
                mav.mav.param_value_send(
                    p_id,
                    float(val),
                    mavutil.mavlink.MAV_PARAM_TYPE_REAL32,
                    len(params),
                    idx
                )
            except Exception:
                pass

    def init_mavlink(self, target_ip: str = "127.0.0.1", target_port: int = 14550):
        """Initializes MAVLink UDP broadcast streams for QGroundControl."""
        if not _MAVLINK_AVAILABLE:
            return
        try:
            sitl_active = is_px4_sitl_running()
            for i, uid in enumerate(("UAV-1", "UAV-2", "UAV-3", "UAV-4"), start=1):
                if sitl_active and uid == "UAV-1":
                    continue
                mav = mavutil.mavlink_connection(
                    f"udpout:{target_ip}:{target_port}",
                    source_system=i,
                    source_component=1
                )
                self._mav_conns[uid] = mav
                self._send_params(mav, i)
                for m_type in (
                    mavutil.mavlink.MAV_MISSION_TYPE_MISSION,
                    mavutil.mavlink.MAV_MISSION_TYPE_FENCE,
                    mavutil.mavlink.MAV_MISSION_TYPE_RALLY,
                ):
                    try:
                        mav.mav.mission_count_send(255, 0, 0, m_type)
                    except Exception:
                        pass

            # If PX4 SITL is running, also connect to its onboard MAVLink port (14540)
            # so the SwarmBus can listen directly to PX4 STATUSTEXT and failure events
            if sitl_active:
                try:
                    self._px4_onboard_conn = mavutil.mavlink_connection("udpin:127.0.0.1:14540")
                    # Send a heartbeat back to PX4 so it knows a client is listening on 14540
                    self._px4_onboard_conn.mav.heartbeat_send(
                        mavutil.mavlink.MAV_TYPE_ONBOARD_CONTROLLER,
                        mavutil.mavlink.MAV_AUTOPILOT_INVALID,
                        0, 0, 0
                    )
                    self.log_event("■ Connected to live PX4 onboard MAVLink on port 14540")
                except Exception as e:
                    self.log_event(f"PX4 onboard bind warning: {e}")

            mode_str = "UAV-2..4 (UAV-1 via PX4 SITL)" if sitl_active else "UAV-1..4 (Standalone)"
            self.log_event(f"■ QGC MAVLink connected on UDP {target_port} for {mode_str}")
        except Exception as e:
            self.log_event(f"MAVLink init error: {e}")

    def is_connected_to_px4(self) -> bool:
        """Returns True if actively receiving telemetry from PX4 SITL + Gazebo."""
        return bool(self._px4_onboard_conn and (time.time() - self._last_px4_msg_time < 3.0))

    def poll_px4_telemetry(self) -> Dict[str, Any]:
        """Polls live PX4 onboard stream for failure injections and telemetry."""
        result: Dict[str, Any] = {}

        # Dynamically connect to PX4 onboard port 14540 if PX4 starts up while swarm is running
        if not self._px4_onboard_conn and is_px4_sitl_running() and _MAVLINK_AVAILABLE:
            try:
                self._px4_onboard_conn = mavutil.mavlink_connection("udpin:127.0.0.1:14540")
                self._px4_onboard_conn.mav.heartbeat_send(
                    mavutil.mavlink.MAV_TYPE_ONBOARD_CONTROLLER,
                    mavutil.mavlink.MAV_AUTOPILOT_INVALID,
                    0, 0, 0
                )
                self.log_event("■ Connected to live PX4 SITL/Gazebo onboard MAVLink on port 14540")
            except Exception:
                pass

        if not self._px4_onboard_conn:
            return result

        try:
            # Send periodic heartbeat so PX4 keeps the stream active
            if not hasattr(self, "_last_onboard_hb") or time.time() - self._last_onboard_hb > 1.0:
                self._last_onboard_hb = time.time()
                try:
                    self._px4_onboard_conn.mav.heartbeat_send(
                        mavutil.mavlink.MAV_TYPE_ONBOARD_CONTROLLER,
                        mavutil.mavlink.MAV_AUTOPILOT_INVALID,
                        0, 0, 0
                    )
                except Exception:
                    pass

            while True:
                msg = self._px4_onboard_conn.recv_match(blocking=False)
                if not msg:
                    break
                self._last_px4_msg_time = time.time()
                mtype = msg.get_type()

                if mtype == "STATUSTEXT":
                    txt = getattr(msg, "text", "")
                    if isinstance(txt, bytes):
                        txt = txt.decode("utf-8", errors="ignore")
```

```
                    txt_lower = txt.lower()
                    if any(w in txt_lower for w in ("warn", "error", "fail", "inject")):
                        self.log_event(f"■ PX4: {txt[:40]}")
                    if "gyro" in txt_lower or "unit: gyro" in txt_lower or "unit: (0)" in txt_lower:
                        fault = "Gyroscope Failure" if not any(ok in txt_lower for ok in ("ok", "recovered", "clear
■ ed")) else "No Failure"
                        with self.lock:
                            if fault == "No Failure":
                                self.active_faults.pop("UAV-1", None)
                            else:
                                self.active_faults["UAV-1"] = "Gyroscope Failure"
                        self.log_event(f"■ PX4 reported: {fault} on UAV-1")
                    elif "accel" in txt_lower or "unit: accel" in txt_lower:
                        fault = "Accelerometer Failure" if not any(ok in txt_lower for ok in ("ok", "recovered", "c
■ leared")) else "No Failure"
                        with self.lock:
                            if fault == "No Failure":
                                self.active_faults.pop("UAV-1", None)
                            else:
                                self.active_faults["UAV-1"] = "Accelerometer Failure"
                        self.log_event(f"■ PX4 reported: {fault} on UAV-1")
                    elif "gps" in txt_lower or "navigation error" in txt_lower or "no valid position" in txt_lower
■ or "blind land" in txt_lower:
                        fault = "GPS Failure" if not any(ok in txt_lower for ok in ("ok", "recovered", "cleared"))
■ else "No Failure"
                        with self.lock:
                            if fault == "No Failure":
                                self.active_faults.pop("UAV-1", None)
                            else:
                                self.active_faults["UAV-1"] = "GPS Failure"
                        self.log_event(f"■ PX4 reported: {fault} on UAV-1")
                    elif "mag" in txt_lower or "compass" in txt_lower:
                        fault = "Compass Failure" if not any(ok in txt_lower for ok in ("ok", "recovered", "cleared
■ ")) else "No Failure"
                        with self.lock:
                            if fault == "No Failure":
                                self.active_faults.pop("UAV-1", None)
                            else:
                                self.active_faults["UAV-1"] = "Compass Failure"
                        self.log_event(f"■ PX4 reported: {fault} on UAV-1")
                    elif "baro" in txt_lower:
                        fault = "Barometer Failure" if not any(ok in txt_lower for ok in ("ok", "recovered", "clear
■ ed")) else "No Failure"
                        with self.lock:
                            if fault == "No Failure":
                                self.active_faults.pop("UAV-1", None)
                            else:
                                self.active_faults["UAV-1"] = "Barometer Failure"
                        self.log_event(f"■ PX4 reported: {fault} on UAV-1")

                elif mtype == "SYS_STATUS":
                    if hasattr(msg, "battery_remaining") and msg.battery_remaining != -1:
                        batt = float(msg.battery_remaining)
                        result["battery_pct"] = batt
                        self.latest_px4_telemetry["battery_pct"] = batt
                    present = getattr(msg, "onboard_control_sensors_present", 0)
                    enabled = getattr(msg, "onboard_control_sensors_enabled", 0)
                    health = getattr(msg, "onboard_control_sensors_health", 0)

                    # When a sensor is failed/disabled in PX4, its health OR enabled bit is cleared
                    if present:
                        # GPS bit = 32 (1<<5)
                        if (present & 32) and (not (health & 32) or not (enabled & 32)):
                            with self.lock:
                                self.active_faults["UAV-1"] = "GPS Failure"
                            self.log_event("■ PX4 SYS_STATUS: GPS Failure (disabled/unhealthy) on UAV-1")
                        elif (present & 32) and (health & 32) and (enabled & 32) and self.active_faults.get("UAV-1"
■ ) == "GPS Failure":
                            with self.lock:
                                self.active_faults.pop("UAV-1", None)
                            self.log_event("■ PX4 SYS_STATUS: GPS recovered to nominal on UAV-1")

                        # 3D Gyro bit = 1 (1<<0)
                        if (present & 1) and (not (health & 1) or not (enabled & 1)):
                            with self.lock:
                                self.active_faults["UAV-1"] = "Gyroscope Failure"
                            self.log_event("■ PX4 SYS_STATUS: Gyroscope Failure on UAV-1")
                        elif (present & 1) and (health & 1) and (enabled & 1) and self.active_faults.get("UAV-1") =
■ = "Gyroscope Failure":
                            with self.lock:
                                self.active_faults.pop("UAV-1", None)

                        # 3D Accel bit = 2 (1<<1)
                        if (present & 2) and (not (health & 2) or not (enabled & 2)):
                            with self.lock:
                                self.active_faults["UAV-1"] = "Accelerometer Failure"
                            self.log_event("■ PX4 SYS_STATUS: Accelerometer Failure on UAV-1")
                        elif (present & 2) and (health & 2) and (enabled & 2) and self.active_faults.get("UAV-1") =
■ = "Accelerometer Failure":
                            with self.lock:
                                self.active_faults.pop("UAV-1", None)

                        # 3D Mag bit = 4 (1<<2)
                        if (present & 4) and (not (health & 4) or not (enabled & 4)):
                            with self.lock:
                                self.active_faults["UAV-1"] = "Compass Failure"
                            self.log_event("■ PX4 SYS_STATUS: Compass Failure on UAV-1")
                        elif (present & 4) and (health & 4) and (enabled & 4) and self.active_faults.get("UAV-1") =
■ = "Compass Failure":
                            with self.lock:
                                self.active_faults.pop("UAV-1", None)

                        # Barometer bit = 8 (1<<3)
                        if (present & 8) and (not (health & 8) or not (enabled & 8)):
                            with self.lock:
                                self.active_faults["UAV-1"] = "Barometer Failure"
```

```
                            self.log_event("■ PX4 SYS_STATUS: Barometer Failure on UAV-1")
                        elif (present & 8) and (health & 8) and (enabled & 8) and self.active_faults.get("UAV-1") =
    ■ = "Barometer Failure":
                            with self.lock:
                                self.active_faults.pop("UAV-1", None)

                elif mtype == "GPS_RAW_INT":
                    fix_type = getattr(msg, "fix_type", 3)
                    satellites = getattr(msg, "satellites_visible", 10)
                    if fix_type < 2 or satellites < 4:
                        with self.lock:
                            if self.active_faults.get("UAV-1") != "GPS Failure":
                                self.active_faults["UAV-1"] = "GPS Failure"
                                self.log_event(f"■ PX4 GPS_RAW_INT: GPS fix lost (fix_type={fix_type}, sats={satell
    ■ ites}) on UAV-1")
                    elif fix_type >= 3 and satellites >= 6 and self.active_faults.get("UAV-1") == "GPS Failure":
                        with self.lock:
                            self.active_faults.pop("UAV-1", None)
                        self.log_event("■ PX4 GPS_RAW_INT: GPS 3D fix restored on UAV-1")

                elif mtype == "COMMAND_LONG" and getattr(msg, "command", 0) == 420:
                    unit = int(getattr(msg, "param1", -1))
                    ftype = int(getattr(msg, "param2", -1))
                    unit_map = {1: "Gyroscope Failure", 2: "Accelerometer Failure", 3: "Compass Failure", 4: "Barom
    ■ eter Failure", 5: "GPS Failure"}
                    with self.lock:
                        if ftype == 0:
                            self.active_faults.pop("UAV-1", None)
                            self.log_event("■ PX4 VEHICLE_CMD_INJECT_FAILURE: Fault cleared on UAV-1")
                        else:
                            fname = unit_map.get(unit, "Sensor Fault")
                            self.active_faults["UAV-1"] = fname
                            self.log_event(f"■ PX4 VEHICLE_CMD_INJECT_FAILURE: {fname} injected on UAV-1")

                elif mtype == "GLOBAL_POSITION_INT":
                    lat = float(msg.lat) / 1e7
                    lon = float(msg.lon) / 1e7
                    result["pos_z"] = max(0.0, float(msg.relative_alt) / 1000.0)
                    result["vx"] = float(msg.vx) / 100.0
                    result["vy"] = float(msg.vy) / 100.0
                    result["vz"] = float(msg.vz) / 100.0
                    result["pos_x"] = (lon - 8.545594) * 111320.0 * 0.6769
                    result["pos_y"] = (lat - 47.397742) * 111320.0
                    self.latest_px4_telemetry.update(result)

                elif mtype == "LOCAL_POSITION_NED":
                    result["pos_x"] = float(msg.x)
                    result["pos_y"] = float(msg.y)
                    result["pos_z"] = max(0.0, -float(msg.z))
                    result["vx"] = float(msg.vx)
                    result["vy"] = float(msg.vy)
                    result["vz"] = -float(msg.vz)
                    self.latest_px4_telemetry.update(result)
            except Exception:
                pass
            return result

        def log_event(self, text: str):
            with self.lock:
                ts = time.strftime("%H:%M:%S")
                self.event_log.append(f"[{ts}] {text}")

        def broadcast_heartbeat(self, uav_id: str, state_summary: Dict[str, Any]):
            with self.lock:
                self.peer_states[uav_id] = state_summary

            # Publish to ROS 2 topic if active
            if self._ros_node and uav_id in self._publishers:
                try:
                    msg = UAVStateReport()
                    msg.uav_id = int(uav_id.split("-")[-1])
                    pos = state_summary.get("pos", [0.0, 0.0, 0.0])
                    msg.position = [float(pos[0]), float(pos[1]), float(pos[2])]
                    vel = state_summary.get("vel", [0.0, 0.0, 0.0])
                    msg.velocity = [float(vel[0]), float(vel[1]), float(vel[2])]
                    msg.battery = float(state_summary.get("battery_pct", 100.0))
                    msg.fault_status = str(state_summary.get("fault", "No Failure"))
                    msg.current_task = str(state_summary.get("task", "NOMINAL"))
                    msg.task_status = str(state_summary.get("action", "NOMINAL_CONTINUE"))
                    self._publishers[uav_id].publish(msg)
                except Exception:
                    pass

            # Publish to QGroundControl via MAVLink
            if uav_id in self._mav_conns:
                try:
                    mav = self._mav_conns[uav_id]
                    sys_id = int(uav_id.split("-")[-1])

                    # Check and reply to QGC incoming requests (parameters, capabilities, mission/rally)
                    try:
                        while True:
                            req = mav.recv_match(blocking=False)
                            if not req:
                                break
                            rtype = req.get_type()
                            if rtype in ("PARAM_REQUEST_LIST", "PARAM_REQUEST_READ"):
                                self._send_params(mav, sys_id)
                            elif rtype == "MISSION_REQUEST_LIST":
                                m_type = getattr(req, "mission_type", 0)
                                mav.mav.mission_count_send(
                                    req.get_srcSystem() or 255,
                                    req.get_srcComponent() or 0,
                                    0,
                                    m_type
                                )
```

```
                        elif rtype == "COMMAND_LONG":
                            mav.mav.command_ack_send(req.command, mavutil.mavlink.MAV_RESULT_ACCEPTED)
                            if req.command == 420:  # MAV_CMD_INJECT_FAILURE
                                unit = int(getattr(req, "param1", -1))
                                failure_type = int(getattr(req, "param2", -1))
                                fault_map = {
                                    4: "GPS Failure",
                                    0: "Gyroscope Failure",
                                    1: "Accelerometer Failure",
                                    2: "Compass Failure",
                                    3: "Barometer Failure",
                                }
                                if failure_type == 0:  # FAILURE_TYPE_OK (clear fault)
                                    with self.lock:
                                        self.active_faults.pop(uav_id, None)
                                    self.log_event(f"■ QGC cleared fault on {uav_id}")
                                    try:
                                        mav.mav.statustext_send(mavutil.mavlink.MAV_SEVERITY_INFO, f"■ [{uav_id}] F
    ■ ault Cleared".encode()[:50])
                                    except Exception:
                                        pass
                                else:
                                    fname = fault_map.get(unit, "Sensor Fault")
                                    with self.lock:
                                        self.active_faults[uav_id] = fname
                                    self.log_event(f"■ QGC injected {fname} on {uav_id}")
                                    try:
                                        mav.mav.statustext_send(mavutil.mavlink.MAV_SEVERITY_WARNING, f"■ [{uav_id}
    ■ ] {fname} Injected".encode()[:50])
                                    except Exception:
                                        pass

                        elif rtype == "SERIAL_CONTROL":
                            # Handles QGroundControl MAVLink Console commands (e.g. 'failure gps off')
                            raw_data = bytes(req.data[:req.count])
                            cmd_str = raw_data.decode("utf-8", errors="ignore").strip()
                            cmd_lower = cmd_str.lower()

                            def _reply_console(txt: str):
                                full = f"\r\n[SWARM] {txt}\r\n"
                                b = full.encode("utf-8")
                                for k in range(0, len(b), 68):
                                    chunk = b[k:k+68]
                                    d_list = list(chunk) + [0] * (70 - len(chunk))
                                    try:
                                        mav.mav.serial_control_send(
                                            mavutil.mavlink.SERIAL_CONTROL_DEV_SHELL,
                                            mavutil.mavlink.SERIAL_CONTROL_FLAG_REPLY,
                                            0, 0,
                                            len(chunk),
                                            d_list
                                        )
                                    except Exception:
                                        pass

                            if not cmd_lower:
                                _reply_console(f"{uav_id} shell ready. Type 'failure help' for FDI commands.")
                            elif "help" in cmd_lower:
                                _reply_console("Commands:\r\n  failure gps off    - Inject GPS failure\r\n  failure
    ■  gyro off   - Inject Gyro failure\r\n  failure accel off  - Inject Accel failure\r\n  failure ok         - Clear all faults\r\n  failure list
                            elif "list" in cmd_lower or "status" in cmd_lower:
                                with self.lock:
                                    flist = ", ".join([f"{k}: {v}" for k, v in self.active_faults.items()]) or "All
    ■  systems nominal (No active faults)"
                                _reply_console(f"Active Faults: {flist}")
                            elif "failure" in cmd_lower or "fault" in cmd_lower:
                                target_uid = uav_id
                                tokens = cmd_lower.split()
                                for tok in tokens:
                                    if tok.startswith("uav-") or (tok.startswith("uav") and len(tok) >= 4):
                                        target_uid = tok.upper()
                                        if not target_uid.startswith("UAV-"):
                                            target_uid = f"UAV-{target_uid[3:]}"

                                is_clear_cmd = any(w in cmd_lower for w in ("ok", "clear", "0", "recover", "reset",
    ■  "none"))
                                fault_map = {
                                    "gps": "GPS Failure",
                                    "gyro": "Gyroscope Failure",
                                    "gyroscope": "Gyroscope Failure",
                                    "accel": "Accelerometer Failure",
                                    "accelerometer": "Accelerometer Failure",
                                    "baro": "Barometer Failure",
                                    "barometer": "Barometer Failure",
                                    "compass": "Compass Failure",
                                    "mag": "Compass Failure",
                                }
                                sensor_hit = None
                                for k_s, f_val in fault_map.items():
                                    if k_s in cmd_lower:
                                        sensor_hit = f_val
                                        break

                                if is_clear_cmd or "clear" in cmd_lower:
                                    with self.lock:
                                        self.active_faults.pop(target_uid, None)
                                    self.log_event(f"■ QGC MAVLink Console cleared fault on {target_uid}")
                                    _reply_console(f"SUCCESS: Fault cleared on {target_uid}. All sensors nominal.")
                                elif sensor_hit:
                                    with self.lock:
                                        self.active_faults[target_uid] = sensor_hit
                                    self.log_event(f"■ QGC MAVLink Console injected {sensor_hit} on {target_uid}")
                                    _reply_console(f"SUCCESS: Injected {sensor_hit} on {target_uid}! Autonomous saf
    ■ e action engaged.")
                                else:
                                    _reply_console(f"Unrecognized sensor in '{cmd_str}'. Try 'failure gps off' or '
```

```
    |     ■ failure help'.")
603 |                             else:
604 |                                 _reply_console(f"Received '{cmd_str}' on {uav_id}. Type 'failure help' for fault in
    |     ■ jection commands.")
605 |                 except Exception:
606 |                     pass
607 |
608 |                 now_ms = int(time.time() * 1000) & 0xFFFFFFFF
609 |                 now_us = int(time.time() * 1e6) & 0xFFFFFFFFFFFFFFFF
610 |                 pos = state_summary.get("pos", [0.0, 0.0, 0.0])
611 |                 vel = state_summary.get("vel", [0.0, 0.0, 0.0])
612 |                 batt_pct = int(max(0.0, min(100.0, state_summary.get("battery_pct", 100.0))))
613 |                 is_ground = pos[2] < 0.2
614 |
615 |                 is_leader = bool(state_summary.get("is_leader", False))
616 |                 # Distinct MAV_TYPE so QGC visually distinguishes the Master/Leader UAV on map
617 |                 veh_type = mavutil.mavlink.MAV_TYPE_HEXAROTOR if is_leader else mavutil.mavlink.MAV_TYPE_QUADROTOR
618 |
619 |                 # PX4 Custom Flight Mode encoding: (main_mode << 16) | (sub_mode << 24)
620 |                 # Main modes: 1=MANUAL, 2=ALTCTL, 3=POSCTL, 4=AUTO, 6=OFFBOARD
621 |                 # Sub modes for AUTO: 1=READY, 2=TAKEOFF, 3=LOITER, 4=MISSION, 5=RTL, 6=LAND
622 |                 act = state_summary.get("action", "NOMINAL_CONTINUE")
623 |                 if is_ground:
624 |                     custom_mode = (3 << 16)  # POSCTL (Position Hold / Standby)
625 |                 elif act == "RETURN_TO_HOME":
626 |                     custom_mode = (4 << 16) | (5 << 24)  # AUTO RTL
627 |                 elif act == "FAILSAFE_LAND":
628 |                     custom_mode = (4 << 16) | (6 << 24)  # AUTO LAND
629 |                 elif act in ("ALTITUDE_HOLD_HOVER", "STANDBY_LOITER", "NFZ_REROUTE"):
630 |                     custom_mode = (4 << 16) | (3 << 24)  # AUTO LOITER / HOLD
631 |                 else:
632 |                     custom_mode = (3 << 16)              # POSCTL (Position Hold / Mission)
633 |
634 |                 # 1. Heartbeat
635 |                 base_mode = mavutil.mavlink.MAV_MODE_FLAG_CUSTOM_MODE_ENABLED
636 |                 if not is_ground:
637 |                     base_mode |= mavutil.mavlink.MAV_MODE_FLAG_SAFETY_ARMED
638 |                 mav.mav.heartbeat_send(
639 |                     veh_type,
640 |                     mavutil.mavlink.MAV_AUTOPILOT_PX4,
641 |                     base_mode,
642 |                     custom_mode,
643 |                     mavutil.mavlink.MAV_STATE_ACTIVE if not is_ground else mavutil.mavlink.MAV_STATE_STANDBY
644 |                 )
645 |
646 |                 # Periodically send Leader / Master banner to QGroundControl
647 |                 last_notice = getattr(self, f"_last_notice_{uav_id}", 0)
648 |                 if (time.time() - last_notice) > 6.0:
649 |                     setattr(self, f"_last_notice_{uav_id}", time.time())
650 |                     try:
651 |                         role_tag = f"■ [MASTER] {uav_id} LEADER" if is_leader else f"[{uav_id}] {state_summary.get(
    |     ■ 'task', 'SWARM')[:16]}"
652 |                         mav.mav.statustext_send(
653 |                             mavutil.mavlink.MAV_SEVERITY_NOTICE if is_leader else mavutil.mavlink.MAV_SEVERITY_INFO
    |     ■ ,
654 |                             role_tag.encode("utf-8")[:50]
655 |                         )
656 |                     except Exception:
657 |                         pass
658 |
659 |                 # 2. Battery & System Status
660 |                 v_mv = int(14000 + batt_pct * 25)
661 |                 mav.mav.sys_status_send(0, 0, 0, 250, v_mv, 1500, batt_pct, 0, 0, 0, 0, 0, 0)
662 |
663 |                 # 3. GPS Coordinates & Altitude (Zurich PX4 Origin)
664 |                 lat0 = 47.397742
665 |                 lon0 = 8.545594
666 |                 d_lat = pos[1] / 111320.0
667 |                 d_lon = pos[0] / (111320.0 * 0.6769)
668 |                 cur_lat = lat0 + d_lat
669 |                 cur_lon = lon0 + d_lon
670 |                 alt_m = max(0.0, pos[2])
671 |                 hdg_deg = float(np.degrees(np.arctan2(vel[1], vel[0])) % 360) if (abs(vel[0]) > 0.01 or abs(vel[1])
    |     ■  > 0.01) else 0.0
672 |
673 |                 mav.mav.global_position_int_send(
674 |                     now_ms,
675 |                     int(cur_lat * 1e7),
676 |                     int(cur_lon * 1e7),
677 |                     int((488.0 + alt_m) * 1000),
678 |                     int(alt_m * 1000),
679 |                     int(vel[0] * 100),
680 |                     int(vel[1] * 100),
681 |                     int(vel[2] * 100),
682 |                     int(hdg_deg * 100)
683 |                 )
684 |
685 |                 # 4. Attitude
686 |                 mav.mav.attitude_send(
687 |                     now_ms, 0.0, 0.0, float(np.radians(hdg_deg)), 0.0, 0.0, 0.0
688 |                 )
689 |             except Exception:
690 |                 pass
691 |
692 |     def request_handoff(self, sender_id: str, waypoints: List[str], reason: str) -> Optional[str]:
693 |         """Solicits peers to take over a degraded UAV's mission."""
694 |         with self.lock:
695 |             candidates = []
696 |             sender_pos = self.peer_states.get(sender_id, {}).get("pos", [0.0, 0.0, 0.0])
697 |
698 |             for uid, info in self.peer_states.items():
699 |                 if uid == sender_id:
700 |                     continue
701 |                 batt = info.get("battery_pct", 0.0)
702 |                 fault = info.get("fault", "No Failure")
703 |                 avail = info.get("available", True)
```

```
                pos = info.get("pos", [0.0, 0.0, 0.0])

                if avail and batt > 35.0 and fault == "No Failure":
                    dist = ((pos[0]-sender_pos[0])**2 + (pos[1]-sender_pos[1])**2)**0.5
                    # Score: Higher battery, closer proximity is best
                    score = batt - (dist * 0.4)
                    candidates.append((uid, score, batt, dist))

            if not candidates:
                self.log_event(f"■■ {sender_id} requested handoff: NO PEERS AVAILABLE!")
                return None

            candidates.sort(key=lambda x: x[1], reverse=True)
            winner_id, score, win_batt, win_dist = candidates[0]
            # Mark assignee as busy
            if winner_id in self.peer_states:
                self.peer_states[winner_id]["available"] = False
                self.peer_states[winner_id]["delegated_from"] = sender_id

            self.log_event(f"■ {sender_id} handed off {len(waypoints)} waypoints → {winner_id} (batt {win_batt:.0f}
    ■ %, dist {win_dist:.1f}m)")
            return winner_id

    def evaluate_leader_election(self, agents: List[SwarmAgentState]) -> str:
        """
        Distributed Swarm Leader Election using fitness scoring.
        If current leader is degraded or in emergency landing, triggers re-election.
        """
        with self.lock:
            if not hasattr(self, "current_leader") or not self.current_leader:
                self.current_leader = "UAV-1"

            curr = next((a for a in agents if a.uav_id == self.current_leader), None)
            leader_healthy = (
                curr is not None
                and not curr.on_ground
                and curr.energy_alert == "NORMAL"
                and curr.fault_label == "No Failure"
                and curr.final_action not in ("FAILSAFE_LAND", "RETURN_TO_HOME")
                and curr.battery_pct > 25.0
            )

            if leader_healthy:
                return self.current_leader

            # Current leader is degraded! Trigger election among active candidates
            old_leader = self.current_leader
            candidates = []
            for a in agents:
                if a.on_ground or a.final_action in ("FAILSAFE_LAND", "RETURN_TO_HOME"):
                    continue
                # ANY fault disqualifies from leadership — a faulty UAV must never lead
                if a.fault_label != "No Failure":
                    continue
                if a.battery_pct <= 20.0:
                    continue

                # Leadership fitness score: battery (40%), low risk (30%), GPS (20%), clean sensors (10%)
                f_batt = a.battery_pct / 100.0
                f_risk = 1.0 - a.composite_risk
                f_gps  = 1.0 if a.gps_valid else 0.0
                f_sens = 1.0 if a.fault_label == "No Failure" else 0.5
                fitness = round(0.4 * f_batt + 0.3 * f_risk + 0.2 * f_gps + 0.1 * f_sens, 3)
                candidates.append((a.uav_id, fitness, a.battery_pct))

            if not candidates:
                self.log_event(f"■■ [LEADER ELECTION] All UAVs degraded! Swarm decentralized.")
                return ""

            # Pick highest fitness candidate
            candidates.sort(key=lambda x: (x[1], x[2]), reverse=True)
            new_leader, win_fitness, win_batt = candidates[0]

            if new_leader != old_leader:
                self.current_leader = new_leader
                self.log_event(
                    f"■ [LEADER ELECTION] {old_leader} degraded → {new_leader} elected SWARM LEADER (Fitness: {win_
    ■ fitness:.2f}, Batt: {win_batt:.0f}%)"
                )
                # Update agent roles
                for a in agents:
                    if a.uav_id == new_leader:
                        a.is_leader = True
                        a.role = "■ Swarm Leader"
                    elif a.uav_id == old_leader:
                        a.is_leader = False
                        a.role = "Degraded (Ex-Leader)"
                    else:
                        a.is_leader = False

            return self.current_leader

    def get_peer_info(self, uav_id: str) -> List[Dict[str, Any]]:
        with self.lock:
            return [info for uid, info in self.peer_states.items() if uid != uav_id]

    def _get_gazebo_world(self) -> str:
        """Discovers active Gazebo world name or defaults to 'default'."""
        if getattr(self, "_gz_world", None):
            return self._gz_world
        try:
            import subprocess
            top_out = subprocess.check_output(["gz", "topic", "-l"], stderr=subprocess.DEVNULL, timeout=2).decode("
    ■ utf-8")
            for line in top_out.splitlines():
                if line.startswith("/world/") and line.endswith("/clock"):
```

```
                    self._gz_world = line.split("/")[2]
                    return self._gz_world
        except Exception:
            pass
        self._gz_world = "default"
        return self._gz_world

    def ensure_gazebo_swarm_models(self):
        """Automatically checks Gazebo and spawns x500_1, x500_2, x500_3 if missing.
        Does NOT require PX4 SITL detection — it probes Gazebo directly via gz CLI."""
        try:
            import subprocess
            out = subprocess.check_output(["gz", "model", "--list"], stderr=subprocess.DEVNULL, timeout=3).decode("
    ■ utf-8")
        except Exception:
            return  # Gazebo not reachable — skip silently

        sdf_path = "/home/samiksha/PX4-Autopilot/Tools/simulation/gz/models/x500/model.sdf"
        if not os.path.exists(sdf_path):
            return

        world = self._get_gazebo_world()
        spawn_targets = [
            ("x500_1", 2.0,  1.5,  0.2),
            ("x500_2", -2.0, 2.5,  0.2),
            ("x500_3",  1.0, -1.5, 0.2),
        ]
        for m_name, sx, sy, sz in spawn_targets:
            if m_name not in out:
                cmd = [
                    "gz", "service", "-s", f"/world/{world}/create",
                    "--reqtype", "gz.msgs.EntityFactory",
                    "--reptype", "gz.msgs.Boolean",
                    "--timeout", "3000",
                    "--req",
                    f'name: "{m_name}", allow_renaming: false, '
                    f'sdf: "<sdf version=\\"1.6\\"><include><uri>file://{sdf_path}</uri>'
                    f'<pose>{sx} {sy} {sz} 0 0 0</pose></include></sdf>"'
                ]
                try:
                    subprocess.run(cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, timeout=5)
                    self.log_event(f"■ Auto-spawned {m_name} into Gazebo (pos {sx},{sy})")
                except Exception:
                    pass

    def sync_gazebo_swarm(self, agents: List["SwarmAgentState"]):
        """Synchronizes 3D poses of peer UAVs (x500_1, x500_2, x500_3) in Gazebo.
        Works regardless of PX4 SITL process detection — probes Gazebo directly."""
        gz_map = {"UAV-2": "x500_1", "UAV-3": "x500_2", "UAV-4": "x500_3"}
        uav1 = next((a for a in agents if a.uav_id == "UAV-1"), None)
        uav1_flying = bool(uav1 is not None and uav1.altitude > 0.5)
        world = self._get_gazebo_world()

        for a in agents:
            if a.uav_id in gz_map:
                m_name = gz_map[a.uav_id]
                alt = a.altitude if uav1_flying else 0.2
                cmd = [
                    "gz", "service", "-s", f"/world/{world}/set_pose",
                    "--reqtype", "gz.msgs.Pose",
                    "--reptype", "gz.msgs.Boolean",
                    "--timeout", "150",
                    "--req", f'name: "{m_name}", position: {{x: {a.pos_x:.2f}, y: {a.pos_y:.2f}, z: {alt:.2f}}}'
                ]
                try:
                    subprocess.Popen(cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
                except Exception:
                    pass


_SWARM_BUS = SwarmBus()


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Individual UAV State Dataclass
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@dataclass
class SwarmAgentState:
    uav_id:           str   = "UAV-1"
    role:             str   = "Primary Survey"
    is_leader:        bool  = False
    active_task:      str   = "Survey Sector Alpha"
    waypoints_queue:  List[str] = field(default_factory=lambda: ["WP1", "WP2", "WP3", "WP4"])
    available:        bool  = True

    # Realistic Routes & Tasks Navigation
    route_coords:     List[Tuple[float, float, float]] = field(default_factory=list)
    current_wp_idx:   int   = 0
    tasks_completed:  int   = 0
    total_tasks:      int   = 0

    # Inter-Drone Mesh Network & Link Quality
    link_quality_pct: float = 100.0
    comms_latency_ms: float = 2.0

    # Telemetry
    pos_x:            float = 0.0
    pos_y:            float = 0.0
    altitude:         float = 10.0
    vx:               float = 0.0
    vy:               float = 0.0
    vz:               float = 0.0
    battery_pct:      float = 95.0
    gps_valid:        bool  = True
    on_ground:        bool  = False

```

```
    # FDI
    fault_label:      str   = "No Failure"
    confidence:       float = 98.0

    # Energy Alert
    energy_alert:     str   = "NORMAL"   # NORMAL | LOW | CRITICAL

    # Risk
    risk_stability:   float = 0.0
    risk_navigation:  float = 0.0
    risk_energy:      float = 0.0
    composite_risk:   float = 0.0

    # Agentic Reasoning
    agent_proposal:   str   = "NOMINAL_CONTINUE"
    reasoning_trace:  str   = "All systems nominal."

    # Swarm Handoff
    delegation_target: str  = ""
    delegation_status: str  = "N/A"

    # Guardrails
    l1_status:        str   = "PASSED"
    l1_notes:         str   = "—"
    l2_status:        str   = "PASSED"
    l2_notes:         str   = "—"

    # No-Fly Zone (NFZ) Geofencing
    nfz_violation:    bool  = False
    nfz_nearest:      str   = "—"
    nfz_dist:         float = 999.0

    # Final Action
    final_action:     str   = "NOMINAL_CONTINUE"
    hover_ticks:      int   = 0
    beacon_active:    bool  = False
    reroute_active:   bool  = False
    reroute_target:   List[float] = field(default_factory=list)
    log:              List[str] = field(default_factory=list)


# ═══════════════════════════════════════════════════════════════════════════
# 10-Node LangGraph Agent Pipeline (Instantiated per UAV)
# ═══════════════════════════════════════════════════════════════════════════

def node_collect_telemetry(state: SwarmAgentState) -> SwarmAgentState:
    # If UAV-1 and live PX4/Gazebo telemetry is streaming, take physical readings directly
    if state.uav_id == "UAV-1" and _SWARM_BUS.is_connected_to_px4() and _SWARM_BUS.latest_px4_telemetry:
        tel = _SWARM_BUS.latest_px4_telemetry
        if "pos_x" in tel:
            state.pos_x = float(tel["pos_x"])
        if "pos_y" in tel:
            state.pos_y = float(tel["pos_y"])
        if "pos_z" in tel:
            state.altitude = float(tel["pos_z"])
        if "vx" in tel:
            state.vx = float(tel["vx"])
        if "vy" in tel:
            state.vy = float(tel["vy"])
        if "vz" in tel:
            state.vz = float(tel["vz"])
        if "battery_pct" in tel:
            state.battery_pct = float(tel["battery_pct"])
    else:
        # Realistic Waypoint Navigation & Movement Simulation
        if state.on_ground:
            state.vx = 0.0
            state.vy = 0.0
            state.vz = 0.0
            state.altitude = 0.0
        elif state.final_action in ("NOMINAL_CONTINUE", "NFZ_REROUTE"):
            if state.route_coords:
                wp = state.route_coords[state.current_wp_idx]

                # Waypoint reached check (< 3.5 meters to real assigned waypoint)
                real_dx = wp[0] - state.pos_x
                real_dy = wp[1] - state.pos_y
                real_dist = (real_dx**2 + real_dy**2)**0.5

                if real_dist < 3.5:
                    if state.tasks_completed < state.total_tasks:
                        state.tasks_completed += 1
                    prev_idx = state.current_wp_idx
                    state.current_wp_idx = (state.current_wp_idx + 1) % len(state.route_coords)
                    state.reroute_active = False
                    state.reroute_target = []
                    _SWARM_BUS.log_event(
                        f"■ [{state.uav_id}] Completed WP-{prev_idx+1} ({state.tasks_completed}/{state.total_tasks}
    ■  Tasks) → Next WP-{state.current_wp_idx+1}"
                    )
                    _SWARM_BUS.notify_task_completed(
                        uav_id=state.uav_id,
                        wp_idx=prev_idx + 1,
                        tasks_done=state.tasks_completed,
                        total_tasks=state.total_tasks,
                    )
                    wp = state.route_coords[state.current_wp_idx]

                # If currently tracking a dynamic bypass and reached within 3m, clear it to resume path to wp
                if getattr(state, "reroute_target", None) and len(state.reroute_target) == 2:
                    b_dist = ((state.reroute_target[0] - state.pos_x)**2 + (state.reroute_target[1] - state.pos_y)*
    ■ *2)**0.5
                    if b_dist < 3.0:
                        state.reroute_target = []
                        state.reroute_active = False
```

```
                target_x, target_y = wp[0], wp[1]
                cur_dx = target_x - state.pos_x
                cur_dy = target_y - state.pos_y
                seg_sq = cur_dx**2 + cur_dy**2

                # ■■ Proactive Predictive Line-Segment NFZ Avoidance ■■
                if seg_sq > 0.1:
                    path_obstructed = False
                    for nfz in _SWARM_BUS.nfz_zones:
                        cx, cy, r_nfz = nfz["x"], nfz["y"], nfz["radius"]
                        buffer_r = r_nfz + 10.0  # 10-meter clearance envelope

                        t_proj = max(0.0, min(1.0, ((cx - state.pos_x)*cur_dx + (cy - state.pos_y)*cur_dy) / seg_sq
    ■ ))
                        qx = state.pos_x + t_proj * cur_dx
                        qy = state.pos_y + t_proj * cur_dy
                        dist_path = ((qx - cx)**2 + (qy - cy)**2)**0.5
                        dist_drone = ((state.pos_x - cx)**2 + (state.pos_y - cy)**2)**0.5

                        if dist_path < buffer_r or dist_drone < buffer_r:
                            path_obstructed = True
                            state.reroute_active = True
                            state.final_action = "NFZ_REROUTE"
                            state.agent_proposal = "NFZ_REROUTE"

                            # If active bypass waypoint is already set and > 3m away, track it stably
                            if getattr(state, "reroute_target", None) and len(state.reroute_target) == 2:
                                rem_b_dist = ((state.reroute_target[0] - state.pos_x)**2 + (state.reroute_target[1]
    ■  - state.pos_y)**2)**0.5
                                if rem_b_dist >= 3.0:
                                    target_x, target_y = state.reroute_target[0], state.reroute_target[1]
                                    break

                            # Normal vector away from center
                            nx = (state.pos_x - cx) if dist_drone > 0.1 else (qx - cx)
                            ny = (state.pos_y - cy) if dist_drone > 0.1 else (qy - cy)
                            n_len = max(0.1, (nx**2 + ny**2)**0.5)
                            nx /= n_len
                            ny /= n_len

                            # Tangent vector: choose direction pointing toward destination wp
                            tx, ty = -ny, nx
                            if (tx * cur_dx + ty * cur_dy) < 0:
                                tx, ty = -tx, -ty

                            # Dynamic Bypass Waypoint: pushed 14m outside the zone
                            bypass_x = cx + nx * (r_nfz + 14.0) + tx * 10.0
                            bypass_y = cy + ny * (r_nfz + 14.0) + ty * 10.0
                            target_x, target_y = bypass_x, bypass_y
                            state.reroute_target = [round(bypass_x, 1), round(bypass_y, 1)]
                            state.reasoning_trace = (
                                f"■ 2-Layer Guardrail: Path to WP blocked by {nfz['name']}! "
                                f"Proactively rerouting around safe perimeter (+14m clearance)."
                            )
                            break

                    if not path_obstructed:
                        state.reroute_active = False
                        state.reroute_target = []

                dx = target_x - state.pos_x
                dy = target_y - state.pos_y
                dist_2d = max(0.1, (dx**2 + dy**2)**0.5)

                # Cruise speed towards active target (either bypass or waypoint)
                cruise_spd = 2.2
                state.vx = (dx / dist_2d) * cruise_spd
                state.vy = (dy / dist_2d) * cruise_spd

                state.pos_x += state.vx * 0.2
                state.pos_y += state.vy * 0.2
                state.battery_pct = max(0.0, state.battery_pct - 0.04)

        elif state.final_action == "RETURN_TO_HOME":
            target_x, target_y = 0.0, 0.0
            state.reroute_active = False
            state.reroute_target = []

            # Check if RTL straight line (pos -> (0,0)) intersects any NFZ
            cur_dx = target_x - state.pos_x
            cur_dy = target_y - state.pos_y
            seg_sq = cur_dx**2 + cur_dy**2

            if seg_sq > 0.1:
                for nfz in _SWARM_BUS.nfz_zones:
                    cx, cy, r_nfz = nfz["x"], nfz["y"], nfz["radius"]
                    buffer_r = r_nfz + 10.0

                    t_proj = max(0.0, min(1.0, ((cx - state.pos_x)*cur_dx + (cy - state.pos_y)*cur_dy) / seg_sq))
                    qx = state.pos_x + t_proj * cur_dx
                    qy = state.pos_y + t_proj * cur_dy
                    dist_path = ((qx - cx)**2 + (qy - cy)**2)**0.5
                    dist_drone = ((state.pos_x - cx)**2 + (state.pos_y - cy)**2)**0.5

                    if dist_path < buffer_r or dist_drone < buffer_r:
                        state.reroute_active = True
                        nx = (state.pos_x - cx) if dist_drone > 0.1 else (qx - cx)
                        ny = (state.pos_y - cy) if dist_drone > 0.1 else (qy - cy)
                        n_len = max(0.1, (nx**2 + ny**2)**0.5)
                        nx /= n_len
                        ny /= n_len
                        tx, ty = -ny, nx
                        if (tx * cur_dx + ty * cur_dy) < 0:
                            tx, ty = -tx, -ty
                        bypass_x = cx + nx * (r_nfz + 14.0) + tx * 8.0
                        bypass_y = cy + ny * (r_nfz + 14.0) + ty * 8.0
```

```
                        target_x, target_y = bypass_x, bypass_y
                        state.reroute_target = [round(bypass_x, 1), round(bypass_y, 1)]
                        break

            dist = max(0.1, ((target_x - state.pos_x)**2 + (target_y - state.pos_y)**2)**0.5)
            if dist > 1.5:
                state.vx = ((target_x - state.pos_x) / dist) * 1.8
                state.vy = ((target_y - state.pos_y) / dist) * 1.8
                state.pos_x += state.vx * 0.2
                state.pos_y += state.vy * 0.2
            else:
                if target_x == 0.0 and target_y == 0.0:
                    state.vx = 0.0
                    state.vy = 0.0
                    state.altitude = max(0.0, state.altitude - 0.4)
                    if state.altitude <= 0.2:
                        state.on_ground = True
                        state.altitude = 0.0
            state.battery_pct = max(0.0, state.battery_pct - 0.03)

        elif state.final_action == "FAILSAFE_LAND":
            state.vx *= 0.5
            state.vy *= 0.5
            state.altitude = max(0.0, state.altitude - 0.5)
            state.battery_pct = max(0.0, state.battery_pct - 0.02)
            if state.altitude <= 0.2:
                state.on_ground = True
                state.altitude = 0.0
                state.vx = 0.0
                state.vy = 0.0

        elif state.final_action == "ALTITUDE_HOLD_HOVER":
            state.vx = 0.0
            state.vy = 0.0
            state.battery_pct = max(0.0, state.battery_pct - 0.03)
            state.hover_ticks += 1

            # 30-Second Hover Failsafe Timeout (120 ticks at 4Hz)
            if state.hover_ticks >= 120 and not state.on_ground:
                state.final_action = "FAILSAFE_LAND"
                state.agent_proposal = "FAILSAFE_LAND"
                state.reasoning_trace = "■■ GPS Hover Timeout Exceeded (30s) — Executing Controlled Failsafe Land t
    ■ o preserve hardware."
                _SWARM_BUS.log_event(f"■■ [{state.uav_id}] GPS Hover Timeout Exceeded (30s) → Auto Failsafe Landing
    ■ ")
            else:
                # If drone was hovering and fault was cleared, announce recovery
                if getattr(state, "hover_ticks", 0) > 0:
                    if state.fault_label == "No Failure":
                        _SWARM_BUS.log_event(f"■■ [{state.uav_id}] GPS Signal Restored → Resumed Navigation / RTL Corri
    ■ dor")
                state.hover_ticks = 0

        # ■■ HARD CBF SAFETY GEOFENCE CLAMP (Layer 2) ■■
        # Guaranteed physical wall: executes AFTER any position update for all airborne UAVs
        if not state.on_ground:
            for nfz in _SWARM_BUS.nfz_zones:
                d_c = ((state.pos_x - nfz["x"])**2 + (state.pos_y - nfz["y"])**2)**0.5
                hard_wall = nfz["radius"] + 5.0  # 5-meter absolute zero-tolerance boundary
                if d_c < hard_wall:
                    norm_x = (state.pos_x - nfz["x"]) / max(0.001, d_c)
                    norm_y = (state.pos_y - nfz["y"]) / max(0.001, d_c)
                    state.pos_x = nfz["x"] + norm_x * hard_wall
                    state.pos_y = nfz["y"] + norm_y * hard_wall
                    state.vx = norm_x * 2.5
                    state.vy = norm_y * 2.5
                    state.final_action = "NFZ_REROUTE"
                    state.reroute_active = True

        # Field Emergency Beacon: When landed on critical battery, activate ELT beacon
        if state.on_ground and state.energy_alert == "CRITICAL":
            if not getattr(state, "beacon_active", False):
                state.beacon_active = True
                state.role = "■ Beacon Active (Awaiting Retrieval)"
                _SWARM_BUS.log_event(f"■ [FIELD BEACON] {state.uav_id} safe touchdown at ({state.pos_x:.1f}, {state
    ■ .pos_y:.1f}) | ELT Beacon Active | Ground Retrieval Dispatched")

    # NFZ proximity check against all registered no-fly zones
    state.nfz_violation = False
    state.nfz_nearest = "—"
    state.nfz_dist = 999.0
    for nfz in _SWARM_BUS.nfz_zones:
        dx = state.pos_x - nfz["x"]
        dy = state.pos_y - nfz["y"]
        dist = (dx**2 + dy**2)**0.5
        margin = dist - nfz["radius"]
        if margin < state.nfz_dist:
            state.nfz_dist = round(margin, 1)
            state.nfz_nearest = nfz["name"]
        if margin < 5.0:  # Within 5m of NFZ boundary → VIOLATION WARNING
            state.nfz_violation = True

    # Mesh Link Quality and Latency to Swarm Leader
    lead_id = getattr(_SWARM_BUS, "current_leader", "UAV-1") or "UAV-1"
    if state.uav_id == lead_id:
        state.link_quality_pct = 100.0
        state.comms_latency_ms = 0.5
    elif state.on_ground:
        state.link_quality_pct = 0.0
        state.comms_latency_ms = 999.0
    else:
        lead_pos = _SWARM_BUS.peer_states.get(lead_id, {}).get("pos", [0.0, 0.0, 10.0])
        dist_lead = ((state.pos_x - lead_pos[0])**2 + (state.pos_y - lead_pos[1])**2)**0.5
        state.link_quality_pct = round(max(25.0, min(100.0, 100.0 - (dist_lead / 160.0) * 55.0)), 1)
        state.comms_latency_ms = round(1.8 + (dist_lead / 28.0), 1)

```

```
    return state


def node_monitor_energy(state: SwarmAgentState) -> SwarmAgentState:
    b = state.battery_pct
    if b <= 15.0:
        state.energy_alert = "CRITICAL"
    elif b <= 30.0:
        state.energy_alert = "LOW"
    else:
        state.energy_alert = "NORMAL"
    return state


def node_run_fdi(state: SwarmAgentState) -> SwarmAgentState:
    # Sensor FDI model check (already loaded from ExtraTrees)
    # Fault label is preserved if injected or computed
    return state


def node_compute_risk(state: SwarmAgentState) -> SwarmAgentState:
    risk_map = {
        "No Failure": 0.0, "Barometer Failure": 0.35, "Compass Failure": 0.30,
        "Accelerometer Failure": 0.75, "Gyroscope Failure": 0.85, "GPS Failure": 0.60
    }
    base = risk_map.get(state.fault_label, 0.0)
    conf = state.confidence / 100.0
    state.risk_stability = round(base * conf * 1.1, 2)
    state.risk_navigation = round(base * conf if state.gps_valid else 0.7, 2)

    b = state.battery_pct
    if b <= 15.0:
        state.risk_energy = round(min(1.0, (15.0 - b) / 15.0 + 0.5), 2)
    elif b <= 30.0:
        state.risk_energy = round((30.0 - b) / 30.0 * 0.7, 2)
    else:
        state.risk_energy = 0.0

    state.composite_risk = max(state.risk_stability, state.risk_navigation, state.risk_energy)
    return state


def node_agentic_reasoning(state: SwarmAgentState) -> SwarmAgentState:
    fault = state.fault_label
    energy = state.energy_alert
    gps_ok = state.gps_valid
    dist_home = (state.pos_x**2 + state.pos_y**2)**0.5

    # If already landed
    if state.on_ground:
        state.agent_proposal = "LANDED_SAFE"
        state.reasoning_trace = "Landed on ground safely. Motors disarmed."
        return state

    # Rule 1: Critical stabilization sensor lost
    if fault in ("Gyroscope Failure", "Accelerometer Failure"):
        if state.waypoints_queue:
            state.agent_proposal = "DELEGATE_TO_SWARM"
            state.reasoning_trace = f"■ {fault} detected! Delegating {len(state.waypoints_queue)} WPs to swarm peer
    ■  and descending safely."
        else:
            state.agent_proposal = "FAILSAFE_LAND"
            state.reasoning_trace = f"■ {fault} detected. Critical attitude control degraded → Land immediately ver
    ■ tically."

    # Rule 2: Battery CRITICAL (<= 15%)
    elif energy == "CRITICAL":
        if state.waypoints_queue:
            state.agent_proposal = "DELEGATE_TO_SWARM"
            state.reasoning_trace = f"Battery critical ({state.battery_pct:.1f}%). Requesting swarm peer to take re
    ■ maining {len(state.waypoints_queue)} WPs, then land."
        else:
            state.agent_proposal = "FAILSAFE_LAND"
            state.reasoning_trace = f"Battery critical ({state.battery_pct:.1f}%) and mission done → Failsafe desce
    ■ nt."

    # Rule 3: GPS Lost
    elif not gps_ok or fault == "GPS Failure":
        if state.waypoints_queue:
            state.agent_proposal = "DELEGATE_TO_SWARM"
            state.reasoning_trace = f"■ GPS Failure! Cannot navigate safely. Delegating {len(state.waypoints_queue)
    ■ } WPs to peer and holding altitude."
        else:
            state.agent_proposal = "ALTITUDE_HOLD_HOVER"
            state.reasoning_trace = "■ GPS signal lost or degraded. Holding position in place."

    # Rule 4: Battery LOW (15-30%)
    elif energy == "LOW":
        if dist_home > 40.0 and state.waypoints_queue:
            state.agent_proposal = "DELEGATE_TO_SWARM"
            state.reasoning_trace = f"Battery low ({state.battery_pct:.1f}%) & far from base ({dist_home:.0f}m). De
    ■ legating task sector to peer; returning home."
        else:
            state.agent_proposal = "RETURN_TO_HOME"
            state.reasoning_trace = f"Battery low ({state.battery_pct:.1f}%), within return radius ({dist_home:.0f}
    ■ m). Returning to home base."

    # Rule 5: Non-critical sensor fault
    elif fault in ("Barometer Failure", "Compass Failure"):
        state.agent_proposal = "ALTITUDE_HOLD_HOVER"
        state.reasoning_trace = f"{fault} detected. Pausing trajectory to stabilize attitude and heading."

    # Rule 6: Moderate risk
    elif state.composite_risk >= 0.4:
        state.agent_proposal = "ALTITUDE_HOLD_HOVER"
        state.reasoning_trace = f"Risk elevated ({state.composite_risk:.2f}). Pausing for assessment."
```

```
    # Rule 6.5: No-Fly Zone violation — reroute or delegate
    elif state.nfz_violation:
        if state.nfz_dist < 0.0:
            # INSIDE the NFZ! Emergency reroute + delegate task
            if state.waypoints_queue:
                state.agent_proposal = "DELEGATE_TO_SWARM"
                state.reasoning_trace = (
                    f"■ INSIDE {state.nfz_nearest} (margin {state.nfz_dist:.1f}m)! "
                    f"Delegating {len(state.waypoints_queue)} WPs to swarm peer and retreating."
                )
            else:
                state.agent_proposal = "RETURN_TO_HOME"
                state.reasoning_trace = f"■ INSIDE {state.nfz_nearest}! No waypoints left, returning to home."
            # Reverse velocity to flee NFZ
            nfz_match = next((n for n in _SWARM_BUS.nfz_zones if n["name"] == state.nfz_nearest), None)
            if nfz_match:
                dx = state.pos_x - nfz_match["x"]
                dy = state.pos_y - nfz_match["y"]
                norm = max(0.01, (dx**2 + dy**2)**0.5)
                state.vx = (dx / norm) * 1.5  # Flee outward at 1.5 m/s
                state.vy = (dy / norm) * 1.5
        else:
            # Approaching NFZ boundary (<5m) — reroute around it
            state.agent_proposal = "NFZ_REROUTE"
            nfz_match = next((n for n in _SWARM_BUS.nfz_zones if n["name"] == state.nfz_nearest), None)
            if nfz_match:
                # Steer tangentially (perpendicular to NFZ center vector)
                dx = state.pos_x - nfz_match["x"]
                dy = state.pos_y - nfz_match["y"]
                norm = max(0.01, (dx**2 + dy**2)**0.5)
                # Tangent vector (rotate 90°) * 2.2 m/s + outward repulsive buffer * 1.2 m/s
                state.vx = (-dy / norm) * 2.2 + (dx / norm) * 1.2
                state.vy = (dx / norm) * 2.2 + (dy / norm) * 1.2
            state.reasoning_trace = (
                f"■■ Approaching {state.nfz_nearest} (margin {state.nfz_dist:.1f}m). "
                f"CBF activated — rerouting tangentially around NFZ boundary."
            )

    # Rule 7: Nominal
    else:
        state.agent_proposal = "NOMINAL_CONTINUE"
        state.reasoning_trace = f"All parameters nominal (batt {state.battery_pct:.0f}%, risk {state.composite_risk
    ■ :.2f})."

    return state


def node_swarm_coordination(state: SwarmAgentState) -> SwarmAgentState:
    if state.agent_proposal == "DELEGATE_TO_SWARM":
        # Capture remaining route waypoints BEFORE clearing
        remaining_route = []
        if state.route_coords:
            remaining_route = list(state.route_coords[state.current_wp_idx:])

        assigned_peer = _SWARM_BUS.request_handoff(
            sender_id=state.uav_id,
            waypoints=list(state.waypoints_queue),
            reason=f"Degradation ({state.energy_alert} batt / {state.fault_label})"
        )
        if assigned_peer:
            state.delegation_target = assigned_peer
            state.delegation_status = "DELEGATED"
            # Store remaining route in SwarmBus for the peer to pick up
            with _SWARM_BUS.lock:
                if not hasattr(_SWARM_BUS, '_pending_handoff_routes'):
                    _SWARM_BUS._pending_handoff_routes = {}
                _SWARM_BUS._pending_handoff_routes[assigned_peer] = {
                    "from": state.uav_id,
                    "remaining_route": remaining_route,
                    "task_name": state.active_task,
                    "wp_names": list(state.waypoints_queue),
                }
            state.waypoints_queue.clear()
            state.active_task = f"Degraded – Transferred to {assigned_peer}"
            # After delegating, recovery action:
            if state.battery_pct > 20.0 and state.gps_valid and state.fault_label not in ("Gyroscope Failure", "Acc
    ■ elerometer Failure"):
                state.agent_proposal = "RETURN_TO_HOME"
            else:
                state.agent_proposal = "FAILSAFE_LAND"
        else:
            state.delegation_status = "NO_PEER_AVAILABLE"
            state.agent_proposal = "FAILSAFE_LAND"
            state.reasoning_trace += " | Swarm exhausted: falling back to emergency land."
    else:
        state.delegation_status = "N/A"

    return state


def node_guardrail_l1(state: SwarmAgentState) -> SwarmAgentState:
    if state.on_ground:
        state.final_action = "LANDED_SAFE"
        state.l1_status = "PASSED"
        state.l1_notes = "Grounded / Disarmed"
        return state

    action = state.agent_proposal
    notes = []
    passed = True

    if action == "RETURN_TO_HOME" and not state.gps_valid:
        action = "ALTITUDE_HOLD_HOVER"
        notes.append("No GPS: RTH blocked → HOVER")
        passed = False
```

```
    if action == "NFZ_REROUTE" and not state.gps_valid:
        action = "ALTITUDE_HOLD_HOVER"
        notes.append("No GPS: NFZ reroute blocked → HOVER")
        passed = False

    if state.altitude < 0.3 and action not in ("FAILSAFE_LAND", "LANDED_SAFE", "NOMINAL_CONTINUE"):
        action = "FAILSAFE_LAND"
        notes.append("Alt < 0.3m → LAND")
        passed = False

    state.l1_status = "PASSED" if passed else "MODIFIED"
    state.l1_notes = " | ".join(notes) if notes else "Policy OK"
    state.final_action = action
    return state


def node_guardrail_l2(state: SwarmAgentState) -> SwarmAgentState:
    action = state.final_action
    if state.on_ground or action == "LANDED_SAFE":
        state.l2_status = "PASSED"
        state.l2_notes = "Zero thrust (Landed)"
        return state
    u_nom = -2.0 if action == "FAILSAFE_LAND" else (-0.8 if action == "RETURN_TO_HOME" else 0.0)

    if _CBF_AVAILABLE:
        cbf = UAVControlBarrierFunction()
        vz_down = max(0.0, -u_nom)
        # Gather peer positions for swarm separation constraint
        peer_positions = []
        for info in _SWARM_BUS.get_peer_info(state.uav_id):
            peer_positions.append(info.get("pos", [0.0, 0.0, 0.0]))
        current_pos = [state.pos_x, state.pos_y, state.altitude]
        # Gather NFZ zones for geofence constraint
        nfz_for_cbf = [{"x": z["x"], "y": z["y"], "radius": z["radius"]} for z in _SWARM_BUS.nfz_zones]

        u_safe_vec, cbf_active, _, cbf_info = cbf.filter_control(
            np.array([state.vx, state.vy, vz_down]),
            state.altitude,
            peer_positions=peer_positions if peer_positions else None,
            current_pos=current_pos,
            nfz_zones=nfz_for_cbf if nfz_for_cbf else None,
        )
        # Apply CBF-corrected velocity back to the agent
        if cbf_active:
            state.vx = float(u_safe_vec[0])
            state.vy = float(u_safe_vec[1])
        intervened = bool(cbf_active)
        cbf_detail = "NFZ+Swarm+Ground" if state.nfz_violation else "Ground+Swarm"
    else:
        intervened = (state.altitude < 1.0 and u_nom < -0.5)
        cbf_detail = "Fallback"

    if intervened:
        state.l2_status = "INTERVENED"
        state.l2_notes = f"CBF modified vel ({cbf_detail})"
    else:
        state.l2_status = "PASSED"
        state.l2_notes = f"Safe barrier ({cbf_detail})"

    return state


def node_execute_action(state: SwarmAgentState) -> SwarmAgentState:
    return state


def node_finalize(state: SwarmAgentState) -> SwarmAgentState:
    # Broadcast heartbeat to the SwarmBus
    _SWARM_BUS.broadcast_heartbeat(
        uav_id=state.uav_id,
        state_summary={
            "pos": [state.pos_x, state.pos_y, state.altitude],
            "vel": [state.vx, state.vy, state.vz],
            "battery_pct": state.battery_pct,
            "fault": state.fault_label,
            "available": state.available,
            "task": state.active_task,
            "action": state.final_action,
            "is_leader": state.is_leader,
            "role": state.role,
            "current_wp_idx": state.current_wp_idx,
            "tasks_completed": state.tasks_completed,
            "total_tasks": state.total_tasks,
            "link_quality_pct": state.link_quality_pct,
            "comms_latency_ms": state.comms_latency_ms,
        }
    )
    return state


def build_uav_graph():
    g = StateGraph(SwarmAgentState)
    g.add_node("telemetry",   node_collect_telemetry)
    g.add_node("energy",      node_monitor_energy)
    g.add_node("fdi",         node_run_fdi)
    g.add_node("risk",        node_compute_risk)
    g.add_node("reasoning",   node_agentic_reasoning)
    g.add_node("swarm",       node_swarm_coordination)
    g.add_node("guardrail_l1",node_guardrail_l1)
    g.add_node("guardrail_l2",node_guardrail_l2)
    g.add_node("execute",     node_execute_action)
    g.add_node("finalize",    node_finalize)

    g.add_edge(START,         "telemetry")
    g.add_edge("telemetry",   "energy")
```

```
    g.add_edge("energy",      "fdi")
    g.add_edge("fdi",         "risk")
    g.add_edge("risk",        "reasoning")
    g.add_edge("reasoning",   "swarm")
    g.add_edge("swarm",       "guardrail_l1")
    g.add_edge("guardrail_l1","guardrail_l2")
    g.add_edge("guardrail_l2","execute")
    g.add_edge("execute",     "finalize")
    g.add_edge("finalize",    END)
    return g.compile()


# ═══════════════════════════════════════════════════════════════════════════════
# Swarm Orchestrator & Live Terminal Dashboard
# ═══════════════════════════════════════════════════════════════════════════════
_COLORS = {
    "CYAN": "\x1b[36m", "GREEN": "\x1b[32m", "YELLOW": "\x1b[33m",
    "RED": "\x1b[31m", "BOLD": "\x1b[1m", "RESET": "\x1b[0m",
    "WHITE": "\x1b[97m", "MAGENTA": "\x1b[35m", "BLUE": "\x1b[34m",
}

def col(text: str, c: str) -> str:
    return f"{_COLORS[c]}{text}{_COLORS['RESET']}"

def mini_bar(pct: float, width: int = 10) -> str:
    filled = max(0, min(width, int(pct / 100.0 * width)))
    color = "GREEN" if pct > 30 else ("YELLOW" if pct > 15 else "RED")
    return col("■" * filled + "■" * (width - filled), color)


def export_master_system_report(agents: List[SwarmAgentState], tick: int, report_path: str = "/home/samiksha/uav_fd
    ■ i_data/master_system_report.json", task_event: dict = None):
    """
    Aggregates full swarm mission and telemetry data from the Master UAV and exports
    to a structured JSON file so external systems and operators have total transparency.
    When task_event is provided, the report is event-triggered (task completion);
    otherwise it is a periodic telemetry snapshot.
    """
    lead_id = getattr(_SWARM_BUS, "current_leader", "UAV-1") or "UAV-1"
    master_agent = next((a for a in agents if a.uav_id == lead_id), None)
    if not master_agent and agents:
        master_agent = agents[0]
        lead_id = master_agent.uav_id

    total_tasks = sum(a.total_tasks for a in agents)
    completed_tasks = sum(min(a.total_tasks, a.tasks_completed) for a in agents)
    progress_pct = min(100.0, round((completed_tasks / max(1, total_tasks)) * 100.0, 1))

    online_agents = sum(1 for a in agents if not a.on_ground)
    fault_agents = [a for a in agents if a.fault_label != "No Failure" or a.energy_alert != "NORMAL"]

    mesh_matrix = {}
    for a in agents:
        mesh_matrix[a.uav_id] = {
            "link_quality_pct": a.link_quality_pct,
            "latency_ms": a.comms_latency_ms,
            "status": "ONLINE" if not a.on_ground else "GROUNDED_DISARMED",
            "rssi_dbm": int(-30 - (100.0 - a.link_quality_pct) * 0.6)
        }

    report_data = {
        "report_type": "AUTONOMOUS_SWARM_SYSTEM_TELEMETRY",
        "timestamp_epoch": time.time(),
        "timestamp_iso": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "tick": tick,
        "swarm_master": {
            "uav_id": lead_id,
            "role": master_agent.role if master_agent else "Swarm Leader",
            "position": [round(master_agent.pos_x, 2), round(master_agent.pos_y, 2), round(master_agent.altitude, 2
    ■ )] if master_agent else [0, 0, 0],
            "battery_pct": round(master_agent.battery_pct, 1) if master_agent else 100.0,
            "health": master_agent.fault_label if master_agent else "No Failure",
        },
        "swarm_health_summary": {
            "total_agents": len(agents),
            "online_agents": online_agents,
            "grounded_agents": len(agents) - online_agents,
            "active_faults_count": len(fault_agents),
            "active_faults": {a.uav_id: a.fault_label for a in fault_agents},
            "system_status": "NOMINAL" if len(fault_agents) == 0 and online_agents == len(agents) else "DEGRADED",
            "simulation_backend": "PX4_SITL_GAZEBO" if _SWARM_BUS.is_connected_to_px4() else "STANDALONE_SIM"
        },
        "swarm_mission_progress": {
            "total_tasks_completed": completed_tasks,
            "total_tasks_assigned": total_tasks,
            "progress_pct": progress_pct,
            "mission_status": "IN_PROGRESS" if progress_pct < 100.0 else "COMPLETED"
        },
        "mesh_connectivity_matrix": mesh_matrix,
        "agents_telemetry": [
            {
                "uav_id": a.uav_id,
                "role": a.role,
                "is_leader": a.is_leader,
                "position": {"x": round(a.pos_x, 2), "y": round(a.pos_y, 2), "z": round(a.altitude, 2)},
                "velocity": {"vx": round(a.vx, 2), "vy": round(a.vy, 2), "vz": round(a.vz, 2)},
                "battery_pct": round(a.battery_pct, 1),
                "energy_alert": a.energy_alert,
                "fdi_fault": a.fault_label,
                "decision_proposal": a.agent_proposal,
                "action": a.final_action,
                "current_waypoint_idx": a.current_wp_idx,
                "tasks_completed": a.tasks_completed,
                "total_tasks": a.total_tasks,
                "active_task": a.active_task,
                "nfz_distance_m": round(a.nfz_dist, 1),
```

```
                "nfz_violation": a.nfz_violation,
                "reroute_active": getattr(a, "reroute_active", False),
                "reroute_target": getattr(a, "reroute_target", []),
                "link_quality_pct": a.link_quality_pct,
                "comms_latency_ms": a.comms_latency_ms
            }
            for a in agents
        ],
        "active_nfz_geofences": _SWARM_BUS.nfz_zones,
        "report_trigger": "TASK_COMPLETION_EVENT" if task_event else "PERIODIC_SNAPSHOT",
        "task_completion_event": task_event if task_event else None,
    }

    try:
        tmp_path = report_path + ".tmp"
        with open(tmp_path, "w") as f:
            json.dump(report_data, f, indent=2)
        os.replace(tmp_path, report_path)
    except Exception:
        pass

    # ■■ Non-disruptive Flight Telemetry CSV Logging ■■
    try:
        csv_path = "/home/samiksha/uav_fdi_data/flight_telemetry_log.csv"
        file_exists = os.path.exists(csv_path)
        with open(csv_path, "a", newline="") as f_csv:
            writer = csv.writer(f_csv)
            if not file_exists:
                writer.writerow([
                    "tick", "timestamp_iso", "uav_id", "role", "is_leader",
                    "pos_x", "pos_y", "altitude", "vx", "vy", "vz",
                    "battery_pct", "fault_label", "final_action",
                    "current_wp_idx", "tasks_completed", "total_tasks",
                    "nfz_dist", "nfz_violation", "link_quality_pct", "comms_latency_ms"
                ])
            for a in agents:
                writer.writerow([
                    tick, time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
                    a.uav_id, a.role, int(a.is_leader),
                    round(a.pos_x, 2), round(a.pos_y, 2), round(a.altitude, 2),
                    round(a.vx, 2), round(a.vy, 2), round(a.vz, 2),
                    round(a.battery_pct, 1), a.fault_label, a.final_action,
                    a.current_wp_idx, a.tasks_completed, a.total_tasks,
                    round(a.nfz_dist, 1), int(a.nfz_violation),
                    round(a.link_quality_pct, 1), round(a.comms_latency_ms, 1)
                ])
    except Exception:
        pass




def render_swarm_dashboard(agents: List[SwarmAgentState], tick: int):
    N = len(agents)
    print("\x1b[2J\x1b[H", end="")
    CW = 26  # Column width per UAV
    W = max(110, N * (CW + 5) + 6)  # Dynamically adapt width to N agents
    print(col("■" + "■" * (W - 2) + "■", "CYAN"))
    title = f"  ■ AUTONOMOUS UAV SWARM — AGENTIC AI & FAULT-TOLERANT CONTROL (LangGraph × {N})  "
    print(col("■", "CYAN") + col(f"{title:^{W-2}}", "BOLD") + col("■", "CYAN"))
    print(col("■" + "■" * (W - 2) + "■", "CYAN"))

    # Global Swarm Status Bar
    active_count = sum(1 for a in agents if not a.on_ground)
    fault_count  = sum(1 for a in agents if a.fault_label != "No Failure" or a.energy_alert != "NORMAL")
    nfz_count    = sum(1 for a in agents if a.nfz_violation)
    avg_batt     = sum(a.battery_pct for a in agents) / max(1, N)

    lead_name = _SWARM_BUS.current_leader if hasattr(_SWARM_BUS, "current_leader") and _SWARM_BUS.current_leader el
    ■ se "DECENTRALIZED"
    lead_badge = col(f"■ LEADER: {lead_name}", "YELLOW")
    health_badge = col(f"HEALTH: {active_count}/{N} Online", "GREEN" if active_count == N else "YELLOW")
    fault_badge  = col(f"FAULTS: {fault_count}", "RED" if fault_count > 0 else "GREEN")
    sim_mode = col("■ PX4+GAZEBO: LIVE", "GREEN") if _SWARM_BUS.is_connected_to_px4() else col("■■ MODE: STANDALONE
    ■ ", "CYAN")
    batt_badge   = f"BATT: {avg_batt:.1f}%"
    tick_badge   = f"TICK: {tick:04d}"

    status_line = f"  {lead_badge}   ■   {health_badge}   ■   {fault_badge}   ■   {sim_mode}   ■   {batt_badge}   ■
    ■    {tick_badge}"
    print(col("■", "CYAN") + f"{status_line:<{W+24}}" + col("■", "CYAN"))
    print(col("■" + "■" * (W - 2) + "■", "CYAN"))

    # ■■ Dynamic N-agent column headers ■■
    def header_card(a: SwarmAgentState) -> str:
        if a.is_leader:
            gz_tag = " (GZ)" if (a.uav_id == "UAV-1" and _SWARM_BUS.is_connected_to_px4()) else ""
            return col(f" [{a.uav_id}] ■ LEADER{gz_tag} ", "YELLOW")
        elif "STEP DOWN" in a.role or "Ex-Leader" in a.role:
            return col(f" [{a.uav_id}] ■■ STEP DOWN ", "RED")
        else:
            return col(f" [{a.uav_id}] {a.role[:13]} ", "BOLD")

    h_line = "  " + "  ■  ".join([f"{header_card(a):<{CW+8}}" for a in agents])
    print(col("■", "CYAN") + f"{h_line:<{W+N*8}}" + col("■", "CYAN"))
    print(col("■" + "■" * (W - 2) + "■", "CYAN"))

    # ■■ Helper: plain row ■■
    def row(items: List[str]) -> None:
        line_content = "  " + "  ■  ".join([f"{it:<{CW}}" for it in items])
        print(col("■", "CYAN") + f"{line_content:<{W-2}}" + col("■", "CYAN"))

    # ■■ Helper: colored row ■■
    def color_row(items: List[str], extra: int = 9) -> None:
        line_content = "  " + "  ■  ".join([f"{it:<{CW+extra}}" for it in items])
        print(col("■", "CYAN") + f"{line_content:<{W + N*extra}}" + col("■", "CYAN"))
```

```
    |
    |     # Row 1: Position & Velocity
    |     row([f"■ ({a.pos_x:5.1f},{a.pos_y:5.1f}) z:{a.altitude:4.1f}m" for a in agents])
    |
    |     # Row 2: Velocity Vector
    |     row([f"■ v=({a.vx:+.1f},{a.vy:+.1f},{a.vz:+.1f}) m/s" for a in agents])
    |
    |     # Row 3: Battery Gauge
    |     row([f"■ {mini_bar(a.battery_pct, 6)} {a.battery_pct:5.1f}%" for a in agents])
    |
    |     # Row 4: FDI Status (colored)
    |     def fdi_color(a: SwarmAgentState) -> str:
    |         if a.fault_label == "No Failure":
    |             return col(f"■ FDI: {a.fault_label}", "GREEN")
    |         return col(f"■ FDI: {a.fault_label[:16]}", "RED")
    |     color_row([fdi_color(a) for a in agents])
    |
    |     # Row 5: Energy Alert (colored)
    |     def energy_color(a: SwarmAgentState) -> str:
    |         c = "GREEN" if a.energy_alert == "NORMAL" else ("YELLOW" if a.energy_alert == "LOW" else "RED")
    |         icon = "■" if a.energy_alert == "NORMAL" else ("■" if a.energy_alert == "LOW" else "■")
    |         return col(f"{icon} Energy: {a.energy_alert}", c)
    |     color_row([energy_color(a) for a in agents])
    |
    |     # Row 6: Risk Breakdown
    |     row([f"■ Risk: {a.composite_risk:.2f} (S:{a.risk_stability:.1f} N:{a.risk_navigation:.1f} E:{a.risk_energy:.1f}
    |     ■ )" for a in agents])
    |
    |     # Row 7: NFZ Status (colored)
    |     def nfz_color(a: SwarmAgentState) -> str:
    |         if a.nfz_violation:
    |             return col(f"■ NFZ: {a.nfz_nearest[:12]} {a.nfz_dist:+.0f}m", "RED")
    |         elif a.nfz_dist < 20.0:
    |             return col(f"■■ NFZ: {a.nfz_nearest[:12]} {a.nfz_dist:+.0f}m", "YELLOW")
    |         return col(f"■ NFZ: CLEAR ({a.nfz_dist:+.0f}m)", "GREEN")
    |     color_row([nfz_color(a) for a in agents])
    |
    |     # Row 8: Agent Decision (colored)
    |     def dec_color(a: SwarmAgentState) -> str:
    |         if a.agent_proposal in ("NOMINAL_CONTINUE", "LANDED_SAFE"):
    |             c = "GREEN"
    |         elif a.agent_proposal in ("RETURN_TO_HOME", "ALTITUDE_HOLD_HOVER", "NFZ_REROUTE"):
    |             c = "YELLOW"
    |         else:
    |             c = "RED"
    |         return col(f"■ {a.agent_proposal[:19]}", c)
    |     color_row([dec_color(a) for a in agents])
    |
    |     # Row 9: L1 Guardrail
    |     row([f"L1: {a.l1_status} {a.l1_notes[:15]}" for a in agents])
    |
    |     # Row 10: CBF Safety Filter (L2)
    |     def cbf_color(a: SwarmAgentState) -> str:
    |         if a.l2_status == "INTERVENED":
    |             return col(f"■■ CBF: INTERVENED", "YELLOW")
    |         return col(f"■■ CBF: {a.l2_notes[:16]}", "GREEN")
    |     color_row([cbf_color(a) for a in agents])
    |
    |     # Row 11: Final Action (colored)
    |     def act_color(a: SwarmAgentState) -> str:
    |         if a.final_action == "LANDED_SAFE":
    |             return col("■ LANDED_SAFE", "MAGENTA")
    |         c = "GREEN" if a.final_action in ("NOMINAL_CONTINUE", "NFZ_REROUTE") else ("YELLOW" if a.final_action in ("
    |     ■ RETURN_TO_HOME", "ALTITUDE_HOLD_HOVER") else "RED")
    |         return col(f"■ {a.final_action[:18]}", c)
    |     color_row([act_color(a) for a in agents])
    |
    |     # Row 12: Delegation Info
    |     def deleg_info(a: SwarmAgentState) -> str:
    |         if a.delegation_status == "DELEGATED":
    |             return f"■ → {a.delegation_target}"
    |         elif a.delegation_status == "NO_PEER_AVAILABLE":
    |             return "■ No peer avail"
    |         return "— N/A"
    |     row([deleg_info(a) for a in agents])
    |
    |     # Row 13: Active Task
    |     row([f"■ {a.active_task[:22]}" for a in agents])
    |
    |     # Row 14: Target Waypoint
    |     row([f"■ Target: WP-{a.current_wp_idx+1}" if a.route_coords else "■ Target: —" for a in agents])
    |
    |     # Row 15: Tasks Completed / Total
    |     def task_str(a: SwarmAgentState) -> str:
    |         tot = max(1, a.total_tasks)
    |         pct = int((a.tasks_completed / tot) * 100)
    |         return f"■ Tasks: {a.tasks_completed}/{a.total_tasks} ({pct}%)"
    |     row([task_str(a) for a in agents])
    |
    |     # Row 16: Mesh Link Quality to Master
    |     def link_badge(a: SwarmAgentState) -> str:
    |         if a.is_leader:
    |             return col("■ Mesh: LOCAL MASTER (0ms)", "GREEN")
    |         elif a.on_ground:
    |             return col("■ Mesh: DISCONNECTED", "RED")
    |         c = "GREEN" if a.link_quality_pct > 75 else ("YELLOW" if a.link_quality_pct > 40 else "RED")
    |         return col(f"■ Mesh: {a.link_quality_pct:.0f}% ({a.comms_latency_ms:.1f}ms)", c)
    |     color_row([link_badge(a) for a in agents])
    |
    |     # Row 17: Reasoning Trace (truncated)
    |     print(col("■" + "■" * (W - 2) + "■", "CYAN"))
    |     print(col("■", "CYAN") + col("  ■ Agent Reasoning Traces:".ljust(W-2), "MAGENTA") + col("■", "CYAN"))
    |     for a in agents:
    |         trace_line = f"    [{a.uav_id}] {a.reasoning_trace[:W-14]}"
    |         c = "RED" if any(w in a.reasoning_trace.lower() for w in ("critical", "fail", "nfz", "■")) else ("YELLOW" i
    |     ■ f any(w in a.reasoning_trace.lower() for w in ("low", "degraded", "hover", "reroute")) else "GREEN")
```

```
        print(col("■", "CYAN") + col(f"{trace_line:<{W-2}}", c) + col("■", "CYAN"))

    # NFZ Zone Legend
    if _SWARM_BUS.nfz_zones:
        print(col("■" + "■" * (W - 2) + "■", "CYAN"))
        nfz_legend = "  ■ NFZ Zones: " + " | ".join([f"{z['name']} ({z['x']:.0f},{z['y']:.0f}) r={z['radius']:.0f}m ■ " for z in _SWARM_BUS.nfz_zones])
        print(col("■", "CYAN") + col(f"{nfz_legend:<{W-2}}", "RED") + col("■", "CYAN"))

    # SwarmBus Event Log Box
    print(col("■" + "■" * (W - 2) + "■", "CYAN"))
    print(col("■", "CYAN") + col("  ■ Inter-Agent SwarmBus Decentralized Event & Handoff Log:".ljust(W-2), "YELLOW") + col("■", "CYAN"))
    print(col("■" + "■" * (W - 2) + "■", "CYAN"))
    with _SWARM_BUS.lock:
        if not _SWARM_BUS.event_log:
            print(col("■", "CYAN") + "    (Swarm nominal — exchanging periodic heartbeats on UDP 14550)".ljust(W-2) + col("■", "CYAN"))
        else:
            for ev in list(_SWARM_BUS.event_log)[-5:]:
                print(col("■", "CYAN") + f"    {ev:<{W-6}}" + col("■", "CYAN"))

    # ■■ Dedicated Master UAV System & Mission Telemetry Report ■■
    print(col("■" + "■" * (W - 2) + "■", "CYAN"))
    print(col("■", "CYAN") + col("  ■ MASTER UAV SYSTEM & MISSION TELEMETRY REPORT".ljust(W-2), "YELLOW") + col("■", "CYAN"))
    print(col("■" + "■" * (W - 2) + "■", "CYAN"))

    tot_tasks = sum(a.total_tasks for a in agents)
    done_tasks = sum(a.tasks_completed for a in agents)
    progress_pct = (done_tasks / max(1, tot_tasks)) * 100.0

    master_summary = f"    Master: {lead_name}  ■  Swarm Progress: {mini_bar(progress_pct, 12)} {progress_pct:5.1f}% ({done_tasks}/{tot_tasks} Tasks)  ■  Mesh Links: {active_count}/{N} Connected"
    print(col("■", "CYAN") + f"{master_summary:<{W+8}}" + col("■", "CYAN"))

    mesh_links = []
    for a in agents:
        if a.uav_id != lead_name:
            st = f"{a.link_quality_pct:.0f}% ({a.comms_latency_ms:.1f}ms)" if not a.on_ground else "OFFLINE"
            mesh_links.append(f"{lead_name}↔{a.uav_id}: {st}")
    if mesh_links:
        mesh_line = "    Mesh Links: " + "  ■  ".join(mesh_links)
        print(col("■", "CYAN") + f"{mesh_line:<{W-2}}" + col("■", "CYAN"))

    report_notice = "    ■ Live Master System Telemetry Exported → /home/samiksha/uav_fdi_data/master_system_report.json"
    print(col("■", "CYAN") + col(f"{report_notice:<{W-2}}", "WHITE") + col("■", "CYAN"))

    print(col("■" + "■" * (W - 2) + "■", "CYAN"))
    print(col(f"  [Agents: {N} | Terminal 4 Commands: 'fault UAV-2 GPS', 'clear UAV-2', 'clear all', 'batt UAV-2 14 ■ ' | Ctrl+C]", "BOLD"))


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Main Swarm Execution Loop
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
async def run_swarm(
    ticks: int = 0,
    inject_uav: str = "UAV-1",
    inject_batt: Optional[float] = None,
    inject_fault: Optional[str] = None,
    inject_tick: int = 1,
    add_uav_specs: Optional[List[str]] = None,
    remove_uav_specs: Optional[List[str]] = None,
    initial_uavs: int = 4,
):
    _SWARM_BUS.init_ros()
    _SWARM_BUS.init_mavlink()

    # Clean, verified flight corridors — ALL waypoints strictly outside No-Fly Zones
    # NFZ-Alpha (Hospital) center: (50.0, 60.0, r=18m) — UAV-2 flies perimeter corridor with >10m clearance
    # NFZ-Bravo (Airport) center: (-15.0, 90.0, r=22m) — UAV-3 flies perimeter corridor with >15m clearance
    uav1_route = [(0.0, 0.0, 12.0), (30.0, 15.0, 12.0), (70.0, 15.0, 12.0), (85.0, -10.0, 12.0), (45.0, -20.0, 12.0 ■ ), (0.0, 0.0, 12.0)]
    uav2_route = [(20.0, 10.0, 14.0), (50.0, 25.0, 14.0), (85.0, 60.0, 14.0), (50.0, 95.0, 14.0), (15.0, 60.0, 14.0 ■ ), (20.0, 10.0, 14.0)]
    uav3_route = [(-20.0, 30.0, 11.0), (-50.0, 65.0, 11.0), (-50.0, 115.0, 11.0), (20.0, 115.0, 11.0), (20.0, 65.0, ■  11.0), (-20.0, 30.0, 11.0)]
    uav4_route = [(10.0, -15.0, 10.0), (35.0, -25.0, 10.0), (50.0, -10.0, 10.0), (25.0, 5.0, 10.0), (10.0, -15.0, 1 ■ 0.0)]

    agents = [
        SwarmAgentState(
            uav_id="UAV-1", role="■ Swarm Leader", is_leader=True, pos_x=0.0, pos_y=0.0, altitude=12.0,
            vx=1.5, vy=1.0, battery_pct=95.0, active_task="Survey Sector Alpha",
            route_coords=uav1_route, total_tasks=len(uav1_route),
            waypoints_queue=[f"WP-1.{i+1}" for i in range(len(uav1_route))]
        ),
        SwarmAgentState(
            uav_id="UAV-2", role="Relay / Escort", pos_x=20.0, pos_y=10.0, altitude=14.0,
            vx=1.2, vy=1.1, battery_pct=92.0, active_task="Comms Relay & Recon",
            route_coords=uav2_route, total_tasks=len(uav2_route),
            waypoints_queue=[f"WP-2.{i+1}" for i in range(len(uav2_route))]
        ),
        SwarmAgentState(
            uav_id="UAV-3", role="Survey Bravo", pos_x=-20.0, pos_y=30.0, altitude=11.0,
            vx=-0.8, vy=1.3, battery_pct=88.0, active_task="Survey Sector Bravo",
            route_coords=uav3_route, total_tasks=len(uav3_route),
            waypoints_queue=[f"WP-3.{i+1}" for i in range(len(uav3_route))]
        ),
        SwarmAgentState(
            uav_id="UAV-4", role="Swarm Reserve", pos_x=10.0, pos_y=-15.0, altitude=10.0,
            vx=1.3, vy=-0.5, battery_pct=94.0, active_task="Perimeter Patrol",
            route_coords=uav4_route, total_tasks=len(uav4_route),
            waypoints_queue=[f"WP-4.{i+1}" for i in range(len(uav4_route))]
```

```
        ),
    ]

    # Support initial horizontal scaling beyond 4 agents
    if initial_uavs > 4:
        for idx in range(5, initial_uavs + 1):
            uid = f"UAV-{idx}"
            scale_route = [
                (15.0 * (idx - 3), -12.0 * (idx - 3), 12.5),
                (15.0 * (idx - 3) + 25.0, -12.0 * (idx - 3) + 15.0, 12.5),
                (15.0 * (idx - 3) + 10.0, -12.0 * (idx - 3) + 30.0, 12.5),
                (15.0 * (idx - 3), -12.0 * (idx - 3), 12.5),
            ]
            agents.append(
                SwarmAgentState(
                    uav_id=uid,
                    role=f"Scale Sector {idx}",
                    pos_x=15.0 * (idx - 3),
                    pos_y=-12.0 * (idx - 3),
                    altitude=12.5,
                    vx=0.8, vy=-0.5,
                    battery_pct=94.0,
                    active_task=f"Sector {chr(64 + idx)}",
                    route_coords=scale_route,
                    total_tasks=len(scale_route),
                    waypoints_queue=[f"WP-{idx}A", f"WP-{idx}B"]
                )
            )

    # Register all agents in the decentralized swarm bus
    for a in agents:
        _SWARM_BUS.register_uav(a.uav_id)

    graphs = {a.uav_id: build_uav_graph() for a in agents}
    current_tick = 0

    print(f"[Swarm] Initializing {len(agents)} autonomous LangGraph UAV agents...")
    _SWARM_BUS.ensure_gazebo_swarm_models()
    await asyncio.sleep(0.5)

    while True:
        current_tick += 1
        if current_tick % 25 == 1:
            _SWARM_BUS.ensure_gazebo_swarm_models()

        # ■■ 1. Live Interactive Commands from Terminal Stdin or /tmp/swarm_cmd ■■
        pending_commands = []
        try:
            import select
            if sys.stdin in select.select([sys.stdin], [], [], 0)[0]:
                raw = sys.stdin.readline().strip()
                if raw:
                    pending_commands.append(raw)
        except Exception:
            pass

        for cmd_path in ("/tmp/swarm_cmd", os.path.join(os.path.dirname(__file__), "swarm_cmd.txt")):
            if os.path.exists(cmd_path):
                try:
                    with open(cmd_path, "r") as f:
                        content = f.read().strip()
                    os.remove(cmd_path)
                    if content:
                        for line in content.splitlines():
                            if line.strip():
                                pending_commands.append(line.strip())
                except Exception:
                    pass

        def _norm_id(tok: str) -> str:
            t = tok.upper().strip()
            if t.isdigit(): return f"UAV-{t}"
            if t.startswith("UAV") and not t.startswith("UAV-"): return f"UAV-{t[3:]}"
            return t

        for raw_cmd in pending_commands:
            try:
                tokens = raw_cmd.split()
                if not tokens: continue

                # Syntax normalizer: if command is 'UAV-2 clear' or '2 clear' or 'UAV-2 batt 14'
                if (tokens[0].isdigit() or tokens[0].lower().startswith("uav")) and len(tokens) >= 2:
                    sub = tokens[1].lower()
                    if sub in ("clear", "ok", "reset"):
                        tokens = ["clear", tokens[0]]
                    elif sub in ("batt", "battery") and len(tokens) >= 3:
                        tokens = ["batt", tokens[0], tokens[2]]
                    elif sub in ("gps", "gyro", "gyroscope", "accel", "accelerometer", "baro", "barometer", "compas
■ s", "mag"):
                        tokens = ["fault", tokens[0], tokens[1]]

                verb = tokens[0].lower()

                if verb in ("takeoff", "arm"):
                    for a in agents:
                        a.on_ground = False
                        a.altitude = max(a.altitude, 12.0)
                        a.final_action = "NOMINAL_CONTINUE"
                        a.agent_proposal = "NOMINAL_CONTINUE"
                    _SWARM_BUS.log_event("■ Swarm commanded to Takeoff to operational altitude (12m)")

                elif verb in ("land", "disarm"):
                    for a in agents:
                        a.final_action = "FAILSAFE_LAND"
                    _SWARM_BUS.log_event("■ Swarm commanded to Land safely")
```

```
2041 |                 elif verb in ("rtl", "home", "return"):
2042 |                     target_id = _norm_id(tokens[1]) if len(tokens) >= 2 and tokens[1].lower() not in ("all", "*", "
     |     ■ swarm") else "ALL"
2043 |                     for a in agents:
2044 |                         if target_id == "ALL" or a.uav_id == target_id:
2045 |                             a.final_action = "RETURN_TO_HOME"
2046 |                             a.agent_proposal = "RETURN_TO_HOME"
2047 |                             a.active_task = "Return To Home (RTL)"
2048 |                             a.role = "RTL Inbound"
2049 |                             a.waypoints_queue.clear()
2050 |                     _SWARM_BUS.log_event(f"■ [RTL] Return To Home commanded for {target_id} -> Vectoring back to Ho
     |     ■ me Base (0,0)")
2051 |
2052 |                 elif verb in ("add", "+") and len(tokens) >= 2:
2053 |                     for new_tok in tokens[1:]:
2054 |                         new_id = _norm_id(new_tok)
2055 |                         if not any(a.uav_id == new_id for a in agents):
2056 |                             idx = len(agents) + 1
2057 |                             scale_route = [
2058 |                                 (10.0 * (idx - 2), -15.0 * (idx - 2), 13.0),
2059 |                                 (10.0 * (idx - 2) + 20.0, -15.0 * (idx - 2) + 20.0, 13.0),
2060 |                                 (10.0 * (idx - 2), -15.0 * (idx - 2), 13.0)
2061 |                             ]
2062 |                             new_uav = SwarmAgentState(
2063 |                                 uav_id=new_id,
2064 |                                 role=f"Dynamic Join {idx}",
2065 |                                 pos_x=10.0 * (idx - 2),
2066 |                                 pos_y=-15.0 * (idx - 2),
2067 |                                 altitude=13.0,
2068 |                                 vx=0.8, vy=-0.6,
2069 |                                 battery_pct=95.0,
2070 |                                 active_task="Dynamic Survey",
2071 |                                 route_coords=scale_route,
2072 |                                 total_tasks=len(scale_route),
2073 |                                 waypoints_queue=[f"WP-D{idx}-1", f"WP-D{idx}-2"]
2074 |                             )
2075 |                             _SWARM_BUS.register_uav(new_id)
2076 |                             graphs[new_id] = build_uav_graph()
2077 |                             agents.append(new_uav)
2078 |                             _SWARM_BUS.log_event(f"■ [HORIZONTAL SCALING] {new_id} added via live operator command
     |     ■ (Swarm size: {len(agents)})")
2079 |
2080 |                 elif verb in ("remove", "-", "rm") and len(tokens) >= 2:
2081 |                     for rem_tok in tokens[1:]:
2082 |                         rem_id = _norm_id(rem_tok)
2083 |                         target = next((a for a in agents if a.uav_id == rem_id), None)
2084 |                         if target:
2085 |                             if target.waypoints_queue:
2086 |                                 _SWARM_BUS.request_handoff(rem_id, list(target.waypoints_queue), "Operator Removed
     |     ■ / Recalled")
2087 |                             agents.remove(target)
2088 |                             graphs.pop(rem_id, None)
2089 |                             with _SWARM_BUS.lock:
2090 |                                 _SWARM_BUS.peer_states.pop(rem_id, None)
2091 |                                 _SWARM_BUS.active_faults.pop(rem_id, None)
2092 |                             _SWARM_BUS.log_event(f"■ [HORIZONTAL SCALING] {rem_id} removed via live operator comman
     |     ■ d (Remaining: {len(agents)})")
2093 |
2094 |                 elif verb in ("fault", "inject", "f"):
2095 |                     # formats: fault 1 gps, fault UAV-2 GPS, or fault gps 2
2096 |                     target_id = "UAV-1"
2097 |                     sensor_name = "GPS Failure"
2098 |                     for tok in tokens[1:]:
2099 |                         tok_low = tok.lower()
2100 |                         if tok.isdigit() or tok_low.startswith("uav"):
2101 |                             target_id = _norm_id(tok)
2102 |                         elif tok_low in ("gps", "nav"):
2103 |                             sensor_name = "GPS Failure"
2104 |                         elif tok_low in ("gyro", "gyroscope"):
2105 |                             sensor_name = "Gyroscope Failure"
2106 |                         elif tok_low in ("accel", "accelerometer"):
2107 |                             sensor_name = "Accelerometer Failure"
2108 |                         elif tok_low in ("baro", "barometer"):
2109 |                             sensor_name = "Barometer Failure"
2110 |                         elif tok_low in ("compass", "mag"):
2111 |                             sensor_name = "Compass Failure"
2112 |                     with _SWARM_BUS.lock:
2113 |                         _SWARM_BUS.active_faults[target_id] = sensor_name
2114 |                     target = next((a for a in agents if a.uav_id == target_id), None)
2115 |                     if target:
2116 |                         target.fault_label = sensor_name
2117 |                         if sensor_name == "GPS Failure":
2118 |                             target.gps_valid = False
2119 |                     _SWARM_BUS.log_event(f"■ Live injected {sensor_name} on {target_id}")
2120 |
2121 |                 elif verb in ("clear", "ok", "reset"):
2122 |                     # formats: clear 1, clear UAV-2, or just 'clear' / 'clear all'
2123 |                     is_all = (len(tokens) == 1) or (len(tokens) >= 2 and tokens[1].lower() in ("all", "*", "swarm")
     |     ■ )
2124 |                     if not is_all:
2125 |                         target_id = _norm_id(tokens[1])
2126 |                         with _SWARM_BUS.lock:
2127 |                             _SWARM_BUS.active_faults.pop(target_id, None)
2128 |                         target = next((a for a in agents if a.uav_id == target_id), None)
2129 |                         if target:
2130 |                             target.fault_label = "No Failure"
2131 |                             target.gps_valid = True
2132 |                             target.battery_pct = max(target.battery_pct, 92.0)
2133 |                             target.energy_alert = "NORMAL"
2134 |                             target.on_ground = False
2135 |                             target.altitude = max(target.altitude, 12.0)
2136 |                             target.final_action = "NOMINAL_CONTINUE"
2137 |                             target.agent_proposal = "NOMINAL_CONTINUE"
2138 |                             target.available = True
2139 |                             target.delegation_status = "N/A"
2140 |                             target.delegation_target = ""
```

```
                            target.beacon_active = False
                            target.hover_ticks = 0
                            if not target.waypoints_queue and target.route_coords:
                                target.waypoints_queue = [f"WP-{target.uav_id}.{i+1}" for i in range(len(target.rou
    ■ te_coords))]
                        _SWARM_BUS.log_event(f"■ Restored {target_id} to 100% nominal (Battery: 92%, Faults Cleared
    ■ , Resumed Route)")
                    else:
                        with _SWARM_BUS.lock:
                            _SWARM_BUS.active_faults.clear()
                        for a in agents:
                            a.fault_label = "No Failure"
                            a.gps_valid = True
                            a.battery_pct = max(a.battery_pct, 92.0)
                            a.energy_alert = "NORMAL"
                            a.on_ground = False
                            a.altitude = max(a.altitude, 12.0)
                            a.final_action = "NOMINAL_CONTINUE"
                            a.agent_proposal = "NOMINAL_CONTINUE"
                            a.available = True
                            a.delegation_status = "N/A"
                            a.delegation_target = ""
                            a.beacon_active = False
                            a.hover_ticks = 0
                            if not a.waypoints_queue and a.route_coords:
                                a.waypoints_queue = [f"WP-{a.uav_id}.{i+1}" for i in range(len(a.route_coords))]
                        _SWARM_BUS.log_event("■ Restored ALL UAVs to nominal health & 92% battery (All Missions Res
    ■ umed)")

                elif verb in ("batt", "battery", "b") and len(tokens) >= 3:
                    target_id = _norm_id(tokens[1])
                    b_val = float(tokens[2])
                    target = next((a for a in agents if a.uav_id == target_id), None)
                    if target:
                        target.battery_pct = b_val
                        _SWARM_BUS.log_event(f"■ Set {target_id} battery to {b_val:.0f}%")

                elif verb in ("idle", "loiter") and len(tokens) >= 2:
                    target_id = _norm_id(tokens[1])
                    target = next((a for a in agents if a.uav_id == target_id), None)
                    if target:
                        target.role = "Swarm Reserve"
                        target.active_task = "Reserve Loiter"
                        target.waypoints_queue.clear()
                        target.final_action = "STANDBY_LOITER"
                        _SWARM_BUS.log_event(f"■ {target_id} placed in standby reserve loiter")

                elif verb == "add" and len(tokens) >= 2:
                    for tok in tokens[1:]:
                        new_id = _norm_id(tok)
                        if not any(a.uav_id == new_id for a in agents):
                            idx = len(agents) + 1
                            scale_route = [
                                (10.0 * (idx - 1), -10.0 * (idx - 1), 12.0),
                                (20.0 * (idx - 1), -5.0 * (idx - 1), 14.0),
                                (10.0 * (idx - 1), -10.0 * (idx - 1), 12.0),
                            ]
                            new_uav = SwarmAgentState(
                                uav_id=new_id,
                                role=f"Escort & Survey",
                                pos_x=10.0 * (idx - 2),
                                pos_y=-15.0 * (idx - 2),
                                altitude=13.0,
                                vx=0.4, vy=-0.3,
                                battery_pct=95.0,
                                active_task="Dynamic Enroute",
                                route_coords=scale_route,
                                total_tasks=len(scale_route),
                                waypoints_queue=[f"WP-{idx}A", f"WP-{idx}B"]
                            )
                            _SWARM_BUS.register_uav(new_id)
                            graphs[new_id] = build_uav_graph()
                            agents.append(new_uav)
                            _SWARM_BUS.log_event(f"■ [HORIZONTAL SCALING] {new_id} dynamically added to swarm (Tota
    ■ l: {len(agents)} UAVs)")

                elif verb in ("remove", "delete", "rm") and len(tokens) >= 2:
                    for tok in tokens[1:]:
                        rem_id = _norm_id(tok)
                        target = next((a for a in agents if a.uav_id == rem_id), None)
                        if target and len(agents) > 1:
                            target.on_ground = True
                            target.final_action = "FAILSAFE_LAND"
                            target.role = "Decommissioned"
                            target.available = False
                            _SWARM_BUS.log_event(f"■ [HORIZONTAL SCALING] {rem_id} decommissioned & landed safely")
            except Exception as e:
                _SWARM_BUS.log_event(f"Command error: {e}")

        # ■■ 2. Dynamic Horizontal Scaling: Add one or more agents at scheduled ticks ■■
        if add_uav_specs:
            for spec in list(add_uav_specs):
                parts = spec.split(":")
                new_id = parts[0].strip().upper()
                at_t = int(parts[1].strip()) if len(parts) > 1 else 1
                if current_tick == at_t and not any(a.uav_id == new_id for a in agents):
                    idx = len(agents) + 1
                    new_uav = SwarmAgentState(
                        uav_id=new_id,
                        role=f"Dynamic Join {idx}",
                        pos_x=10.0 * (idx - 2),
                        pos_y=-15.0 * (idx - 2),
                        altitude=13.0,
                        vx=0.4, vy=-0.3,
                        battery_pct=95.0,
                        active_task="Dynamic Survey",
```

```
                        waypoints_queue=[f"WP-D{idx}-1", f"WP-D{idx}-2"]
                    )
                    _SWARM_BUS.register_uav(new_id)
                    graphs[new_id] = build_uav_graph()
                    agents.append(new_uav)
                    _SWARM_BUS.log_event(f"■ [HORIZONTAL SCALING] {new_id} joined active swarm mesh seamlessly (Swa
    ■ rm size: {len(agents)})")

        # ■■ 3. Dynamic Removal: Safely decommission/recall agents at scheduled ticks ■■
        if remove_uav_specs:
            for spec in list(remove_uav_specs):
                parts = spec.split(":")
                rem_id = parts[0].strip().upper()
                at_t = int(parts[1].strip()) if len(parts) > 1 else 1
                if current_tick == at_t:
                    target = next((a for a in agents if a.uav_id == rem_id), None)
                    if target:
                        if target.waypoints_queue:
                            _SWARM_BUS.request_handoff(rem_id, list(target.waypoints_queue), "Scheduled Removal / R
    ■ ecalled")
                        agents.remove(target)
                        graphs.pop(rem_id, None)
                        with _SWARM_BUS.lock:
                            _SWARM_BUS.peer_states.pop(rem_id, None)
                            _SWARM_BUS.active_faults.pop(rem_id, None)
                        _SWARM_BUS.log_event(f"■ [HORIZONTAL SCALING] {rem_id} safely decommissioned at tick {curre
    ■ nt_tick} (Remaining: {len(agents)})")

        # ■■ 4. CLI Fault Injection ■■
        if current_tick >= inject_tick:
            for a in agents:
                if a.uav_id == inject_uav:
                    if inject_batt is not None:
                        a.battery_pct = float(inject_batt)
                    if inject_fault:
                        a.fault_label = inject_fault
                        if inject_fault == "GPS Failure":
                            a.gps_valid = False

        # ■■ 5. Poll live PX4 telemetry ■■
        px4_data = _SWARM_BUS.poll_px4_telemetry()
        if px4_data:
            for a in agents:
                if a.uav_id == "UAV-1":
                    if "pos_x" in px4_data:
                        a.pos_x = float(px4_data["pos_x"])
                    if "pos_y" in px4_data:
                        a.pos_y = float(px4_data["pos_y"])
                    if "pos_z" in px4_data:
                        a.altitude = float(px4_data["pos_z"])
                    if "vx" in px4_data:
                        a.vx = float(px4_data["vx"])
                    if "vy" in px4_data:
                        a.vy = float(px4_data["vy"])
                    if "vz" in px4_data:
                        a.vz = float(px4_data["vz"])
                    if "battery_pct" in px4_data:
                        a.battery_pct = float(px4_data["battery_pct"])

        # ■■ 6. Check live fault injection from QGroundControl, Terminal, or PX4 ■■
        with _SWARM_BUS.lock:
            for a in agents:
                if a.uav_id in _SWARM_BUS.active_faults:
                    qgc_fault = _SWARM_BUS.active_faults[a.uav_id]
                    a.fault_label = qgc_fault
                    if qgc_fault == "GPS Failure":
                        a.gps_valid = False
                    else:
                        a.gps_valid = True
                elif not inject_fault or a.uav_id != inject_uav or current_tick < inject_tick:
                    if a.fault_label != "No Failure":
                        a.fault_label = "No Failure"
                        a.gps_valid = True

        # ■■ 7. Execute each UAV's LangGraph pipeline ■■
        for i, a in enumerate(agents):
            res_dict = graphs[a.uav_id].invoke(a)
            # Update state with LangGraph outputs
            for k, v in res_dict.items():
                if hasattr(a, k):
                    setattr(a, k, v)

            # If task was delegated to a peer, transfer the actual route to the peer
            if a.delegation_status == "DELEGATED" and a.delegation_target:
                for peer in agents:
                    if peer.uav_id == a.delegation_target and "from" not in peer.active_task:
                        # Pick up the stored route from SwarmBus
                        handoff_data = None
                        with _SWARM_BUS.lock:
                            handoff_routes = getattr(_SWARM_BUS, '_pending_handoff_routes', {})
                            handoff_data = handoff_routes.pop(peer.uav_id, None)

                        if handoff_data and handoff_data.get("remaining_route"):
                            delegated_wps = handoff_data["remaining_route"]
                            from_uav = handoff_data["from"]
                            # Append delegated route to peer's existing route
                            # Insert after current waypoint so peer finishes current target first
                            insert_pos = peer.current_wp_idx + 1
                            for idx_d, wp_d in enumerate(delegated_wps):
                                peer.route_coords.insert(insert_pos + idx_d, wp_d)
                            peer.total_tasks += len(delegated_wps)
                            # Add waypoint names
                            for idx_d in range(len(delegated_wps)):
                                peer.waypoints_queue.append(f"WP-{from_uav[-1]}.D{idx_d+1}")
                            _SWARM_BUS.log_event(
                                f"■ [{peer.uav_id}] Received {len(delegated_wps)} waypoints from {from_uav} — appen
```

```
    |     ■ ded to route"
2346 |                           )
2347 |
2348 |                       peer.active_task = f"Sector A (from {a.uav_id})"
2349 |                       peer.role = "Task Reallocated"
2350 |                       peer.available = False
2351 |
2352 |         # ■■ 7.5. Immediate Master Report on Task Completion ■■
2353 |         with _SWARM_BUS.lock:
2354 |             pending_completions = list(_SWARM_BUS._pending_task_completions)
2355 |             _SWARM_BUS._pending_task_completions.clear()
2356 |
2357 |         if pending_completions:
2358 |             lead_id         = getattr(_SWARM_BUS, "current_leader", "UAV-1") or "UAV-1"
2359 |             tot_tasks_all  = sum(a.total_tasks    for a in agents)
2360 |             done_tasks_all = sum(a.tasks_completed for a in agents)
2361 |             swarm_pct      = round((done_tasks_all / max(1, tot_tasks_all)) * 100.0, 1)
2362 |             online_cnt     = sum(1 for a in agents if not a.on_ground)
2363 |             fault_cnt      = sum(1 for a in agents if a.fault_label != "No Failure")
2364 |             nfz_cnt        = sum(1 for a in agents if a.nfz_violation)
2365 |             sys_status     = "NOMINAL" if fault_cnt == 0 and nfz_cnt == 0 else "DEGRADED"
2366 |             ts_iso         = time.strftime("%Y-%m-%d %H:%M:%S UTC", time.gmtime())
2367 |
2368 |             for evt in pending_completions:
2369 |                 uid        = evt["uav_id"]
2370 |                 wp_idx     = evt["wp_idx"]
2371 |                 tasks_done = evt["tasks_done"]
2372 |                 total_t    = evt["total_tasks"]
2373 |                 agent_pct  = round((tasks_done / max(1, total_t)) * 100.0, 1)
2374 |
2375 |                 # Find completing agent for live telemetry
2376 |                 ag       = next((a for a in agents if a.uav_id == uid), None)
2377 |                 ag_pos   = f"({ag.pos_x:.1f}, {ag.pos_y:.1f}, {ag.altitude:.1f}m)" if ag else "N/A"
2378 |                 ag_batt  = f"{ag.battery_pct:.1f}%" if ag else "N/A"
2379 |                 ag_fault = ag.fault_label if ag else "No Failure"
2380 |                 ag_action= ag.final_action if ag else "N/A"
2381 |                 ag_nfz   = f"VIOLATION — {ag.nfz_nearest} ({ag.nfz_dist:.1f}m)" if (ag and ag.nfz_violation) else "
    |     ■ Clear"
2382 |                 ag_link  = f"{ag.link_quality_pct:.0f}% quality / {ag.comms_latency_ms:.1f}ms latency" if ag else "
    |     ■ N/A"
2383 |
2384 |                 # Progress bar (20 chars wide)
2385 |                 pbar_fill = int(swarm_pct / 100 * 20)
2386 |                 pbar      = "■" * pbar_fill + "■" * (20 - pbar_fill)
2387 |
2388 |                 W = 70  # inner content width
2389 |                 def _r(lbl, val):
2390 |                     row = f"  {lbl:<28}{val}"
2391 |                     return f"■ {row:<{W}} ■"
2392 |                 def _div():
2393 |                     return f"■{'■'*(W+2)}■"
2394 |                 def _sep():
2395 |                     return f"■ {'·'*W} ■"
2396 |                 hdr = f"■{'■'*(W+2)}■"
2397 |                 ftr = f"■{'■'*(W+2)}■"
2398 |                 blank = f"■ {' '*W} ■"
2399 |
2400 |                 lines = [
2401 |                     "",
2402 |                     hdr,
2403 |                     f"■  {'■  MASTER UAV — OPERATIONAL TASK REPORT':<{W}} ■",
2404 |                     f"■  {'Reported by: ' + lead_id + '   ·   System: ' + sys_status + '   ·   ' + ts_iso:<{W}} ■",
2405 |                     f"■  {'Simulation Tick: ' + str(current_tick) + '   ·   Backend: ' + ('PX4_SITL+GAZEBO' if _SWA
    |     ■ RM_BUS.is_connected_to_px4() else 'STANDALONE_SIM'):<{W}} ■",
2406 |                     _div(),
2407 |                     f"■  {'■  COMPLETING AGENT — LIVE TELEMETRY':<{W}} ■",
2408 |                     _sep(),
2409 |                     _r("Agent ID:", uid),
2410 |                     _r("Waypoint Completed:", f"WP-{wp_idx}"),
2411 |                     _r("Agent Task Progress:", f"{tasks_done} / {total_t}  ({agent_pct}%)"),
2412 |                     _r("Current Position (X,Y,Z):", ag_pos),
2413 |                     _r("Battery Level:", ag_batt),
2414 |                     _r("Fault / Health Status:", ag_fault),
2415 |                     _r("Current Flight Action:", ag_action),
2416 |                     _r("NFZ Geofence Status:", ag_nfz),
2417 |                     _r("Mesh Link Quality:", ag_link),
2418 |                     _div(),
2419 |                     f"■  {'■  SWARM MISSION SUMMARY':<{W}} ■",
2420 |                     _sep(),
2421 |                     _r("Swarm Leader:", lead_id),
2422 |                     _r("UAVs Online / Total:", f"{online_cnt} / {len(agents)}"),
2423 |                     _r("Active Fault Count:", f"{fault_cnt} UAV(s) with sensor fault"),
2424 |                     _r("NFZ Violations:", f"{nfz_cnt} active violation(s)"),
2425 |                     _r("Tasks Completed / Total:", f"{done_tasks_all} / {tot_tasks_all}"),
2426 |                     _r("Overall Progress:", f"[{pbar}] {swarm_pct}%"),
2427 |                     _div(),
2428 |                     f"■  {'■  PER-AGENT STATUS TABLE':<{W}} ■",
2429 |                     _sep(),
2430 |                 ]
2431 |                 for a in agents:
2432 |                     icon   = "■" if a.uav_id == lead_id else ("■" if a.fault_label != "No Failure" else "✈")
2433 |                     marker = " ■ COMPLETED NOW" if a.uav_id == uid else ""
2434 |                     bfill  = int(a.battery_pct / 100 * 8)
2435 |                     bbr    = "■" * bfill + "■" * (8 - bfill)
2436 |                     row = (
2437 |                         f"{icon} {a.uav_id:<6}  "
2438 |                         f"WP {a.current_wp_idx+1:>2}/{len(a.route_coords):<2}  "
2439 |                         f"Tasks {a.tasks_completed}/{a.total_tasks}  "
2440 |                         f"[{bbr}]{a.battery_pct:5.1f}%  "
2441 |                         f"Alt {a.altitude:5.1f}m  "
2442 |                         f"{a.fault_label:<20}{marker}"
2443 |                     )
2444 |                     lines.append(f"■   {row:<{W-1}} ■")
2445 |
2446 |                 lines += [blank, ftr, ""]
2447 |                 print("\033[93m" + "\n".join(lines) + "\033[0m")
```

```
                # ■■ Immediate JSON report export with embedded task event ■■
                export_master_system_report(agents, current_tick,
                    task_event={
                        "event_type": "TASK_COMPLETION",
                        "completing_agent": uid,
                        "waypoint_completed": wp_idx,
                        "agent_tasks_done": tasks_done,
                        "agent_total_tasks": total_t,
                        "agent_progress_pct": agent_pct,
                        "swarm_progress_pct": swarm_pct,
                        "timestamp_iso": ts_iso,
                        "reported_by_master": lead_id,
                    }
                )

                # ■■ QGC STATUSTEXT for this specific completion ■■
                if _MAVLINK_AVAILABLE and lead_id in _SWARM_BUS._mav_conns:
                    mav_m   = _SWARM_BUS._mav_conns[lead_id]
                    qgc_txt = f"[MASTER] {uid} WP-{wp_idx} OK {tasks_done}/{total_t} | Swarm {swarm_pct}%"
                    try:
                        mav_m.mav.statustext_send(
                            mavutil.mavlink.MAV_SEVERITY_INFO,
                            qgc_txt.encode("utf-8")[:50],
                        )
                    except Exception:
                        pass

        # ■■ 8. Distributed Swarm Leader Re-Election ■■
        _SWARM_BUS.evaluate_leader_election(agents)

        # ■■ 9. Render combined swarm dashboard ■■
        render_swarm_dashboard(agents, current_tick)

        # ■■ 10. Sync 3D poses to Gazebo world so all quadrotors fly in Gazebo ■■
        _SWARM_BUS.sync_gazebo_swarm(agents)

        # ■■ 11. Master System Report Export (written to disk every 2 ticks) ■■
        if current_tick % 2 == 0 or current_tick == 1:
            export_master_system_report(agents, current_tick)

        # ■■ 12. Master UAV periodic STATUSTEXT update to QGroundControl ■■
        lead_id = getattr(_SWARM_BUS, "current_leader", "UAV-1") or "UAV-1"
        if _MAVLINK_AVAILABLE and lead_id in _SWARM_BUS._mav_conns and current_tick % 16 == 0:
            mav_m = _SWARM_BUS._mav_conns[lead_id]
            tot_tasks = sum(a.total_tasks for a in agents)
            done_tasks = sum(a.tasks_completed for a in agents)
            p_pct = int((done_tasks / max(1, tot_tasks)) * 100)
            active_cnt = sum(1 for a in agents if not a.on_ground)
            msg_txt = f"■ [MASTER] Progress: {p_pct}% ({done_tasks}/{tot_tasks} Tasks) | {active_cnt}/{len(agents)} ■ Online"
            try:
                mav_m.mav.statustext_send(mavutil.mavlink.MAV_SEVERITY_NOTICE, msg_txt.encode("utf-8")[:50])
            except Exception:
                pass


        if ticks > 0 and current_tick >= ticks:
            print(f"\n[Done] Completed {ticks} swarm simulation ticks.")
            break

        await asyncio.sleep(0.25)


def main():
    parser = argparse.ArgumentParser(description="Multi-Agent Autonomous UAV Swarm (LangGraph + ROS2)")
    parser.add_argument("--ticks", type=int, default=0, help="Number of ticks to run (0 = continuous)")
    parser.add_argument("--inject-uav", type=str, default="UAV-1", help="Target UAV for fault injection (UAV-1..4)" ■ )
    parser.add_argument("--inject-battery", type=float, default=None, help="Inject battery percentage (e.g. 14.0)")
    parser.add_argument("--inject-fault", type=str, default=None, help="Inject sensor fault (e.g. 'GPS Failure', 'G■ yroscope Failure')")
    parser.add_argument("--inject-at-tick", type=int, default=3, help="Tick at which to trigger fault injection")
    parser.add_argument("--initial-uavs", type=int, default=4, help="Initial number of UAV agents in swarm (default ■ 4)")
    parser.add_argument("--add-uav-at-tick", nargs="*", default=[], help="Dynamically add UAVs at tick, e.g. --add-■ uav-at-tick UAV-5:5 UAV-6:5")
    parser.add_argument("--remove-uav-at-tick", nargs="*", default=[], help="Dynamically remove UAVs at tick, e.g. ■ --remove-uav-at-tick UAV-2:15")
    args = parser.parse_args()

    try:
        asyncio.run(run_swarm(
            ticks=args.ticks,
            inject_uav=args.inject_uav,
            inject_batt=args.inject_battery,
            inject_fault=args.inject_fault,
            inject_tick=args.inject_at_tick,
            add_uav_specs=args.add_uav_at_tick,
            remove_uav_specs=args.remove_uav_at_tick,
            initial_uavs=args.initial_uavs,
        ))
    except KeyboardInterrupt:
        print("\n[Swarm] Orchestrator terminated by user.")


if __name__ == "__main__":
    main()
```

Part 3.2: step76_control_barrier_function.py (CBF-QP Safety Filter)

Source File Path: `/home/samiksha/uav_fdi_data/step76_control_barrier_function.py`
Character Encoding: UTF-8 | **Preservation:** Verbatim line-numbered software deposit

```
#!/usr/bin/env python3
"""
step76_control_barrier_function.py
==================================
Formal Control Barrier Function (CBF) Safety Filter for UAVs.

Formulation:
1. Ground Impact & Descent Barrier:
   h_alt(z, vz) = (z - z_min) - (vz^2 / (2 * a_max)) >= 0
   Ensures the UAV always has enough braking distance before hitting ground.

2. Maximum Kinetic Speed Barrier:
   h_speed(v) = v_max^2 - (vx^2 + vy^2 + vz^2) >= 0

3. Inter-UAV Swarm Separation Barrier (for multi-agent collision avoidance):
   h_swarm(p_i, p_j) = ||p_i - p_j||^2 - d_min^2 >= 0
   Derivative condition: 2 * (p_i - p_j)^T * (v_i - v_j) + gamma * h_swarm >= 0

4. No-Fly Zone (NFZ) Geofence Barrier:
   h_nfz(p, c) = ||p - c||^2 - r_nfz^2 >= 0
   Repels UAV away from NFZ center c with radius r_nfz.
   Derivative condition: 2 * (p - c)^T * v + gamma * h_nfz >= 0

5. CBF-QP Safety Filter:
   Solves: min_u 0.5 * ||u - u_nom||^2
           s.t.  A_cbf * u <= b_cbf
   Using quadratic programming (or closed-form KKT when single-constraint).
"""

import numpy as np
from scipy.optimize import minimize


class UAVControlBarrierFunction:
    """
    Control Barrier Function (CBF) Runtime Assurance Filter.
    Takes agent's proposed velocity/descent command u_nom = [vx, vy, vz]
    and projects it onto the forward-invariant safe set C.
    """

    def __init__(
        self,
        z_min=0.3,           # Minimum ground clearance (meters)
        a_max=2.5,           # Maximum vertical deceleration capability (m/s^2)
        v_max=5.0,           # Maximum allowable speed (m/s)
        d_min_swarm=2.5,     # Minimum inter-UAV separation distance (meters)
        gamma=1.5,           # Class-K barrier decay rate parameter
    ):
        self.z_min = z_min
        self.a_max = a_max
        self.v_max = v_max
        self.d_min_swarm = d_min_swarm
        self.gamma = gamma

    def ground_barrier(self, altitude, vz):
        """
        h_alt(z, vz) = (z - z_min) - (vz^2 / (2 * a_max))
        z: altitude above ground (positive up)
        vz: vertical speed downward (positive descending)
        """
        if vz <= 0:
            # Ascending or hovering: pure distance barrier
            return altitude - self.z_min
        return (altitude - self.z_min) - (vz**2 / (2.0 * self.a_max))

    def swarm_separation_barrier(self, p_i, p_j):
        """
        h_swarm(p_i, p_j) = ||p_i - p_j||^2 - d_min^2
        """
        diff = np.array(p_i) - np.array(p_j)
        dist_sq = float(np.sum(diff**2))
        return dist_sq - (self.d_min_swarm**2)

    def nfz_barrier(self, current_pos_2d, nfz_center_2d, nfz_radius):
        """
        No-Fly Zone geofence barrier:
        h_nfz(p, c) = ||p - c||^2 - r_nfz^2  >=  0
        Keeps UAV outside the NFZ circle.
        """
        diff = np.array(current_pos_2d[:2]) - np.array(nfz_center_2d[:2])
        dist_sq = float(np.sum(diff**2))
        return dist_sq - (nfz_radius**2)

    def filter_control(self, u_nom, altitude, peer_positions=None, current_pos=None, nfz_zones=None):
        """
        CBF-QP Safety Filter:
        u_nom = [vx_nom, vy_nom, vz_down_nom]
        nfz_zones: list of dicts {"x": float, "y": float, "radius": float}
        Returns:
            u_safe: [vx_safe, vy_safe, vz_safe]
            cbf_active: bool (True if CBF modified the nominal proposal)
            h_val: float (current barrier value)
```

```
            slack_info: dict
        """
        u_nom = np.array(u_nom, dtype=float)

        # 1. Evaluate current ground barrier value
        h_alt = self.ground_barrier(altitude, u_nom[2])

        # If barrier is safe and speed within bounds with no swarm peers and no NFZ, pass through
        speed_nom = np.linalg.norm(u_nom)
        if h_alt > 0.5 and speed_nom <= self.v_max and not peer_positions and not nfz_zones:
            return u_nom, False, float(h_alt), "SAFE (h_alt > 0)"

        # 2. Formulate CBF-QP optimization:
        # min 0.5 * ||u - u_nom||^2
        def objective(u):
            return 0.5 * np.sum((u - u_nom) ** 2)

        def grad_objective(u):
            return u - u_nom

        constraints = []

        # Constraint 1: Ground CBF Lie derivative condition
        # dot(h_alt) + gamma * h_alt >= 0
        def cbf_ground_constraint(u):
            vz = u[2]
            # dh/dt = -vz - (vz / a_max) * u_acc >= -gamma * h
            # Direct kinematic velocity bound:
            if altitude <= self.z_min:
                return -vz  # must not descend
            # Maximum allowed descent velocity from barrier:
            v_safe_descent_sq = max(0.0, 2.0 * self.a_max * (altitude - self.z_min))
            v_max_descent = np.sqrt(v_safe_descent_sq)
            return v_max_descent - vz

        constraints.append({'type': 'ineq', 'fun': cbf_ground_constraint})

        # Constraint 2: Kinetic velocity limit
        def speed_constraint(u):
            return self.v_max - np.linalg.norm(u)

        constraints.append({'type': 'ineq', 'fun': speed_constraint})

        # Constraint 3: Swarm peer separation barriers (if peers present)
        if peer_positions and current_pos is not None:
            p_i = np.array(current_pos)
            for p_j in peer_positions:
                p_j = np.array(p_j)
                diff = p_i - p_j
                dist_sq = np.sum(diff**2)
                h_ij = dist_sq - (self.d_min_swarm**2)

                def make_swarm_constraint(d_vec, h_val):
                    def cbf_swarm(u):
                        # dot(h) + gamma * h = 2 * (p_i - p_j)^T * v_i + gamma * h >= 0
                        return 2.0 * np.dot(d_vec, u) + self.gamma * h_val
                    return cbf_swarm

                constraints.append({'type': 'ineq', 'fun': make_swarm_constraint(diff, h_ij)})

        # Constraint 4: No-Fly Zone (NFZ) geofence barriers
        if nfz_zones and current_pos is not None:
            p_i_2d = np.array(current_pos[:2])
            for nfz in nfz_zones:
                c_2d = np.array([nfz["x"], nfz["y"]])
                r_nfz = nfz["radius"]
                diff_nfz = p_i_2d - c_2d
                h_nfz = float(np.sum(diff_nfz**2)) - r_nfz**2

                def make_nfz_constraint(d_vec, h_val):
                    def cbf_nfz(u):
                        # dot(h_nfz) + gamma * h_nfz = 2*(p-c)^T * v + gamma * h >= 0
                        return 2.0 * np.dot(d_vec, u[:2]) + self.gamma * h_val
                    return cbf_nfz

                constraints.append({'type': 'ineq', 'fun': make_nfz_constraint(diff_nfz, h_nfz)})

        # Solve QP
        res = minimize(
            objective,
            x0=u_nom,
            jac=grad_objective,
            constraints=constraints,
            method='SLSQP',
            options={'ftol': 1e-4, 'maxiter': 50}
        )

        if res.success:
            u_safe = res.x
            deviation = np.linalg.norm(u_safe - u_nom)
            cbf_active = deviation > 0.05
            status = "CBF INTERVENED" if cbf_active else "CBF INVARIANT (SAFE)"
            return u_safe, cbf_active, float(h_alt), status
        else:
            # Conservative fallback if optimization fails
            u_safe = np.copy(u_nom)
            u_safe[2] = min(u_safe[2], 0.5)  # hard-cap descent to 0.5 m/s
            return u_safe, True, float(h_alt), "CBF CONSERVATIVE FALLBACK"


# Self-test when run directly
if __name__ == "__main__":
    cbf = UAVControlBarrierFunction(z_min=0.5, a_max=2.0, v_max=4.0)

    print("================================================")
    print("CONTROL BARRIER FUNCTION (CBF) UNIT VALIDATION")
```

```
    print("==================================================")

    # Test 1: Safe descent high altitude
    u_nom = np.array([1.0, 0.0, 1.0])  # vx=1, vy=0, vz_down=1
    u_safe, active, h, status = cbf.filter_control(u_nom, altitude=10.0)
    print(f"Test 1 (High Alt = 10m): u_nom={u_nom} -> u_safe={np.round(u_safe, 2)} | Status: {status} (h={h:.2f})")

    # Test 2: Unsafe aggressive descent near ground
    u_nom_unsafe = np.array([0.0, 0.0, 3.5])  # aggressive descent near ground!
    u_safe, active, h, status = cbf.filter_control(u_nom_unsafe, altitude=1.0)
    print(f"Test 2 (Low Alt = 1.0m, unsafe vz=3.5): u_nom={u_nom_unsafe} -> u_safe={np.round(u_safe, 2)} | Status:
    ■ {status} (h={h:.2f})")

    # Test 3: Close to peer collision
    p_i = [5.0, 5.0, 2.0]
    p_j = [6.0, 5.0, 2.0]  # only 1.0m away! (d_min is 2.5m)
    u_nom_towards = [1.0, 0.0, 0.0]  # flying towards peer
    u_safe, active, h, status = cbf.filter_control(u_nom_towards, altitude=5.0, peer_positions=[p_j], current_pos=p
    ■ _i)
    print(f"Test 3 (Peer collision risk): u_nom={u_nom_towards} -> u_safe={np.round(u_safe, 2)} | Status: {status}"
    ■ )
    print("==================================================")
    print("CBF MODULE VERIFIED.")
```

Part 3.3: mission_control.py (Tornado Asynchronous Web Server)

Source File Path: `/home/samiksha/uav_fdi_data/mission_control.py`
Character Encoding: UTF-8 | **Preservation:** Verbatim line-numbered software deposit

```
#!/usr/bin/env python3
"""
================================================================================
mission_control.py — Unified Web Ground Control & Multi-Agent Swarm Mission Center
================================================================================
Provides a single, all-in-one web interface to:
  1. Configure and launch autonomous swarm missions in one go (Fleet size, Faults, Tasks)
  2. View live 2D Swarm Radar, No-Fly Zones, and animated drone waypoints
  3. Real-time telemetry cards (Battery, FDI status, CBF Guardrails, Roles)
  4. One-click in-flight command console (Takeoff, Land, Inject Faults, Recover, Elect Leader)
  5. Live Master System Telemetry & Event stream
  6. Instant scientific evaluation graphs and mission summary report
================================================================================
"""

import os
import sys
import json
import time
import subprocess
import threading
import tornado.ioloop
import tornado.web
import tornado.websocket
import tornado.httpserver

PORT = 8080
BASE_DIR = "/home/samiksha/uav_fdi_data"
REPORT_PATH = os.path.join(BASE_DIR, "master_system_report.json")
CMD_FILE = "/tmp/swarm_cmd"
CMD_FILE_LOCAL = os.path.join(BASE_DIR, "swarm_cmd.txt")
PYTHON_BIN = os.path.join(BASE_DIR, "fdi_env/bin/python3")

swarm_process = None
process_lock = threading.Lock()
connected_websockets = set()


def is_swarm_running():
    global swarm_process
    if swarm_process is not None and swarm_process.poll() is None:
        return True
    try:
        res = subprocess.run(["pgrep", "-f", "step80_multiagent_swarm.py"], capture_output=True, text=True)
        return res.returncode == 0 and bool(res.stdout.strip())
    except Exception:
        return False


def send_swarm_command(cmd_text: str):
    """Writes command to both cmd files for maximum compatibility."""
    for p in (CMD_FILE, CMD_FILE_LOCAL):
        try:
            with open(p, "w") as f:
                f.write(cmd_text + "\n")
        except Exception:
            pass


class MainHandler(tornado.web.RequestHandler):
    def get(self):
        index_file = os.path.join(BASE_DIR, "web_gui", "index.html")
        if os.path.exists(index_file):
            with open(index_file, "r") as f:
                self.write(f.read())
        else:
            self.write("<h1>Mission Control Interface Loading...</h1>")


class TelemetryAPIHandler(tornado.web.RequestHandler):
    def get(self):
        running = is_swarm_running()
        data = {
            "status": "RUNNING" if running else "IDLE",
            "is_running": running,
            "telemetry": None
        }
        if os.path.exists(REPORT_PATH):
            try:
                with open(REPORT_PATH, "r") as f:
                    data["telemetry"] = json.load(f)
            except Exception:
                pass
        self.set_header("Content-Type", "application/json")
        self.write(json.dumps(data))


class StartMissionAPIHandler(tornado.web.RequestHandler):
    def post(self):
        global swarm_process
        try:
            req = json.loads(self.request.body.decode("utf-8")) if self.request.body else {}
```

```
        except Exception:
            req = {}

        fleet_size = int(req.get("fleet_size", 4))
        fault_scenario = req.get("fault_scenario", "none")
        ticks = int(req.get("ticks", 0))

        with process_lock:
            # If already running, stop previous first
            if swarm_process is not None and swarm_process.poll() is None:
                swarm_process.terminate()
                try:
                    swarm_process.wait(timeout=2)
                except Exception:
                    swarm_process.kill()

            cmd = [
                PYTHON_BIN,
                os.path.join(BASE_DIR, "step80_multiagent_swarm.py"),
                "--initial-uavs", str(fleet_size),
                "--ticks", str(ticks)
            ]

            if fault_scenario == "gps_uav2":
                cmd.extend(["--inject-uav", "UAV-2", "--inject-fault", "GPS Failure", "--inject-at-tick", "25"])
            elif fault_scenario == "batt_uav1":
                cmd.extend(["--inject-uav", "UAV-1", "--inject-battery", "14.0", "--inject-at-tick", "20"])
            elif fault_scenario == "gyro_uav3":
                cmd.extend(["--inject-uav", "UAV-3", "--inject-fault", "Gyroscope Failure", "--inject-at-tick", "30
    ■ "])

            try:
                # Start swarm in background
                swarm_process = subprocess.Popen(
                    cmd,
                    cwd=BASE_DIR,
                    stdout=subprocess.DEVNULL,
                    stderr=subprocess.DEVNULL
                )
                self.write(json.dumps({"success": True, "message": f"Mission launched with {fleet_size} UAVs."}))
            except Exception as e:
                self.write(json.dumps({"success": False, "error": str(e)}))


class StopMissionAPIHandler(tornado.web.RequestHandler):
    def post(self):
        global swarm_process
        with process_lock:
            if swarm_process is not None and swarm_process.poll() is None:
                send_swarm_command("land")
                time.sleep(0.5)
                swarm_process.terminate()
                try:
                    swarm_process.wait(timeout=2)
                except Exception:
                    swarm_process.kill()
                swarm_process = None
                self.write(json.dumps({"success": True, "message": "Mission terminated."}))
            else:
                self.write(json.dumps({"success": True, "message": "No mission was actively running."}))


class CommandAPIHandler(tornado.web.RequestHandler):
    def post(self):
        try:
            req = json.loads(self.request.body.decode("utf-8")) if self.request.body else {}
        except Exception:
            req = {}
        cmd_text = req.get("command", "").strip()
        running = is_swarm_running()
        if cmd_text:
            send_swarm_command(cmd_text)
            self.write(json.dumps({
                "success": True,
                "sent": cmd_text,
                "is_running": running,
                "warning": None if running else "Swarm is not actively running. Start mission to execute commands."
            }))
        else:
            self.write(json.dumps({"success": False, "error": "Empty command"}))


class GeneratePlotsAPIHandler(tornado.web.RequestHandler):
    def post(self):
        try:
            res = subprocess.run(
                [PYTHON_BIN, os.path.join(BASE_DIR, "plot_results.py")],
                cwd=BASE_DIR,
                capture_output=True,
                text=True,
                timeout=15
            )
            self.write(json.dumps({
                "success": res.returncode == 0,
                "output": res.stdout,
                "plots": [
                    "/plots/swarm_3d_flight_trajectories.png",
                    "/plots/swarm_fdi_safety_timeline.png",
                    "/plots/swarm_mesh_performance.png"
                ]
            }))
        except Exception as e:
            self.write(json.dumps({"success": False, "error": str(e)}))


class TelemetryWebSocket(tornado.websocket.WebSocketHandler):
```

```
    def open(self):
        connected_websockets.add(self)

    def on_close(self):
        connected_websockets.discard(self)

    def check_origin(self, origin):
        return True


def broadcast_telemetry_tick():
    """Executed directly on Tornado IOLoop every 250ms — sends live JSON to all active WebSockets."""
    if not connected_websockets:
        return
    if os.path.exists(REPORT_PATH):
        try:
            with open(REPORT_PATH, "r") as f:
                data = json.load(f)
            payload = json.dumps({"is_running": is_swarm_running(), "data": data})
            dead = set()
            for ws in list(connected_websockets):
                try:
                    if ws.ws_connection and not ws.ws_connection.is_closing():
                        ws.write_message(payload)
                    else:
                        dead.add(ws)
                except Exception:
                    dead.add(ws)
            connected_websockets.difference_update(dead)
        except Exception:
            pass


class DownloadFileHandler(tornado.web.RequestHandler):
    def get(self, ftype):
        if ftype == "json" and os.path.exists(REPORT_PATH):
            self.set_header("Content-Type", "application/json")
            self.set_header("Content-Disposition", "attachment; filename=master_system_report.json")
            with open(REPORT_PATH, "rb") as f:
                self.write(f.read())
        elif ftype == "csv":
            cpath = os.path.join(BASE_DIR, "flight_telemetry_log.csv")
            if os.path.exists(cpath):
                self.set_header("Content-Type", "text/csv")
                self.set_header("Content-Disposition", "attachment; filename=flight_telemetry_log.csv")
                with open(cpath, "rb") as f:
                    self.write(f.read())
            else:
                self.set_status(404)
                self.write("CSV log not created yet.")
        else:
            self.set_status(404)


def make_app():
    return tornado.web.Application([
        (r"/", MainHandler),
        (r"/api/telemetry", TelemetryAPIHandler),
        (r"/api/start", StartMissionAPIHandler),
        (r"/api/stop", StopMissionAPIHandler),
        (r"/api/command", CommandAPIHandler),
        (r"/api/generate_plots", GeneratePlotsAPIHandler),
        (r"/api/download/(json|csv)", DownloadFileHandler),
        (r"/ws/telemetry", TelemetryWebSocket),
        (r"/plots/(.*)", tornado.web.StaticFileHandler, {"path": os.path.join(BASE_DIR, "plots")}),
    ])


def main():
    print("=" * 70)
    print("■ STARTING AUTONOMOUS UAV SWARM MISSION CONTROL CENTER")
    print("=" * 70)

    app = make_app()
    bound_port = None
    for port in [PORT, 8081, 8082, 8085, 8090]:
        try:
            app.listen(port, address="0.0.0.0")
            bound_port = port
            break
        except Exception:
            continue

    if bound_port is None:
        bound_port = PORT
        app.listen(bound_port)

    print(f"■ Web Interface LIVE at: http://localhost:{bound_port}")
    print("=" * 70)

    # Start native periodic telemetry broadcast on Tornado main loop
    tornado.ioloop.PeriodicCallback(broadcast_telemetry_tick, 250).start()

    tornado.ioloop.IOLoop.current().start()


if __name__ == "__main__":
    main()
```

Part 3.4: web_gui/index.html (3D/2D Mission Control Dashboard)

Source File Path: `/home/samiksha/uav_fdi_data/web_gui/index.html`
Character Encoding: UTF-8 | **Preservation:** Verbatim line-numbered software deposit

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>UAV Swarm — Autonomous 3D Mission Control Center</title>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
  <!-- Three.js & OrbitControls for Real-time 3D Swarm Theater -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/three@0.128.0/examples/js/controls/OrbitControls.js"></script>
  <style>
    :root {
      --bg-dark: #0f172a;
      --card-bg: #1e293b;
      --card-border: #334155;
      --accent-blue: #38bdf8;
      --accent-gold: #facc15;
      --accent-green: #22c55e;
      --accent-red: #ef4444;
      --accent-orange: #f97316;
      --text-main: #f8fafc;
      --text-muted: #94a3b8;
    }
    * { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI', system-ui, -apple-system, sans-seri
    ■ f; }
    body { background: var(--bg-dark); color: var(--text-main); height: 100vh; display: flex; flex-direction: colum
    ■ n; overflow: hidden; }

    /* Top Navigation Bar */
    header {
      background: #090d16;
      border-bottom: 1px solid var(--card-border);
      padding: 10px 24px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      z-index: 20;
    }
    .brand { display: flex; align-items: center; gap: 12px; font-size: 1.25rem; font-weight: 700; color: var(--text
    ■ -main); }
    .brand i { color: var(--accent-blue); font-size: 1.4rem; }
    .top-stats { display: flex; gap: 14px; align-items: center; }
    .stat-badge {
      background: rgba(30, 41, 59, 0.8);
      border: 1px solid var(--card-border);
      padding: 6px 14px;
      border-radius: 8px;
      font-size: 0.85rem;
      display: flex;
      align-items: center;
      gap: 8px;
      font-weight: 600;
    }
    .badge-nominal { color: var(--accent-green); border-color: rgba(34, 197, 94, 0.4); }
    .badge-degraded { color: var(--accent-red); border-color: rgba(239, 68, 68, 0.4); animation: pulseRed 2s infini
    ■ te; }
    @keyframes pulseRed {
      0%, 100% { box-shadow: 0 0 0 0 rgba(239, 68, 68, 0.4); }
      50% { box-shadow: 0 0 10px 2px rgba(239, 68, 68, 0.5); }
    }

    /* Main Workspace Layout */
    .workspace {
      display: grid;
      grid-template-columns: 320px 1fr 380px;
      gap: 12px;
      padding: 12px;
      flex: 1;
      overflow: hidden;
    }

    /* Common Card Styling */
    .panel {
      background: var(--card-bg);
      border: 1px solid var(--card-border);
      border-radius: 12px;
      display: flex;
      flex-direction: column;
      overflow: hidden;
    }
    .panel-header {
      background: rgba(15, 23, 42, 0.6);
      padding: 10px 16px;
      border-bottom: 1px solid var(--card-border);
      font-size: 0.92rem;
      font-weight: 600;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
    .panel-body { padding: 14px; overflow-y: auto; flex: 1; }

```

```
    /* Left Panel: Mission Setup */
    .form-group { margin-bottom: 12px; }
    .form-group label { display: block; font-size: 0.8rem; color: var(--text-muted); margin-bottom: 5px; }
    select, input {
      width: 100%;
      background: #090d16;
      border: 1px solid var(--card-border);
      border-radius: 8px;
      padding: 9px 12px;
      color: var(--text-main);
      font-size: 0.88rem;
      outline: none;
    }
    select:focus, input:focus { border-color: var(--accent-blue); }

    .btn {
      width: 100%;
      padding: 11px;
      border-radius: 8px;
      border: none;
      font-weight: 600;
      font-size: 0.92rem;
      cursor: pointer;
      display: flex;
      align-items: center;
      justify-content: center;
      gap: 8px;
      transition: all 0.2s;
    }
    .btn-primary { background: linear-gradient(135deg, #0284c7, #0369a1); color: white; margin-bottom: 8px; }
    .btn-primary:hover { background: linear-gradient(135deg, #0369a1, #075985); }
    .btn-danger { background: #dc2626; color: white; }
    .btn-danger:hover { background: #b91c1c; }
    .btn-action { background: #334155; color: white; padding: 8px 10px; font-size: 0.8rem; border-radius: 6px; marg
    ■ in-bottom: 6px; }
    .btn-action:hover { background: #475569; }

    /* Center: Visualizer & Master Report */
    .center-col { display: flex; flex-direction: column; gap: 10px; overflow: hidden; }
    .visualizer-container {
      flex: 1;
      position: relative;
      background: #050811;
      border-radius: 12px;
      border: 1px solid var(--card-border);
      overflow: hidden;
      min-height: 420px;
    }
    #threeContainer { width: 100%; height: 100%; position: absolute; top: 0; left: 0; display: block; }
    #radarCanvas { width: 100%; height: 100%; position: absolute; top: 0; left: 0; display: none; }

    /* Overlay Controls on 3D/2D View */
    .view-overlay {
      position: absolute;
      background: rgba(15, 23, 42, 0.85);
      backdrop-filter: blur(6px);
      padding: 6px 12px;
      border-radius: 8px;
      font-size: 0.78rem;
      border: 1px solid rgba(255, 255, 255, 0.12);
      z-index: 10;
    }

    .view-mode-toggle {
      display: flex;
      gap: 4px;
      background: rgba(9, 13, 22, 0.9);
      border: 1px solid #334155;
      border-radius: 8px;
      padding: 3px;
    }
    .view-toggle-btn {
      background: transparent;
      border: none;
      color: var(--text-muted);
      padding: 4px 10px;
      border-radius: 6px;
      font-size: 0.75rem;
      font-weight: 600;
      cursor: pointer;
      display: flex;
      align-items: center;
      gap: 5px;
      transition: all 0.2s;
    }
    .view-toggle-btn.active {
      background: var(--accent-blue);
      color: #090d16;
    }

    .cam-btn {
      background: rgba(30, 41, 59, 0.85);
      border: 1px solid var(--card-border);
      color: var(--text-main);
      padding: 4px 8px;
      border-radius: 6px;
      font-size: 0.72rem;
      cursor: pointer;
      display: flex;
      align-items: center;
      gap: 4px;
      transition: background 0.2s;
    }
    .cam-btn:hover { background: #3b82f6; }

    /* Master Report Box */
```

```
    .master-box {
      background: #0c1427;
      border: 1px solid #1e3a8a;
      border-radius: 10px;
      padding: 10px 14px;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
    .master-title { font-size: 0.85rem; color: var(--accent-gold); font-weight: 700; margin-bottom: 2px; }
    .master-detail { font-size: 0.78rem; color: var(--text-muted); }
    .progress-bar-bg { background: #1e293b; height: 8px; width: 170px; border-radius: 4px; overflow: hidden; margin
    ■ -top: 4px; }
    .progress-bar-fill { background: var(--accent-blue); height: 100%; width: 0%; transition: width 0.3s; }

    /* Right Panel: Per-UAV Telemetry Cards */
    .uav-card {
      background: #151f32;
      border: 1px solid var(--card-border);
      border-radius: 10px;
      padding: 10px 12px;
      margin-bottom: 9px;
      transition: all 0.25s;
    }
    .uav-card.fault {
      border-color: var(--accent-red);
      background: rgba(239, 68, 68, 0.08);
      animation: pulseBorder 1.5s infinite;
    }
    @keyframes pulseBorder {
      0%, 100% { border-color: rgba(239, 68, 68, 0.6); box-shadow: 0 0 0 0 rgba(239, 68, 68, 0.4); }
      50% { border-color: rgba(239, 68, 68, 1); box-shadow: 0 0 10px 2px rgba(239, 68, 68, 0.3); }
    }
    .uav-card.leader { border-color: var(--accent-gold); }
    .uav-header { display: flex; justify-content: space-between; align-items: center; margin-bottom: 6px; }
    .uav-title { font-weight: 700; font-size: 0.92rem; display: flex; align-items: center; gap: 6px; }
    .role-tag { font-size: 0.7rem; padding: 2px 6px; border-radius: 4px; background: rgba(56, 189, 248, 0.15); colo
    ■ r: var(--accent-blue); }
    .battery-track { background: #090d16; height: 6px; border-radius: 3px; overflow: hidden; margin: 5px 0; }
    .battery-fill { height: 100%; background: var(--accent-green); transition: width 0.3s; }
    .telemetry-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 5px; font-size: 0.74rem; color: var(--tex
    ■ t-muted); }
    .telemetry-grid span b { color: var(--text-main); }

    /* Event Log */
    .log-container {
      background: #090d16;
      border: 1px solid var(--card-border);
      border-radius: 8px;
      padding: 7px 12px;
      height: 95px;
      overflow-y: auto;
      font-family: monospace;
      font-size: 0.74rem;
    }
    .log-line { margin-bottom: 3px; color: #cbd5e1; }
    .log-line.fault { color: #f87171; font-weight: bold; }
    .log-line.success { color: #4ade80; }

    /* Modal for Evaluation Plots */
    .modal {
      position: fixed;
      top: 0; left: 0; width: 100%; height: 100%;
      background: rgba(0, 0, 0, 0.85);
      display: none;
      align-items: center;
      justify-content: center;
      z-index: 100;
    }
    .modal.active { display: flex; }
    .modal-content {
      background: var(--card-bg);
      border: 1px solid var(--card-border);
      border-radius: 14px;
      width: 90%;
      max-width: 1050px;
      max-height: 90vh;
      display: flex;
      flex-direction: column;
      overflow: hidden;
    }
    .modal-header { padding: 14px 20px; border-bottom: 1px solid var(--card-border); display: flex; justify-content
    ■ : space-between; align-items: center; }
    .modal-body { padding: 20px; overflow-y: auto; display: flex; flex-direction: column; gap: 20px; }
    .plot-img { width: 100%; border-radius: 8px; border: 1px solid var(--card-border); }
  </style>
</head>
<body>

  <!-- Top Header -->
  <header>
    <div class="brand">
      <i class="fa-solid fa-drone"></i>
      <span>AUTONOMOUS UAV SWARM — 3D MISSION CONTROL</span>
    </div>
    <div class="top-stats">
      <div class="stat-badge" id="backendBadge"><i class="fa-solid fa-server"></i> Standby</div>
      <div class="stat-badge badge-nominal" id="statusBadge"><i class="fa-solid fa-shield-heart"></i> Swarm: IDLE</
    ■ div>
      <div class="stat-badge" id="leaderBadge"><i class="fa-solid fa-crown"></i> Master: —</div>
    </div>
  </header>

  <!-- Main Grid Workspace -->
  <div class="workspace">

```

```
    <!-- LEFT: Mission Configuration & Flight Controls -->
    <div class="panel">
      <div class="panel-header">
        <span><i class="fa-solid fa-sliders"></i> Mission Parameters</span>
      </div>
      <div class="panel-body">
        <div class="form-group">
          <label>Fleet Size (UAV Agents)</label>
          <select id="fleetSize">
            <option value="4" selected>4 UAVs (Standard Swarm)</option>
            <option value="3">3 UAVs</option>
            <option value="2">2 UAVs</option>
            <option value="5">5 UAVs (Scaled)</option>
            <option value="6">6 UAVs (Large Swarm)</option>
          </select>
        </div>

        <div class="form-group">
          <label>Mission Profile</label>
          <select id="missionProfile">
            <option value="survey">Area Surveillance & 3D Waypoint Mapping</option>
            <option value="perimeter">Perimeter Geofence Defense</option>
            <option value="sar">Autonomous Search & Rescue</option>
          </select>
        </div>

        <div class="form-group">
          <label>Automated Fault Injection Scenario</label>
          <select id="faultScenario">
            <option value="none" selected>Nominal Flight (Zero Injected Faults)</option>
            <option value="gps_uav2">Inject GPS Failure on UAV-2 at 25s</option>
            <option value="batt_uav1">Deplete UAV-1 Battery (Triggers Leader Election)</option>
            <option value="gyro_uav3">Inject Gyroscope Failure on UAV-3 at 30s</option>
          </select>
        </div>

        <button class="btn btn-primary" id="btnStart" onclick="startMission()">
          <i class="fa-solid fa-rocket"></i> INITIATE MISSION
        </button>
        <button class="btn btn-danger" id="btnStop" onclick="stopMission()">
          <i class="fa-solid fa-hand"></i> EMERGENCY LAND & STOP
        </button>

        <hr style="border: 0; border-top: 1px solid var(--card-border); margin: 12px 0;">

        <!-- Quick Flight Control Actions including Return to Home -->
        <div style="font-size: 0.85rem; font-weight: 700; color: var(--accent-gold); margin-bottom: 6px;">
          <i class="fa-solid fa-gamepad"></i> SWARM FLIGHT COMMANDS
        </div>
        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 6px; margin-bottom: 6px;">
          <button class="btn btn-action" onclick="sendCommand('takeoff')"><i class="fa-solid fa-plane-departure"></
    ■ i> Takeoff All</button>
          <button class="btn btn-action" onclick="sendCommand('land')"><i class="fa-solid fa-plane-arrival"></i> La
    ■ nd All</button>
        </div>
        <!-- Return to Home (RTL) Primary Button -->
        <button class="btn btn-action" style="background: linear-gradient(135deg, #eab308, #ca8a04); font-weight: 7
    ■ 00; color: #0f172a; margin-bottom: 12px;" onclick="triggerRTL('all')">
          <i class="fa-solid fa-house-chimney"></i> RETURN TO HOME (RTL ALL)
        </button>

        <hr style="border: 0; border-top: 1px solid var(--card-border); margin: 10px 0;">

        <!-- Advanced Fault Injection Console -->
        <div style="font-size: 0.85rem; font-weight: 700; color: var(--accent-blue); margin-bottom: 8px;">
          <i class="fa-solid fa-bolt"></i> FAULT INJECTION CONSOLE
        </div>

        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 8px; margin-bottom: 8px;">
          <div>
            <label style="font-size: 0.72rem; color: var(--text-muted);">Target Drone</label>
            <select id="faultTargetUav" style="padding: 8px 10px; font-size: 0.85rem;">
              <option value="UAV-1">UAV-1 (Master)</option>
              <option value="UAV-2" selected>UAV-2</option>
              <option value="UAV-3">UAV-3</option>
              <option value="UAV-4">UAV-4</option>
            </select>
          </div>
          <div>
            <label style="font-size: 0.72rem; color: var(--text-muted);">Fault Type</label>
            <select id="faultTypeSelect" style="padding: 8px 10px; font-size: 0.85rem;">
              <option value="GPS">■ GPS Failure</option>
              <option value="Gyroscope">■ Gyroscope Failure</option>
              <option value="Accelerometer">■ Accelerometer Failure</option>
              <option value="Barometer">■■ Barometer Failure</option>
              <option value="Compass">■ Compass Failure</option>
              <option value="batt 14">■ Low Battery (14% - Re-elect)</option>
              <option value="batt 8">■ Critical Battery (8% - Land)</option>
            </select>
          </div>
        </div>

        <div style="display: flex; gap: 8px; margin-bottom: 10px;">
          <button class="btn btn-action" style="flex: 1; background: #dc2626; color: white;" onclick="triggerCustom
    ■ Fault()">
            <i class="fa-solid fa-bolt"></i> Inject Fault
          </button>
          <button class="btn btn-action" style="flex: 1; background: #059669; color: white;" onclick="clearTargetUa
    ■ v()">
            <i class="fa-solid fa-wrench"></i> Clear Target
          </button>
        </div>

        <button class="btn btn-action" onclick="sendCommand('clear all')">
          <i class="fa-solid fa-rotate-left"></i> Restore All UAVs to 100% Nominal
        </button>
```

```
396 |
397 |         <hr style="border: 0; border-top: 1px solid var(--card-border); margin: 12px 0;">
398 |
399 |         <button class="btn btn-action" style="background: #2563eb;" onclick="openPlotsModal()">
400 |           <i class="fa-solid fa-chart-line"></i> View Scientific Evaluation Plots
401 |         </button>
402 |
403 |         <div style="display: flex; gap: 6px; margin-top: 6px;">
404 |           <a href="/api/download/json" download class="btn btn-action" style="text-decoration: none; text-align: ce
    |      ■ nter; flex: 1; background: #1e293b;">
405 |             <i class="fa-solid fa-file-code"></i> JSON Report
406 |           </a>
407 |           <a href="/api/download/csv" download class="btn btn-action" style="text-decoration: none; text-align: cen
    |      ■ ter; flex: 1; background: #1e293b;">
408 |             <i class="fa-solid fa-table"></i> CSV Flight Log
409 |           </a>
410 |         </div>
411 |       </div>
412 |     </div>
413 |
414 |     <!-- CENTER: 3D Tactical Theater & Real-Time Radar -->
415 |     <div class="center-col">
416 |       <div class="visualizer-container" id="visualizerContainer">
417 |         <!-- 3D Three.js WebGL Container -->
418 |         <div id="threeContainer"></div>
419 |         <!-- 2D Canvas Fallback View -->
420 |         <canvas id="radarCanvas"></canvas>
421 |
422 |         <!-- Top Left Map Title & Geofence Badges -->
423 |         <div class="view-overlay" style="top: 10px; left: 12px;">
424 |           <div style="font-weight: 700; margin-bottom: 3px; display: flex; align-items: center; gap: 6px;">
425 |             <i class="fa-solid fa-cube" style="color: var(--accent-blue);"></i>
426 |             <span id="arenaTitle">3D Swarm Airspace & Geofence Arena</span>
427 |           </div>
428 |           <span style="color: #facc15; font-weight: 700;">■ Base (0,0,0)</span> |
429 |           <span style="color: #ef4444;">● NFZ-Alpha Hospital (r=18m)</span> |
430 |           <span style="color: #f97316;">● NFZ-Bravo Airport (r=22m)</span>
431 |         </div>
432 |
433 |         <!-- Top Right View Mode Toggle & Display Options -->
434 |         <div class="view-overlay" style="top: 10px; right: 12px; display: flex; align-items: center; gap: 10px;">
435 |           <div class="view-mode-toggle">
436 |             <button id="btnView3D" class="view-toggle-btn active" onclick="setViewMode('3D')">
437 |               <i class="fa-solid fa-cube"></i> 3D Theater
438 |             </button>
439 |             <button id="btnView2D" class="view-toggle-btn" onclick="setViewMode('2D')">
440 |               <i class="fa-solid fa-map"></i> 2D Radar
441 |             </button>
442 |           </div>
443 |           <div style="display: flex; gap: 7px; font-size: 0.72rem; align-items: center;">
444 |             <label style="cursor: pointer;"><input type="checkbox" id="chkRoutes" checked onchange="toggleLayer('ro
    |      ■ utes')"> Routes</label>
445 |             <label style="cursor: pointer;"><input type="checkbox" id="chkWPs" checked onchange="toggleLayer('wps')
    |      ■ "> WPs</label>
446 |             <label style="cursor: pointer;"><input type="checkbox" id="chkTrails" checked onchange="toggleLayer('tr
    |      ■ ails')"> Trails</label>
447 |             <label style="cursor: pointer; color: #facc15; font-weight: 700;"><input type="checkbox" id="chkRTL" ch
    |      ■ ecked onchange="toggleLayer('rtl')"> RTL Paths</label>
448 |           </div>
449 |           <div id="radarZoomControls" style="display: none; gap: 5px; align-items: center; border-left: 1px solid #
    |      ■ 334155; padding-left: 8px;">
450 |             <button onclick="zoomRadar(0.18)" title="Zoom In 2D Radar" style="background: #1e293b; border: 1px soli
    |      ■ d #475569; color: #38bdf8; padding: 2px 8px; border-radius: 4px; font-weight: bold; cursor: pointer; font-size: 0.75rem;">
451 |               <i class="fa-solid fa-plus"></i>
452 |             </button>
453 |             <button onclick="zoomRadar(-0.18)" title="Zoom Out 2D Radar" style="background: #1e293b; border: 1px so
    |      ■ lid #475569; color: #38bdf8; padding: 2px 8px; border-radius: 4px; font-weight: bold; cursor: pointer; font-size: 0.75rem;">
454 |               <i class="fa-solid fa-minus"></i>
455 |             </button>
456 |             <button onclick="resetRadarZoom()" title="Reset 2D Radar View & Center" style="background: #1e293b; bor
    |      ■ der: 1px solid #475569; color: #94a3b8; padding: 2px 8px; border-radius: 4px; font-size: 0.70rem; cursor: pointer;">
457 |               <i class="fa-solid fa-rotate-left"></i> Fit All
458 |             </button>
459 |             <span id="zoomLevelDisplay" style="color: #94a3b8; font-size: 0.70rem; font-family: monospace; font-wei
    |      ■ ght: 600;">100%</span>
460 |           </div>
461 |         </div>
462 |
463 |         <!-- Bottom Left Route Color Legend & RTL Status -->
464 |         <div class="view-overlay" style="bottom: 10px; left: 12px; font-size: 0.72rem; line-height: 1.4;">
465 |           <span style="color: #facc15;">■ UAV-1: Sector Alpha (Survey)</span><br>
466 |           <span style="color: #38bdf8;">■ UAV-2: Comms Relay & Recon</span><br>
467 |           <span style="color: #c084fc;">■ UAV-3: Sector Bravo (Airport)</span><br>
468 |           <span style="color: #34d399;">■ UAV-4: Perimeter Defense</span><br>
469 |           <span style="color: #fbbf24; font-weight: 700;">■ Yellow Dashed: Inbound RTL Recovery Path</span>
470 |         </div>
471 |
472 |         <!-- Bottom Right 3D Camera Presets -->
473 |         <div id="cameraControls3D" style="position: absolute; bottom: 10px; right: 12px; display: flex; gap: 6px; z
    |      ■ -index: 10;">
474 |           <button class="cam-btn" onclick="setCamPreset('orbit')"><i class="fa-solid fa-compass"></i> 3D Orbit</but
    |      ■ ton>
475 |           <button class="cam-btn" onclick="setCamPreset('home')"><i class="fa-solid fa-house"></i> Focus Home</butt
    |      ■ on>
476 |           <button class="cam-btn" onclick="setCamPreset('top')"><i class="fa-solid fa-arrow-down"></i> Top-Down</bu
    |      ■ tton>
477 |           <button class="cam-btn" onclick="setCamPreset('leader')"><i class="fa-solid fa-crown"></i> Track Master</
    |      ■ button>
478 |           <button class="cam-btn" onclick="setCamPreset('fault')"><i class="fa-solid fa-triangle-exclamation"></i>
    |      ■ Focus Fault</button>
479 |           <button class="cam-btn" onclick="reset3DCamera()"><i class="fa-solid fa-rotate-left"></i> Reset</button>
480 |         </div>
481 |       </div>
482 |
483 |       <!-- Live AI Decision Engine Pipeline Tracker -->
484 |       <div style="background: #090d16; border: 1px solid #1e3a8a; border-radius: 8px; padding: 7px 14px; font-size:
```

```
    |     ■  0.76rem; display: flex; align-items: center; justify-content: space-between;">
485 |         <span style="color: var(--accent-blue); font-weight: 700;"><i class="fa-solid fa-brain"></i> AGENTIC AI PIP
    |     ■ ELINE:</span>
486 |         <div id="aiPipelineNodes" style="display: flex; gap: 6px; align-items: center; color: var(--text-muted); fo
    |     ■ nt-size: 0.72rem;">
487 |           <span style="background: #1e293b; padding: 2px 6px; border-radius: 4px;">1. Telemetry</span> →
488 |           <span style="background: #1e293b; padding: 2px 6px; border-radius: 4px;" id="pipeFdi">2. FDI Model: Nomin
    |     ■ al</span> →
489 |           <span style="background: #1e293b; padding: 2px 6px; border-radius: 4px;" id="pipeRisk">3. Risk: 0.00</spa
    |     ■ n> →
490 |           <span style="background: #1e293b; padding: 2px 6px; border-radius: 4px;" id="pipeCbf">4. CBF: Safe Barrie
    |     ■ r</span> →
491 |           <span style="background: #1e293b; padding: 2px 6px; border-radius: 4px;" id="pipeSwarm">5. Swarm: Nominal
    |     ■ </span>
492 |         </div>
493 |       </div>
494 |
495 |       <!-- Master System Telemetry Box -->
496 |       <div class="master-box">
497 |         <div>
498 |           <div class="master-title" id="masterTitle">■ MASTER UAV TELEMETRY REPORT</div>
499 |           <div class="master-detail" id="masterDetail">Awaiting mission launch...</div>
500 |         </div>
501 |         <div>
502 |           <div style="font-size: 0.75rem; color: var(--text-muted); text-align: right;" id="progressText">0% Comple
    |     ■ te</div>
503 |           <div class="progress-bar-bg">
504 |             <div class="progress-bar-fill" id="progressBar"></div>
505 |           </div>
506 |         </div>
507 |       </div>
508 |
509 |       <!-- Live Event Log -->
510 |       <div class="log-container" id="eventLog">
511 |         <div class="log-line">System initialized. Configure mission parameters and click 'Initiate Mission'.</div>
512 |       </div>
513 |     </div>
514 |
515 |     <!-- RIGHT: Live Per-UAV Telemetry Cards -->
516 |     <div class="panel">
517 |       <div class="panel-header">
518 |         <span><i class="fa-solid fa-microchip"></i> Live Swarm Telemetry</span>
519 |         <span style="font-size: 0.8rem; color: var(--accent-blue);" id="fleetCount">0 Active</span>
520 |       </div>
521 |       <div class="panel-body" id="uavCardsContainer">
522 |         <div style="color: var(--text-muted); text-align: center; margin-top: 40px; font-size: 0.85rem;">
523 |           No active telemetry.<br>Click <b>Initiate Mission</b> to launch.
524 |         </div>
525 |       </div>
526 |     </div>
527 |
528 |   </div>
529 |
530 |   <!-- Modal for Scientific Plots -->
531 |   <div class="modal" id="plotsModal">
532 |     <div class="modal-content">
533 |       <div class="modal-header">
534 |         <span style="font-weight: 700; font-size: 1.1rem;"><i class="fa-solid fa-chart-pie"></i> Mission Scientific
    |     ■  Evaluation & Trajectory Plots</span>
535 |         <button style="background: none; border: none; color: white; font-size: 1.3rem; cursor: pointer;" onclick="
    |     ■ closePlotsModal()">&times;</button>
536 |       </div>
537 |       <div class="modal-body">
538 |         <p style="font-size: 0.85rem; color: var(--text-muted);">
539 |           High-resolution 300-DPI analysis figures generated by <code>plot_results.py</code> based on <code>flight_
    |     ■ telemetry_log.csv</code>:
540 |         </p>
541 |         <img class="plot-img" id="imgTraj" src="/plots/swarm_3d_flight_trajectories.png" alt="3D Flight Trajectorie
    |     ■ s">
542 |         <img class="plot-img" id="imgFdi" src="/plots/swarm_fdi_safety_timeline.png" alt="FDI Safety Timeline">
543 |         <img class="plot-img" id="imgMesh" src="/plots/swarm_mesh_performance.png" alt="Mesh Network Performance">
544 |       </div>
545 |     </div>
546 |   </div>
547 |
548 |   <script>
549 |     // ■■■
```

```
    |     ■ z:11}, {x:-20, y:30, z:11}]
577 |       },
578 |       "UAV-4": {
579 |         name: "Perimeter Patrol",
580 |         color: "#34d399",
581 |         hexColor: 0x34d399,
582 |         alt: 10,
583 |         points: [{x:10, y:-15, z:10}, {x:35, y:-25, z:10}, {x:50, y:-10, z:10}, {x:25, y:5, z:10}, {x:10, y:-15, z:
    |     ■ 10}]
584 |       }
585 |     };
586 |
587 |     // Designated Return-to-Home touchdown slot locations on Home Base pad
588 |     const RTL_SLOTS = {
589 |       "UAV-1": { x: -3.5, y: 0.35, z: 3.5, label: "Slot 1" },
590 |       "UAV-2": { x: 3.5,  y: 0.35, z: 3.5, label: "Slot 2" },
591 |       "UAV-3": { x: -3.5, y: 0.35, z: -3.5, label: "Slot 3" },
592 |       "UAV-4": { x: 3.5,  y: 0.35, z: -3.5, label: "Slot 4" }
593 |     };
594 |
595 |     // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
596 |     // 3D Three.js Spatial Visualizer Engine
597 |     // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
598 |     let scene, camera, renderer, controls;
599 |     const droneMeshes = {};        // { "UAV-1": THREE.Group }
600 |     const waypointGroup = new THREE.Group();
601 |     const routeGroup = new THREE.Group();
602 |     const homeGroup = new THREE.Group();
603 |     const rtlVectorGroup = new THREE.Group();
604 |     let threeInitialized = false;
605 |
606 |     function initThreeScene() {
607 |       const container = document.getElementById('threeContainer');
608 |       const width = container.clientWidth || 800;
609 |       const height = container.clientHeight || 500;
610 |
611 |       scene = new THREE.Scene();
612 |       scene.background = new THREE.Color(0x050811);
613 |       scene.fog = new THREE.FogExp2(0x050811, 0.0035);
614 |
615 |       camera = new THREE.PerspectiveCamera(48, width / height, 0.5, 1000);
616 |       camera.position.set(20, 85, 125);
617 |
618 |       renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
619 |       renderer.setSize(width, height);
620 |       renderer.setPixelRatio(window.devicePixelRatio);
621 |       renderer.shadowMap.enabled = true;
622 |       container.appendChild(renderer.domElement);
623 |
624 |       controls = new THREE.OrbitControls(camera, renderer.domElement);
625 |       controls.enableDamping = true;
626 |       controls.dampingFactor = 0.06;
627 |       controls.maxPolarAngle = Math.PI / 2 - 0.02; // Keep camera above ground
628 |       controls.target.set(15, 10, -20);
629 |
630 |       // Lighting
631 |       const ambient = new THREE.AmbientLight(0xffffff, 0.75);
632 |       scene.add(ambient);
633 |
634 |       const dirLight = new THREE.DirectionalLight(0x38bdf8, 1.2);
635 |       dirLight.position.set(60, 120, 60);
636 |       scene.add(dirLight);
637 |
638 |       const secondaryLight = new THREE.DirectionalLight(0xa855f7, 0.55);
639 |       secondaryLight.position.set(-60, 50, -60);
640 |       scene.add(secondaryLight);
641 |
642 |       // Ground Aerospace Grid
643 |       const grid = new THREE.GridHelper(260, 26, 0x1e3a8a, 0x0f172a);
644 |       grid.position.y = 0;
645 |       scene.add(grid);
646 |
647 |       // Concentric Range Rings on Ground
648 |       [25, 50, 75, 100].forEach(r => {
649 |         const ringGeo = new THREE.RingGeometry(r - 0.2, r + 0.2, 64);
650 |         const ringMat = new THREE.MeshBasicMaterial({ color: 0x1e293b, side: THREE.DoubleSide });
651 |         const ringMesh = new THREE.Mesh(ringGeo, ringMat);
652 |         ringMesh.rotation.x = -Math.PI / 2;
653 |         ringMesh.position.y = 0.05;
654 |         scene.add(ringMesh);
655 |       });
656 |
657 |       // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
658 |       // HIGHLIGHTED HOME BASE / RTL RECOVERY ZONE (0, 0, 0)
659 |       // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
660 |       scene.add(homeGroup);
661 |
662 |       // 1. Helipad Main Deck (radius 10m)
663 |       const padGeo = new THREE.CylinderGeometry(10, 10, 0.35, 36);
664 |       const padMat = new THREE.MeshStandardMaterial({ color: 0x0369a1, metalness: 0.5, roughness: 0.25 });
665 |       const pad = new THREE.Mesh(padGeo, padMat);
666 |       pad.position.set(0, 0.17, 0);
667 |       homeGroup.add(pad);
668 |
669 |       // 2. Helipad Outer Safety Border (Gold Ring)
670 |       const padBorderGeo = new THREE.RingGeometry(9.6, 10.4, 36);
671 |       const padBorderMat = new THREE.MeshBasicMaterial({ color: 0xfacc15, side: THREE.DoubleSide });
672 |       const padBorder = new THREE.Mesh(padBorderGeo, padBorderMat);
673 |       padBorder.rotation.x = -Math.PI / 2;
674 |       padBorder.position.y = 0.36;
675 |       homeGroup.add(padBorder);
676 |
677 |       // 3. Inner Landing Circle (Cyan Ring)
678 |       const padInnerGeo = new THREE.RingGeometry(4.8, 5.4, 36);
679 |       const padInnerMat = new THREE.MeshBasicMaterial({ color: 0x38bdf8, side: THREE.DoubleSide });
680 |       const padInner = new THREE.Mesh(padInnerGeo, padInnerMat);
```

```
681 |       padInner.rotation.x = -Math.PI / 2;
682 |       padInner.position.y = 0.37;
683 |       homeGroup.add(padInner);
684 |
685 |       // 4. Helipad 'H' Symbol (White/Gold crossbars)
686 |       const hMat = new THREE.MeshBasicMaterial({ color: 0xffffff, side: THREE.DoubleSide });
687 |       const bar1Geo = new THREE.PlaneGeometry(0.8, 5.0);
688 |       const bar1 = new THREE.Mesh(bar1Geo, hMat);
689 |       bar1.rotation.x = -Math.PI / 2; bar1.position.set(-1.6, 0.38, 0);
690 |       homeGroup.add(bar1);
691 |
692 |       const bar2 = new THREE.Mesh(bar1Geo, hMat);
693 |       bar2.rotation.x = -Math.PI / 2; bar2.position.set(1.6, 0.38, 0);
694 |       homeGroup.add(bar2);
695 |
696 |       const barCrossGeo = new THREE.PlaneGeometry(2.4, 0.8);
697 |       const barCross = new THREE.Mesh(barCrossGeo, hMat);
698 |       barCross.rotation.x = -Math.PI / 2; barCross.position.set(0, 0.38, 0);
699 |       homeGroup.add(barCross);
700 |
701 |       // 5. Four Dedicated UAV Touchdown Slots on Helipad
702 |       const slotColors = [0xfacc15, 0x38bdf8, 0xc084fc, 0x34d399];
703 |       Object.keys(RTL_SLOTS).forEach((uid, idx) => {
704 |         const slot = RTL_SLOTS[uid];
705 |         const sRingGeo = new THREE.RingGeometry(1.0, 1.35, 20);
706 |         const sRingMat = new THREE.MeshBasicMaterial({ color: slotColors[idx], side: THREE.DoubleSide });
707 |         const sRing = new THREE.Mesh(sRingGeo, sRingMat);
708 |         sRing.rotation.x = -Math.PI / 2;
709 |         sRing.position.set(slot.x, 0.39, slot.z);
710 |         homeGroup.add(sRing);
711 |       });
712 |
713 |       // 6. High-Intensity Vertical Sky Beacon Pillar (50m Light Column)
714 |       const beaconPillarGeo = new THREE.CylinderGeometry(1.8, 1.8, 55, 24, 1, true);
715 |       const beaconPillarMat = new THREE.MeshBasicMaterial({
716 |         color: 0x38bdf8,
717 |         transparent: true,
718 |         opacity: 0.35,
719 |         side: THREE.DoubleSide,
720 |         blending: THREE.AdditiveBlending
721 |       });
722 |       const beaconPillar = new THREE.Mesh(beaconPillarGeo, beaconPillarMat);
723 |       beaconPillar.position.set(0, 27.5, 0);
724 |       homeGroup.add(beaconPillar);
725 |
726 |       // Inner Core Light Beam
727 |       const corePillarGeo = new THREE.CylinderGeometry(0.4, 0.4, 55, 16, 1, true);
728 |       const corePillarMat = new THREE.MeshBasicMaterial({
729 |         color: 0xffffff,
730 |         transparent: true,
731 |         opacity: 0.7,
732 |         side: THREE.DoubleSide,
733 |         blending: THREE.AdditiveBlending
734 |       });
735 |       const corePillar = new THREE.Mesh(corePillarGeo, corePillarMat);
736 |       corePillar.position.set(0, 27.5, 0);
737 |       homeGroup.add(corePillar);
738 |
739 |       // 7. Concentric Animated Radar Wave Rings
740 |       const pulseGeo1 = new THREE.RingGeometry(0.5, 1.2, 32);
741 |       const pulseMat1 = new THREE.MeshBasicMaterial({ color: 0x38bdf8, transparent: true, opacity: 0.8, side: THREE
    |     ■ .DoubleSide });
742 |       const pulse1 = new THREE.Mesh(pulseGeo1, pulseMat1);
743 |       pulse1.rotation.x = -Math.PI / 2;
744 |       pulse1.position.y = 0.22;
745 |       homeGroup.add(pulse1);
746 |
747 |       const pulseGeo2 = new THREE.RingGeometry(0.5, 1.2, 32);
748 |       const pulseMat2 = new THREE.MeshBasicMaterial({ color: 0xfacc15, transparent: true, opacity: 0.8, side: THREE
    |     ■ .DoubleSide });
749 |       const pulse2 = new THREE.Mesh(pulseGeo2, pulseMat2);
750 |       pulse2.rotation.x = -Math.PI / 2;
751 |       pulse2.position.y = 0.24;
752 |       homeGroup.add(pulse2);
753 |
754 |       homeGroup.userData.pulse1 = pulse1;
755 |       homeGroup.userData.pulse2 = pulse2;
756 |
757 |       // 8. Floating Holographic 3D Text Banner
758 |       const homeCanvas = document.createElement('canvas');
759 |       homeCanvas.width = 340; homeCanvas.height = 76;
760 |       const hctx = homeCanvas.getContext('2d');
761 |       hctx.fillStyle = 'rgba(3, 105, 161, 0.88)';
762 |       hctx.roundRect ? hctx.roundRect(0, 0, 340, 76, 14) : hctx.fillRect(0, 0, 340, 76);
763 |       hctx.fill();
764 |       hctx.lineWidth = 3.5; hctx.strokeStyle = '#facc15'; hctx.stroke();
765 |       hctx.font = 'bold 28px sans-serif';
766 |       hctx.fillStyle = '#ffffff';
767 |       hctx.textAlign = 'center';
768 |       hctx.fillText('■ HOME / RTL BASE (0,0)', 170, 48);
769 |
770 |       const homeTex = new THREE.CanvasTexture(homeCanvas);
771 |       const homeSpriteMat = new THREE.SpriteMaterial({ map: homeTex, transparent: true });
772 |       const homeSprite = new THREE.Sprite(homeSpriteMat);
773 |       homeSprite.scale.set(11, 2.5, 1);
774 |       homeSprite.position.set(0, 16, 0);
775 |       homeGroup.add(homeSprite);
776 |
777 |       // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
778 |       // 3D No-Fly Zones
779 |       // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
780 |       // NFZ-Alpha Hospital (x=50, y=60, r=18, alt=25) -> Three.js: X=50, Y=12.5, Z=-60
781 |       const nfz1Geo = new THREE.CylinderGeometry(18, 18, 25, 32, 1, false);
782 |       const nfz1Mat = new THREE.MeshStandardMaterial({
783 |         color: 0xef4444,
784 |         transparent: true,
```

```
785 |         opacity: 0.22,
786 |         roughness: 0.3,
787 |         side: THREE.DoubleSide
788 |       });
789 |       const nfz1 = new THREE.Mesh(nfz1Geo, nfz1Mat);
790 |       nfz1.position.set(50, 12.5, -60);
791 |       scene.add(nfz1);
792 |
793 |       const nfz1WireGeo = new THREE.CylinderGeometry(18.05, 18.05, 25.1, 24, 2, true);
794 |       const nfz1WireMat = new THREE.MeshBasicMaterial({ color: 0xef4444, wireframe: true });
795 |       const nfz1Wire = new THREE.Mesh(nfz1WireGeo, nfz1WireMat);
796 |       nfz1Wire.position.set(50, 12.5, -60);
797 |       scene.add(nfz1Wire);
798 |
799 |       // Outer Guardrail Layer 1 Detection Envelope (r=28m)
800 |       const nfz1BufGeo = new THREE.CylinderGeometry(28, 28, 25.2, 24, 2, true);
801 |       const nfz1BufMat = new THREE.MeshBasicMaterial({ color: 0xef4444, wireframe: true, transparent: true, opacity
    |     ■ : 0.35 });
802 |       const nfz1Buf = new THREE.Mesh(nfz1BufGeo, nfz1BufMat);
803 |       nfz1Buf.position.set(50, 12.5, -60);
804 |       scene.add(nfz1Buf);
805 |
806 |       // NFZ-Bravo Airport (x=-15, y=90, r=22, alt=30) -> Three.js: X=-15, Y=15, Z=-90
807 |       const nfz2Geo = new THREE.CylinderGeometry(22, 22, 30, 32, 1, false);
808 |       const nfz2Mat = new THREE.MeshStandardMaterial({
809 |         color: 0xf97316,
810 |         transparent: true,
811 |         opacity: 0.22,
812 |         roughness: 0.3,
813 |         side: THREE.DoubleSide
814 |       });
815 |       const nfz2 = new THREE.Mesh(nfz2Geo, nfz2Mat);
816 |       nfz2.position.set(-15, 15, -90);
817 |       scene.add(nfz2);
818 |
819 |       const nfz2WireGeo = new THREE.CylinderGeometry(22.05, 22.05, 30.1, 24, 2, true);
820 |       const nfz2WireMat = new THREE.MeshBasicMaterial({ color: 0xf97316, wireframe: true });
821 |       const nfz2Wire = new THREE.Mesh(nfz2WireGeo, nfz2WireMat);
822 |       nfz2Wire.position.set(-15, 15, -90);
823 |       scene.add(nfz2Wire);
824 |
825 |       // Outer Guardrail Layer 1 Detection Envelope (r=32m)
826 |       const nfz2BufGeo = new THREE.CylinderGeometry(32, 32, 30.2, 24, 2, true);
827 |       const nfz2BufMat = new THREE.MeshBasicMaterial({ color: 0xf97316, wireframe: true, transparent: true, opacity
    |     ■ : 0.35 });
828 |       const nfz2Buf = new THREE.Mesh(nfz2BufGeo, nfz2BufMat);
829 |       nfz2Buf.position.set(-15, 15, -90);
830 |       scene.add(nfz2Buf);
831 |
832 |       // Add route and RTL groups
833 |       scene.add(routeGroup);
834 |       scene.add(waypointGroup);
835 |       scene.add(rtlVectorGroup);
836 |
837 |       build3DRoutesAndWaypoints();
838 |
839 |       window.addEventListener('resize', onWindowResize3D);
840 |       threeInitialized = true;
841 |       animateThree();
842 |     }
843 |
844 |     function onWindowResize3D() {
845 |       if (!renderer || !camera) return;
846 |       const container = document.getElementById('threeContainer');
847 |       const width = container.clientWidth;
848 |       const height = container.clientHeight;
849 |       camera.aspect = width / height;
850 |       camera.updateProjectionMatrix();
851 |       renderer.setSize(width, height);
852 |     }
853 |
854 |     // Build 3D Route Ribbons & Waypoint Crystals
855 |     function build3DRoutesAndWaypoints() {
856 |       routeGroup.clear();
857 |       waypointGroup.clear();
858 |
859 |       Object.keys(PLANNED_ROUTES).forEach(uid => {
860 |         const route = PLANNED_ROUTES[uid];
861 |         const pts = route.points.map(p => new THREE.Vector3(p.x, p.z, -p.y));
862 |
863 |         // 3D Route Line
864 |         const lineGeo = new THREE.BufferGeometry().setFromPoints(pts);
865 |         const lineMat = new THREE.LineDashedMaterial({
866 |           color: route.hexColor,
867 |           dashSize: 3,
868 |           gapSize: 2,
869 |           linewidth: 2,
870 |           transparent: true,
871 |           opacity: 0.6
872 |         });
873 |         const line = new THREE.Line(lineGeo, lineMat);
874 |         line.computeLineDistances();
875 |         routeGroup.add(line);
876 |
877 |         // 3D Floating Waypoint Crystals
878 |         pts.forEach((pt, idx) => {
879 |           if (idx === pts.length - 1 && pts.length > 1) return; // skip loop closer
880 |           const octGeo = new THREE.OctahedronGeometry(1.0);
881 |           const octMat = new THREE.MeshStandardMaterial({
882 |             color: route.hexColor,
883 |             emissive: route.hexColor,
884 |             emissiveIntensity: 0.4,
885 |             metalness: 0.5,
886 |             roughness: 0.2
887 |           });
888 |           const crystal = new THREE.Mesh(octGeo, octMat);
```

```
889 |           crystal.position.copy(pt);
890 |           waypointGroup.add(crystal);
891 |
892 |           // Thin drop-beacon light to ground
893 |           const beamGeo = new THREE.CylinderGeometry(0.08, 0.08, pt.y, 8);
894 |           const beamMat = new THREE.MeshBasicMaterial({ color: route.hexColor, transparent: true, opacity: 0.25 });
895 |           const beam = new THREE.Mesh(beamGeo, beamMat);
896 |           beam.position.set(pt.x, pt.y / 2, pt.z);
897 |           waypointGroup.add(beam);
898 |         });
899 |       });
900 |     }
901 |
902 |     // Generate Full 3D Quadcopter Model
903 |     function create3DDroneMesh(uid, hexColor) {
904 |       const drone = new THREE.Group();
905 |
906 |       // Fuselage central pod
907 |       const bodyGeo = new THREE.BoxGeometry(2.4, 0.6, 2.4);
908 |       const bodyMat = new THREE.MeshStandardMaterial({ color: 0x1e293b, metalness: 0.8, roughness: 0.3 });
909 |       const body = new THREE.Mesh(bodyGeo, bodyMat);
910 |       drone.add(body);
911 |
912 |       // Top Cockpit Dome
913 |       const domeGeo = new THREE.SphereGeometry(0.85, 16, 16, 0, Math.PI * 2, 0, Math.PI / 2);
914 |       const domeMat = new THREE.MeshStandardMaterial({ color: hexColor, metalness: 0.5, roughness: 0.2, emissive: h
    |     ■ exColor, emissiveIntensity: 0.25 });
915 |       const dome = new THREE.Mesh(domeGeo, domeMat);
916 |       dome.position.y = 0.3;
917 |       drone.add(dome);
918 |
919 |       // 4 Carbon Arms (diagonal)
920 |       const armGeo = new THREE.CylinderGeometry(0.12, 0.12, 4.4, 8);
921 |       const armMat = new THREE.MeshStandardMaterial({ color: 0x0f172a, metalness: 0.9, roughness: 0.2 });
922 |
923 |       const arm1 = new THREE.Mesh(armGeo, armMat);
924 |       arm1.rotation.z = Math.PI / 2;
925 |       arm1.rotation.y = Math.PI / 4;
926 |       drone.add(arm1);
927 |
928 |       const arm2 = new THREE.Mesh(armGeo, armMat);
929 |       arm2.rotation.z = Math.PI / 2;
930 |       arm2.rotation.y = -Math.PI / 4;
931 |       drone.add(arm2);
932 |
933 |       // 4 Motors & Spinning Propellers
934 |       const rotors = [];
935 |       const offsets = [
936 |         { x: 1.5, z: 1.5 },
937 |         { x: -1.5, z: 1.5 },
938 |         { x: 1.5, z: -1.5 },
939 |         { x: -1.5, z: -1.5 }
940 |       ];
941 |
942 |       offsets.forEach(off => {
943 |         const nacelleGeo = new THREE.CylinderGeometry(0.35, 0.35, 0.4, 12);
944 |         const nacelleMat = new THREE.MeshStandardMaterial({ color: 0x334155, metalness: 0.7 });
945 |         const nacelle = new THREE.Mesh(nacelleGeo, nacelleMat);
946 |         nacelle.position.set(off.x, 0.1, off.z);
947 |         drone.add(nacelle);
948 |
949 |         const propGeo = new THREE.CylinderGeometry(1.1, 1.1, 0.04, 16);
950 |         const propMat = new THREE.MeshBasicMaterial({ color: 0x94a3b8, transparent: true, opacity: 0.55 });
951 |         const prop = new THREE.Mesh(propGeo, propMat);
952 |         prop.position.set(off.x, 0.32, off.z);
953 |         drone.add(prop);
954 |         rotors.push(prop);
955 |       });
956 |       drone.userData.rotors = rotors;
957 |
958 |       // LED Top Beacon
959 |       const beaconGeo = new THREE.SphereGeometry(0.32, 12, 12);
960 |       const beaconMat = new THREE.MeshStandardMaterial({ color: hexColor, emissive: hexColor, emissiveIntensity: 1.
    |     ■ 0 });
961 |       const beacon = new THREE.Mesh(beaconGeo, beaconMat);
962 |       beacon.position.y = 0.95;
963 |       drone.add(beacon);
964 |       drone.userData.beacon = beacon;
965 |
966 |       // Ground Altitude Drop Laser Line
967 |       const dropLineGeo = new THREE.BufferGeometry().setFromPoints([new THREE.Vector3(0, 0, 0), new THREE.Vector3(0
    |     ■ , -12, 0)]);
968 |       const dropLineMat = new THREE.LineDashedMaterial({ color: hexColor, dashSize: 0.8, gapSize: 0.5, transparent:
    |     ■  true, opacity: 0.5 });
969 |       const dropLine = new THREE.Line(dropLineGeo, dropLineMat);
970 |       drone.add(dropLine);
971 |       drone.userData.dropLine = dropLine;
972 |
973 |       // Ground Shadow Projection Disc
974 |       const shadowGeo = new THREE.RingGeometry(0.4, 1.6, 24);
975 |       const shadowMat = new THREE.MeshBasicMaterial({ color: hexColor, side: THREE.DoubleSide, transparent: true, o
    |     ■ pacity: 0.45 });
976 |       const shadow = new THREE.Mesh(shadowGeo, shadowMat);
977 |       shadow.rotation.x = -Math.PI / 2;
978 |       scene.add(shadow);
979 |       drone.userData.shadow = shadow;
980 |
981 |       // Floating 3D Text Tag
982 |       const canvasText = document.createElement('canvas');
983 |       canvasText.width = 256; canvasText.height = 64;
984 |       const ctxText = canvasText.getContext('2d');
985 |       ctxText.fillStyle = 'rgba(15, 23, 42, 0.85)';
986 |       ctxText.roundRect ? ctxText.roundRect(0, 0, 256, 64, 12) : ctxText.fillRect(0, 0, 256, 64);
987 |       ctxText.fill();
988 |       ctxText.font = 'bold 28px sans-serif';
989 |       ctxText.fillStyle = '#ffffff';
```

```
      ctxText.textAlign = 'center';
      ctxText.fillText(`${uid}`, 128, 42);

      const texture = new THREE.CanvasTexture(canvasText);
      const spriteMat = new THREE.SpriteMaterial({ map: texture, transparent: true });
      const sprite = new THREE.Sprite(spriteMat);
      sprite.scale.set(6, 1.5, 1);
      sprite.position.y = 2.4;
      drone.add(sprite);
      drone.userData.sprite = sprite;

      scene.add(drone);
      droneMeshes[uid] = drone;
      return drone;
    }

    // Three.js Render & Physics Loop
    function animateThree() {
      requestAnimationFrame(animateThree);

      const time = Date.now() * 0.003;

      // Spin propellers
      Object.values(droneMeshes).forEach(d => {
        if (d.userData.rotors) {
          d.userData.rotors.forEach(r => r.rotation.y += 0.45);
        }
      });

      // Bob crystals
      waypointGroup.children.forEach((c, i) => {
        if (c.geometry && c.geometry.type === 'OctahedronGeometry') {
          c.rotation.y += 0.015;
          c.rotation.x = Math.sin(time + i) * 0.15;
        }
      });

      // Animate Home Base expanding pulse waves
      if (homeGroup.userData.pulse1) {
        const s1 = ((time * 7) % 25);
        homeGroup.userData.pulse1.scale.set(s1 + 0.1, s1 + 0.1, 1);
        homeGroup.userData.pulse1.material.opacity = Math.max(0, 0.85 - (s1 / 25));
      }
      if (homeGroup.userData.pulse2) {
        const s2 = (((time * 7) + 12.5) % 25);
        homeGroup.userData.pulse2.scale.set(s2 + 0.1, s2 + 0.1, 1);
        homeGroup.userData.pulse2.material.opacity = Math.max(0, 0.85 - (s2 / 25));
      }

      controls.update();
      renderer.render(scene, camera);
    }

    // ═══════════════════════════════════════════════════════════════════════════
    // 2D Tactical Radar Engine (Alternate View)
    // ═══════════════════════════════════════════════════════════════════════════
    const canvas2D = document.getElementById('radarCanvas');
    const ctx2D = canvas2D.getContext('2d');

    function resizeCanvas2D() {
      canvas2D.width = canvas2D.parentElement.clientWidth;
      canvas2D.height = canvas2D.parentElement.clientHeight;
    }
    window.addEventListener('resize', resizeCanvas2D);
    resizeCanvas2D();

    let radarZoom = 1.0;
    let radarPanX = 0;
    let radarPanY = 0;
    let isPanningRadar = false;
    let panStartX = 0, panStartY = 0;

    function zoomRadar(delta) {
      radarZoom = Math.max(0.35, Math.min(3.5, Math.round((radarZoom + delta) * 100) / 100));
      const disp = document.getElementById('zoomLevelDisplay');
      if (disp) disp.innerText = Math.round(radarZoom * 100) + '%';
      render2DRadar();
    }

    function resetRadarZoom() {
      radarZoom = 1.0;
      radarPanX = 0;
      radarPanY = 0;
      const disp = document.getElementById('zoomLevelDisplay');
      if (disp) disp.innerText = '100%';
      render2DRadar();
    }

    // Mouse wheel zoom
    canvas2D.addEventListener('wheel', (e) => {
      e.preventDefault();
      const delta = e.deltaY < 0 ? 0.15 : -0.15;
      zoomRadar(delta);
    }, { passive: false });

    // Drag to Pan
    canvas2D.style.cursor = 'grab';
    canvas2D.addEventListener('mousedown', (e) => {
      isPanningRadar = true;
      panStartX = e.clientX;
      panStartY = e.clientY;
      canvas2D.style.cursor = 'grabbing';
    });

    window.addEventListener('mousemove', (e) => {
      if (!isPanningRadar || currentViewMode !== '2D') return;
```

```
      const dx = e.clientX - panStartX;
      const dy = e.clientY - panStartY;
      panStartX = e.clientX;
      panStartY = e.clientY;

      const spanX = (110 - (-70)) / radarZoom;
      const spanY = (140 - (-40)) / radarZoom;

      radarPanX -= (dx / canvas2D.width) * spanX;
      radarPanY += (dy / canvas2D.height) * spanY;
      render2DRadar();
    });

    window.addEventListener('mouseup', () => {
      if (isPanningRadar) {
        isPanningRadar = false;
        canvas2D.style.cursor = 'grab';
      }
    });

    function worldToCanvas2D(wx, wy) {
      // Full coverage bounds (-70 to 110 in X, -40 to 140 in Y) — comfortably fits all 6 UAV routes
      const baseMinX = -70, baseMaxX = 110;
      const baseMinY = -40, baseMaxY = 140;

      const midX = (baseMinX + baseMaxX) / 2.0;
      const midY = (baseMinY + baseMaxY) / 2.0;
      const spanX = (baseMaxX - baseMinX) / radarZoom;
      const spanY = (baseMaxY - baseMinY) / radarZoom;

      const minX = midX - spanX / 2.0 + radarPanX;
      const maxX = midX + spanX / 2.0 + radarPanX;
      const minY = midY - spanY / 2.0 + radarPanY;
      const maxY = midY + spanY / 2.0 + radarPanY;

      const px = ((wx - minX) / (maxX - minX)) * canvas2D.width;
      const py = canvas2D.height - (((wy - minY) / (maxY - minY)) * canvas2D.height);
      return { x: px, y: py };
    }

    function render2DRadar() {
      if (currentViewMode !== '2D') return;
      ctx2D.clearRect(0, 0, canvas2D.width, canvas2D.height);

      const showRoutes = document.getElementById('chkRoutes')?.checked ?? true;
      const showWPs = document.getElementById('chkWPs')?.checked ?? true;
      const showTrails = document.getElementById('chkTrails')?.checked ?? true;
      const showRTL = document.getElementById('chkRTL')?.checked ?? true;

      // Grid lines
      ctx2D.strokeStyle = '#1e293b';
      ctx2D.lineWidth = 1;
      for (let x = 0; x < canvas2D.width; x += 40) {
        ctx2D.beginPath(); ctx2D.moveTo(x, 0); ctx2D.lineTo(x, canvas2D.height); ctx2D.stroke();
      }
      for (let y = 0; y < canvas2D.height; y += 40) {
        ctx2D.beginPath(); ctx2D.moveTo(0, y); ctx2D.lineTo(canvas2D.width, y); ctx2D.stroke();
      }

      // Base Station Launchpad at (0, 0) with glowing pulse waves
      const basePt = worldToCanvas2D(0, 0);
      const pulseR = ((Date.now() / 40) % 28);
      ctx2D.strokeStyle = 'rgba(56, 189, 248, ' + Math.max(0, 1 - pulseR / 28) + ')';
      ctx2D.lineWidth = 2;
      ctx2D.beginPath(); ctx2D.arc(basePt.x, basePt.y, pulseR + 10, 0, 2 * Math.PI); ctx2D.stroke();

      ctx2D.fillStyle = 'rgba(2, 132, 199, 0.4)';
      ctx2D.beginPath(); ctx2D.arc(basePt.x, basePt.y, 18, 0, 2 * Math.PI); ctx2D.fill();
      ctx2D.strokeStyle = '#facc15'; ctx2D.lineWidth = 2.5; ctx2D.stroke();
      ctx2D.fillStyle = '#facc15'; ctx2D.font = 'bold 10px sans-serif';
      ctx2D.fillText('■ HOME / RTL BASE (0,0)', basePt.x - 55, basePt.y + 32);

      // Exact world-to-pixel scaling factor based on current radarZoom
      // Full world span in X is 110 - (-70) = 180 meters
      const meterToPixel = canvas2D.width / ((110 - (-70)) / radarZoom);

      // NFZ-Alpha (Hospital): Center (50, 60), r=18m, buffer=28m
      const nfz1 = worldToCanvas2D(50, 60);
      const r1 = 18.0 * meterToPixel;
      const r1_buf = (18.0 + 10.0) * meterToPixel; // Guardrail Layer 1 Detection Envelope (28m)

      // Outer Layer 1 Guardrail Envelope (Dashed red perimeter)
      ctx2D.save();
      ctx2D.strokeStyle = 'rgba(239, 68, 68, 0.55)';
      ctx2D.lineWidth = 1.5;
      ctx2D.setLineDash([5, 4]);
      ctx2D.beginPath(); ctx2D.arc(nfz1.x, nfz1.y, r1_buf, 0, 2 * Math.PI); ctx2D.stroke();
      ctx2D.fillStyle = 'rgba(239, 68, 68, 0.06)'; ctx2D.fill();
      ctx2D.restore();

      // Inner Hard NFZ No-Fly Zone
      ctx2D.fillStyle = 'rgba(239, 68, 68, 0.22)';
      ctx2D.strokeStyle = '#ef4444'; ctx2D.lineWidth = 2.2;
      ctx2D.beginPath(); ctx2D.arc(nfz1.x, nfz1.y, r1, 0, 2 * Math.PI); ctx2D.fill(); ctx2D.stroke();
      ctx2D.fillStyle = '#fca5a5'; ctx2D.font = 'bold 10px sans-serif';
      ctx2D.fillText('■ NFZ-Alpha: Hospital (18m)', nfz1.x - 65, nfz1.y - 4);
      ctx2D.fillStyle = 'rgba(252, 165, 165, 0.85)'; ctx2D.font = '9px sans-serif';
      ctx2D.fillText('■■ Guardrail Threshold: 28m', nfz1.x - 65, nfz1.y + 11);

      // NFZ-Bravo (Airport): Center (-15, 90), r=22m, buffer=32m
      const nfz2 = worldToCanvas2D(-15, 90);
      const r2 = 22.0 * meterToPixel;
      const r2_buf = (22.0 + 10.0) * meterToPixel; // Guardrail Layer 1 Detection Envelope (32m)

      // Outer Layer 1 Guardrail Envelope (Dashed orange perimeter)
      ctx2D.save();
```

```
      ctx2D.strokeStyle = 'rgba(249, 115, 22, 0.55)';
      ctx2D.lineWidth = 1.5;
      ctx2D.setLineDash([5, 4]);
      ctx2D.beginPath(); ctx2D.arc(nfz2.x, nfz2.y, r2_buf, 0, 2 * Math.PI); ctx2D.stroke();
      ctx2D.fillStyle = 'rgba(249, 115, 22, 0.06)'; ctx2D.fill();
      ctx2D.restore();

      // Inner Hard NFZ No-Fly Zone
      ctx2D.fillStyle = 'rgba(249, 115, 22, 0.22)';
      ctx2D.strokeStyle = '#f97316'; ctx2D.lineWidth = 2.2;
      ctx2D.beginPath(); ctx2D.arc(nfz2.x, nfz2.y, r2, 0, 2 * Math.PI); ctx2D.fill(); ctx2D.stroke();
      ctx2D.fillStyle = '#fed7aa'; ctx2D.font = 'bold 10px sans-serif';
      ctx2D.fillText('■ NFZ-Bravo: Airport (22m)', nfz2.x - 65, nfz2.y - 4);
      ctx2D.fillStyle = 'rgba(254, 215, 170, 0.85)'; ctx2D.font = '9px sans-serif';
      ctx2D.fillText('■■ Guardrail Threshold: 32m', nfz2.x - 65, nfz2.y + 11);

      // Planned Waypoint Routes
      Object.keys(PLANNED_ROUTES).forEach(uid => {
        const route = PLANNED_ROUTES[uid];
        const pts = route.points.map(p => worldToCanvas2D(p.x, p.y));

        if (showRoutes && pts.length > 1) {
          ctx2D.beginPath(); ctx2D.moveTo(pts[0].x, pts[0].y);
          for (let i = 1; i < pts.length; i++) ctx2D.lineTo(pts[i].x, pts[i].y);
          ctx2D.strokeStyle = route.color + '55';
          ctx2D.lineWidth = 1.8; ctx2D.setLineDash([5, 5]); ctx2D.stroke(); ctx2D.setLineDash([]);
        }

        if (showWPs) {
          pts.forEach((wp, idx) => {
            if (idx === pts.length - 1 && pts.length > 1) return;
            ctx2D.fillStyle = route.color + 'aa';
            ctx2D.beginPath(); ctx2D.arc(wp.x, wp.y, 4, 0, 2 * Math.PI); ctx2D.fill();
            ctx2D.fillStyle = '#94a3b8'; ctx2D.font = '8px sans-serif';
            ctx2D.fillText(`${idx+1}`, wp.x + 6, wp.y + 3);
          });
        }
      });

      // Drone Trails, RTL Vectors, Dynamic Reroute Vectors & Drones
      const trails = window.droneTrails = window.droneTrails || {};
      if (swarmData && swarmData.agents_telemetry) {
        swarmData.agents_telemetry.forEach(u => {
          const pt = worldToCanvas2D(u.position.x, u.position.y);
          const isLeader = u.is_leader;
          const isFdi = u.fdi_fault && u.fdi_fault !== 'No Failure';
          const isEnergy = (u.energy_alert && u.energy_alert !== 'NORMAL') || (u.battery_pct <= 15);
          const isFault = isFdi || isEnergy;
          const isRtl = u.action === 'RETURN_TO_HOME' || (u.role && u.role.includes('RTL'));
          const droneCol = isLeader ? '#facc15' : (isFault ? '#ef4444' : '#38bdf8');

          if (!trails[u.uav_id]) trails[u.uav_id] = [];
          trails[u.uav_id].push({ x: pt.x, y: pt.y });
          if (trails[u.uav_id].length > 35) trails[u.uav_id].shift();

          // Flight Trail
          if (showTrails && trails[u.uav_id].length > 1) {
            ctx2D.beginPath(); ctx2D.moveTo(trails[u.uav_id][0].x, trails[u.uav_id][0].y);
            for (let i = 1; i < trails[u.uav_id].length; i++) ctx2D.lineTo(trails[u.uav_id][i].x, trails[u.uav_id][
■ i].y);
            ctx2D.strokeStyle = isFault ? 'rgba(239, 68, 68, 0.45)' : (isLeader ? 'rgba(250, 204, 21, 0.4)' : 'rgba
■ (56, 189, 248, 0.35)');
            ctx2D.lineWidth = 2; ctx2D.setLineDash([4, 4]); ctx2D.stroke(); ctx2D.setLineDash([]);
          }

          // Dynamic Guardrail Reroute Vector & Bypass Target Visualization
          if (u.reroute_active && u.reroute_target && u.reroute_target.length === 2) {
            const bypassPt = worldToCanvas2D(u.reroute_target[0], u.reroute_target[1]);
            ctx2D.save();
            ctx2D.strokeStyle = '#06b6d4';
            ctx2D.lineWidth = 2.8;
            ctx2D.setLineDash([5, 3]);
            ctx2D.shadowColor = '#22d3ee';
            ctx2D.shadowBlur = 10;
            ctx2D.beginPath();
            ctx2D.moveTo(pt.x, pt.y);
            ctx2D.lineTo(bypassPt.x, bypassPt.y);
            ctx2D.stroke();

            // Diamond target marker at dynamic bypass waypoint
            ctx2D.fillStyle = '#22d3ee';
            ctx2D.beginPath();
            ctx2D.arc(bypassPt.x, bypassPt.y, 5, 0, 2 * Math.PI);
            ctx2D.fill();
            ctx2D.strokeStyle = '#ffffff';
            ctx2D.lineWidth = 1.5;
            ctx2D.stroke();

            ctx2D.fillStyle = '#38bdf8';
            ctx2D.font = 'bold 10px monospace';
            ctx2D.fillText(`■ GUARDRAIL BYPASS [${u.uav_id}] (${u.reroute_target[0]}, ${u.reroute_target[1]})`, byp
■ assPt.x + 8, bypassPt.y - 4);
            ctx2D.restore();
          }

          // Return to Home (RTL) Trajectory Vector
          if (showRTL) {
            ctx2D.beginPath();
            ctx2D.moveTo(pt.x, pt.y);
            ctx2D.lineTo(basePt.x, basePt.y);
            ctx2D.strokeStyle = isRtl ? '#eab308' : 'rgba(234, 179, 8, 0.35)';
            ctx2D.lineWidth = isRtl ? 2.5 : 1.2;
            ctx2D.setLineDash([6, 4]);
            ctx2D.stroke();
            ctx2D.setLineDash([]);
```

```
            // Draw Return-To-Home Arrow Label along vector
            const midX = (pt.x + basePt.x) / 2;
            const midY = (pt.y + basePt.y) / 2;
            ctx2D.fillStyle = '#eab308';
            ctx2D.font = 'bold 9px sans-serif';
            ctx2D.fillText(`■ RTL: ${u.uav_id}`, midX + 3, midY - 3);
          }

          if (isFault) {
            ctx2D.strokeStyle = 'rgba(239, 68, 68, 0.8)';
            ctx2D.lineWidth = 4;
            ctx2D.beginPath(); ctx2D.arc(pt.x, pt.y, 16, 0, 2 * Math.PI); ctx2D.stroke();
          }

          if (u.reroute_active) {
            // Pulsing cyan guardrail halo
            const guardR = 12 + ((Date.now() / 60) % 8);
            ctx2D.strokeStyle = 'rgba(6, 182, 212, 0.9)';
            ctx2D.lineWidth = 2;
            ctx2D.beginPath(); ctx2D.arc(pt.x, pt.y, guardR, 0, 2 * Math.PI); ctx2D.stroke();
          }

          ctx2D.fillStyle = droneCol;
          ctx2D.beginPath(); ctx2D.arc(pt.x, pt.y, 7, 0, 2 * Math.PI); ctx2D.fill();
          ctx2D.strokeStyle = '#ffffff'; ctx2D.lineWidth = 1.5; ctx2D.stroke();

          ctx2D.fillStyle = '#ffffff'; ctx2D.font = 'bold 11px sans-serif';
          ctx2D.fillText(`${u.uav_id} (${u.position.z.toFixed(0)}m)`, pt.x + 10, pt.y - 2);
        });
      }

      if (currentViewMode === '2D') requestAnimationFrame(render2DRadar);
    }

    // View Switcher: 3D Theater vs 2D Radar
    function setViewMode(mode) {
      currentViewMode = mode;
      const btn3D = document.getElementById('btnView3D');
      const btn2D = document.getElementById('btnView2D');
      const threeBox = document.getElementById('threeContainer');
      const canvasBox = document.getElementById('radarCanvas');
      const camControls = document.getElementById('cameraControls3D');
      const title = document.getElementById('arenaTitle');

      const zoomControls = document.getElementById('radarZoomControls');

      if (mode === '3D') {
        btn3D.classList.add('active');
        btn2D.classList.remove('active');
        threeBox.style.display = 'block';
        canvasBox.style.display = 'none';
        camControls.style.display = 'flex';
        if (zoomControls) zoomControls.style.display = 'none';
        title.innerText = '3D Swarm Airspace & Geofence Arena';
        onWindowResize3D();
      } else {
        btn2D.classList.add('active');
        btn3D.classList.remove('active');
        threeBox.style.display = 'none';
        canvasBox.style.display = 'block';
        camControls.style.display = 'none';
        if (zoomControls) zoomControls.style.display = 'flex';
        title.innerText = '2D Tactical Radar & Flight Corridors';
        resizeCanvas2D();
        render2DRadar();
      }
    }

    function toggleLayer(layer) {
      if (layer === 'routes') {
        const v = document.getElementById('chkRoutes').checked;
        routeGroup.visible = v;
      } else if (layer === 'wps') {
        const v = document.getElementById('chkWPs').checked;
        waypointGroup.visible = v;
      } else if (layer === 'rtl') {
        const v = document.getElementById('chkRTL').checked;
        rtlVectorGroup.visible = v;
      }
      if (currentViewMode === '2D') render2DRadar();
    }

    // Camera Presets
    function setCamPreset(preset) {
      if (!controls || !camera) return;
      if (preset === 'orbit') {
        camera.position.set(20, 85, 125);
        controls.target.set(15, 10, -20);
      } else if (preset === 'home') {
        camera.position.set(0, 35, 45);
        controls.target.set(0, 5, 0);
      } else if (preset === 'top') {
        camera.position.set(20, 160, -25);
        controls.target.set(20, 0, -25);
      } else if (preset === 'leader') {
        const masterId = swarmData?.swarm_master?.uav_id || 'UAV-1';
        const d = droneMeshes[masterId];
        if (d) {
          camera.position.set(d.position.x - 25, d.position.y + 20, d.position.z + 30);
          controls.target.copy(d.position);
        }
      } else if (preset === 'fault') {
        const faultUav = swarmData?.agents_telemetry?.find(a => (a.fdi_fault && a.fdi_fault !== 'No Failure') || a.
■ battery_pct <= 15);
        if (faultUav && droneMeshes[faultUav.uav_id]) {
          const d = droneMeshes[faultUav.uav_id];
```

```
1410 |           camera.position.set(d.position.x - 18, d.position.y + 12, d.position.z + 20);
1411 |           controls.target.copy(d.position);
1412 |         } else {
1413 |           addLog('■■ No faulted UAV currently active in swarm.', 'success');
1414 |         }
1415 |       }
1416 |     }
1417 |
1418 |     function reset3DCamera() {
1419 |       setCamPreset('orbit');
1420 |     }
1421 |
1422 |     // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1423 |     // Real-Time Telemetry & WebSocket Connection
1424 |     // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1425 |     function connectTelemetry() {
1426 |       const proto = window.location.protocol === 'https:' ? 'wss:' : 'ws:';
1427 |       const ws = new WebSocket(`${proto}//${window.location.host}/ws/telemetry`);
1428 |
1429 |       ws.onmessage = (evt) => {
1430 |         try {
1431 |           const msg = JSON.parse(evt.data);
1432 |           if (msg.data) {
1433 |             updateUI(msg.data, msg.is_running);
1434 |           }
1435 |         } catch (e) {}
1436 |       };
1437 |
1438 |       ws.onclose = () => {
1439 |         setTimeout(connectTelemetry, 1000);
1440 |       };
1441 |     }
1442 |     connectTelemetry();
1443 |
1444 |     // Fallback polling every 800ms
1445 |     setInterval(() => {
1446 |       fetch('/api/telemetry')
1447 |         .then(res => res.json())
1448 |         .then(res => {
1449 |           if (res.telemetry) updateUI(res.telemetry, res.is_running ?? (res.status === 'RUNNING'));
1450 |         })
1451 |         .catch(() => {});
1452 |     }, 800);
1453 |
1454 |     // Update Dashboard UI & 3D Objects
1455 |     function updateUI(data, isRunning) {
1456 |       swarmData = data;
1457 |       isMissionRunning = isRunning;
1458 |
1459 |       // Status Badges
1460 |       const health = data.swarm_health_summary;
1461 |       const statusBadge = document.getElementById('statusBadge');
1462 |
1463 |       const isFaultUav = a => (a.fdi_fault && a.fdi_fault !== 'No Failure') || (a.energy_alert && a.energy_alert !
     | ■ = 'NORMAL') || (a.battery_pct <= 15);
1464 |       const faultUav = data.agents_telemetry ? data.agents_telemetry.find(isFaultUav) : null;
1465 |       const faultCount = data.agents_telemetry ? data.agents_telemetry.filter(isFaultUav).length : 0;
1466 |
1467 |       if (!isRunning) {
1468 |         statusBadge.className = 'stat-badge badge-degraded';
1469 |         statusBadge.innerHTML = '<i class="fa-solid fa-power-off"></i> Swarm: OFFLINE (Click Initiate Mission)';
1470 |       } else if (faultCount > 0 || (health && health.system_status !== 'NOMINAL')) {
1471 |         statusBadge.className = 'stat-badge badge-degraded';
1472 |         statusBadge.innerHTML = `<i class="fa-solid fa-triangle-exclamation"></i> Swarm: DEGRADED (${faultCount} Al
     | ■ ert${faultCount > 1 ? 's' : ''})`;
1473 |       } else {
1474 |         statusBadge.className = 'stat-badge badge-nominal';
1475 |         statusBadge.innerHTML = '<i class="fa-solid fa-shield-heart"></i> Swarm: NOMINAL';
1476 |       }
1477 |
1478 |       document.getElementById('leaderBadge').innerHTML = `<i class="fa-solid fa-crown"></i> Master: ${data.swarm_ma
     | ■ ster.uav_id}`;
1479 |       document.getElementById('backendBadge').innerHTML = `<i class="fa-solid fa-server"></i> ${health ? health.sim
     | ■ ulation_backend : 'STANDALONE_SIM'}`;
1480 |       document.getElementById('fleetCount').innerText = `${health ? health.online_agents : 0}/${health ? health.tot
     | ■ al_agents : 4} Online`;
1481 |
1482 |       // Master Progress Box
1483 |       const mission = data.swarm_mission_progress;
1484 |       const cappedTasks = Math.min(mission.total_tasks_completed, mission.total_tasks_assigned);
1485 |       const cappedPct = Math.min(100.0, Math.max(0.0, mission.progress_pct));
1486 |       document.getElementById('masterDetail').innerText =
1487 |         `Master: ${data.swarm_master.uav_id} | Online: ${health.online_agents}/${health.total_agents} | Tasks: ${ca
     | ■ ppedTasks}/${mission.total_tasks_assigned}`;
1488 |       document.getElementById('progressText').innerText = `${cappedPct}% Complete`;
1489 |       document.getElementById('progressBar').style.width = `${cappedPct}%`;
1490 |
1491 |       // Update AI Decision Pipeline Banner
1492 |       const pipeFdi = document.getElementById('pipeFdi');
1493 |       const pipeRisk = document.getElementById('pipeRisk');
1494 |       const pipeCbf = document.getElementById('pipeCbf');
1495 |       const pipeSwarm = document.getElementById('pipeSwarm');
1496 |
1497 |       if (faultUav) {
1498 |         const faultDesc = (faultUav.fdi_fault && faultUav.fdi_fault !== 'No Failure')
1499 |           ? faultUav.fdi_fault
1500 |           : `Battery ${faultUav.energy_alert} (${faultUav.battery_pct}%)`;
1501 |         if (pipeFdi) { pipeFdi.style.background = '#dc2626'; pipeFdi.style.color = '#ffffff'; pipeFdi.innerText = `
     | ■ 2. FDI: ■ ${faultUav.uav_id} ${faultDesc}`; }
1502 |         if (pipeRisk) { pipeRisk.style.background = '#d97706'; pipeRisk.style.color = '#ffffff'; pipeRisk.innerText
     | ■  = `3. Risk: ■ 0.85 (Critical Mitigation)`; }
1503 |         if (pipeCbf) { pipeCbf.style.background = '#2563eb'; pipeCbf.style.color = '#ffffff'; pipeCbf.innerText = `
     | ■ 4. CBF: ■■ ${faultUav.action}`; }
1504 |         if (pipeSwarm) { pipeSwarm.style.background = '#7c3aed'; pipeSwarm.style.color = '#ffffff'; pipeSwarm.inner
     | ■ Text = `5. Swarm: ■ Dynamic Handoff Active`; }
1505 |       } else {
```

```
        if (pipeFdi) { pipeFdi.style.background = '#1e293b'; pipeFdi.style.color = '#4ade80'; pipeFdi.innerText = `■ 2. FDI Model: ■ Nominal`; }
        if (pipeRisk) { pipeRisk.style.background = '#1e293b'; pipeRisk.style.color = '#94a3b8'; pipeRisk.innerText = `3. Risk: 0.00`; }
        if (pipeCbf) { pipeCbf.style.background = '#1e293b'; pipeCbf.style.color = '#94a3b8'; pipeCbf.innerText = `■ 4. CBF: Safe Barrier`; }
        if (pipeSwarm) { pipeSwarm.style.background = '#1e293b'; pipeSwarm.style.color = '#94a3b8'; pipeSwarm.innerText = `5. Swarm: Synchronized`; }
      }

      // Render Telemetry Cards
      const container = document.getElementById('uavCardsContainer');
      container.innerHTML = '';
      data.agents_telemetry.forEach(u => {
        const isFdi = u.fdi_fault && u.fdi_fault !== 'No Failure';
        const isBatt = (u.energy_alert && u.energy_alert !== 'NORMAL') || (u.battery_pct <= 15);
        const isFault = isFdi || isBatt;
        const isRtl = u.action === 'RETURN_TO_HOME' || (u.role && u.role.includes('RTL'));
        const cardClass = isFault ? 'uav-card fault' : (u.is_leader ? 'uav-card leader' : 'uav-card');
        const icon = u.is_leader ? '■' : (isFault ? '■■' : '↝■');
        const battColor = u.battery_pct > 30 ? '#22c55e' : (u.battery_pct > 15 ? '#facc15' : '#ef4444');

        const card = document.createElement('div');
        card.className = cardClass;
        card.innerHTML = `
          <div class="uav-header">
            <span class="uav-title">${icon} ${u.uav_id}</span>
            <div style="display: flex; gap: 5px; align-items: center;">
              <span class="role-tag">${u.role}</span>
              <button onclick="triggerRTL('${u.uav_id}')" title="Order this drone to Return to Home Base" style="background: #eab308; border: none; color: #000; font-weight: 700; font-size: 0.68rem; padding: 2px 7px; border-radius: 4px; cursor: pointer;">
                <i class="fa-solid fa-house"></i> RTL
              </button>
              <button onclick="sendCommand('clear ${u.uav_id}')" title="Reset this drone to nominal" style="background: #334155; border: none; color: #f8fafc; font-size: 0.68rem; padding: 2px 7px; border-radius: 4px; cursor: pointer;">
                <i class="fa-solid fa-wrench"></i> Reset
              </button>
            </div>
          </div>
          <div class="battery-track">
            <div class="battery-fill" style="width: ${u.battery_pct}%; background: ${battColor};"></div>
          </div>
          <div class="telemetry-grid">
            <span>Pos: <b>(${u.position.x}, ${u.position.y})</b></span>
            <span>Alt: <b>${u.position.z}m</b></span>
            <span>Batt: <b style="color: ${battColor};">${u.battery_pct}% (${u.energy_alert})</b></span>
            <span>Tasks: <b>${Math.min(u.tasks_completed, u.total_tasks)}/${u.total_tasks}</b></span>
            <span>FDI: <b style="color: ${isFdi ? '#ef4444' : '#22c55e'};">${u.fdi_fault}</b></span>
            <span>Action: <b style="color: ${isRtl ? '#eab308' : (u.reroute_active ? '#06b6d4' : '#38bdf8')};">${u.reroute_active ? '■ NFZ REROUTE' : u.action}</b></span>
            <span>Mesh Link: <b>${u.link_quality_pct}% (${u.comms_latency_ms}ms)</b></span>
            <span>Geofence: <b style="color: ${u.reroute_active ? '#06b6d4' : (u.nfz_violation ? '#f97316' : '#22c55e')};">${u.reroute_active ? '■ DYNAMIC REROUTE' : (u.nfz_violation ? '■■ PROXIMITY' : '✓ CLEAR')}</b></span>
          </div>
        `;
        container.appendChild(card);
      });

      // Update 3D Scene Objects & RTL Inbound Corridors
      update3DSwarmTelemetry(data.agents_telemetry);
    }

    // Update 3D Meshes & RTL Inbound Corridors
    function update3DSwarmTelemetry(telemetryList) {
      if (!threeInitialized || !telemetryList) return;

      // Rebuild RTL vectors in 3D
      rtlVectorGroup.clear();
      const showRTL = document.getElementById('chkRTL')?.checked ?? true;

      telemetryList.forEach(u => {
        let drone = droneMeshes[u.uav_id];
        const route = PLANNED_ROUTES[u.uav_id] || { hexColor: 0x38bdf8 };
        if (!drone) {
          drone = create3DDroneMesh(u.uav_id, route.hexColor);
        }

        // Coordinate conversion: Step80 X -> 3D X, Z -> 3D Y, Y -> 3D -Z
        const targetX = u.position.x;
        const targetY = u.position.z;
        const targetZ = -u.position.y;

        // Position interpolation
        drone.position.set(targetX, targetY, targetZ);

        // Ground shadow & drop line
        if (drone.userData.shadow) {
          drone.userData.shadow.position.set(targetX, 0.05, targetZ);
        }

        if (drone.userData.dropLine) {
          const positions = drone.userData.dropLine.geometry.attributes.position.array;
          positions[0] = 0; positions[1] = 0; positions[2] = 0;
          positions[3] = 0; positions[4] = -targetY; positions[5] = 0;
          drone.userData.dropLine.geometry.attributes.position.needsUpdate = true;
          drone.userData.dropLine.computeLineDistances();
        }

        // Fault beacon & colors
        const isFdi = u.fdi_fault && u.fdi_fault !== 'No Failure';
        const isBatt = (u.energy_alert && u.energy_alert !== 'NORMAL') || (u.battery_pct <= 15);
        const isFault = isFdi || isBatt;
        const isRtl = u.action === 'RETURN_TO_HOME' || (u.role && u.role.includes('RTL'));

        if (drone.userData.beacon) {
          if (isFault) {
```

```
            drone.userData.beacon.material.color.setHex(0xef4444);
            drone.userData.beacon.material.emissive.setHex(0xef4444);
            drone.userData.beacon.material.emissiveIntensity = 2.0;
          } else if (isRtl) {
            drone.userData.beacon.material.color.setHex(0xeab308);
            drone.userData.beacon.material.emissive.setHex(0xeab308);
            drone.userData.beacon.material.emissiveIntensity = 1.8;
          } else if (u.is_leader) {
            drone.userData.beacon.material.color.setHex(0xfacc15);
            drone.userData.beacon.material.emissive.setHex(0xfacc15);
            drone.userData.beacon.material.emissiveIntensity = 1.2;
          } else {
            drone.userData.beacon.material.color.setHex(route.hexColor);
            drone.userData.beacon.material.emissive.setHex(route.hexColor);
            drone.userData.beacon.material.emissiveIntensity = 0.6;
          }
        }

        // Draw 3D Inbound RTL Return Trajectory Vector leading directly to Home Helipad Slot
        if (showRTL) {
          const slot = RTL_SLOTS[u.uav_id] || { x: 0, y: 0.35, z: 0 };
          const rtlPts = [
            new THREE.Vector3(targetX, targetY, targetZ),
            new THREE.Vector3(slot.x, slot.y, slot.z)
          ];
          const rtlGeo = new THREE.BufferGeometry().setFromPoints(rtlPts);
          const rtlMat = new THREE.LineDashedMaterial({
            color: isRtl ? 0xfacc15 : 0xeab308,
            dashSize: 2.0,
            gapSize: 1.2,
            transparent: true,
            opacity: isRtl ? 0.95 : 0.45
          });
          const rtlLine = new THREE.Line(rtlGeo, rtlMat);
          rtlLine.computeLineDistances();
          rtlVectorGroup.add(rtlLine);
        }
      });
    }

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // Action Handlers & Mission Control API
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    function startMission() {
      const fleetSize = document.getElementById('fleetSize').value;
      const faultScenario = document.getElementById('faultScenario').value;
      addLog(`Initiating autonomous swarm mission (Fleet: ${fleetSize}, Scenario: ${faultScenario})...`, 'success')
    ■ ;

      fetch('/api/start', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ fleet_size: fleetSize, fault_scenario: faultScenario, ticks: 0 })
      })
      .then(r => r.json())
      .then(res => {
        if (res.success) {
          addLog(`■ Mission active! Engine initialized with ${fleetSize} UAV agents.`, 'success');
        } else {
          addLog(`■ Launch Error: ${res.error}`, 'fault');
        }
      });
    }

    function stopMission() {
      addLog('Terminating mission and issuing emergency failsafe landing...', 'fault');
      fetch('/api/stop', { method: 'POST' });
    }

    function sendCommand(cmd) {
      fetch('/api/command', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ command: cmd })
      })
      .then(res => res.json())
      .then(res => {
        if (res.warning) {
          addLog(`■■ ${res.warning}`, 'fault');
        } else {
          addLog(`■ Command sent to Swarm: "${cmd}"`, 'success');
        }
      })
      .catch(err => {
        addLog(`■ Command failed: ${err}`, 'fault');
      });
    }

    function triggerRTL(target) {
      if (target === 'all') {
        sendCommand('rtl');
        addLog('■ SWARM RTL ACTIVATED: All drones vectoring back to Home Base (0,0)!', 'success');
      } else {
        sendCommand(`rtl ${target}`);
        addLog(`■ RTL ACTIVATED for ${target}: Inbound recovery to Home Base (0,0)!`, 'success');
      }
    }

    function triggerCustomFault() {
      const target = document.getElementById('faultTargetUav').value;
      const fType = document.getElementById('faultTypeSelect').value;

      if (fType.startsWith('batt')) {
        const parts = fType.split(' ');
        const battVal = parts[1];
        sendCommand(`batt ${target} ${battVal}`);
```

```
1709 |         addLog(`■ INJECTED BATTERY DRAIN: Setting ${target} battery to ${battVal}%`, 'fault');
1710 |       } else {
1711 |         sendCommand(`fault ${target} ${fType}`);
1712 |         addLog(`■ INJECTED SENSOR FAULT: ${target} ${fType} Failure`, 'fault');
1713 |       }
1714 |     }
1715 |
1716 |     function clearTargetUav() {
1717 |       const target = document.getElementById('faultTargetUav').value;
1718 |       sendCommand(`clear ${target}`);
1719 |       addLog(`■ Restoring ${target} to 100% nominal state...`, 'success');
1720 |     }
1721 |
1722 |     function addLog(txt, cls='') {
1723 |       const log = document.getElementById('eventLog');
1724 |       const time = new Date().toLocaleTimeString();
1725 |       const div = document.createElement('div');
1726 |       div.className = `log-line ${cls}`;
1727 |       div.innerText = `[${time}] ${txt}`;
1728 |       log.prepend(div);
1729 |     }
1730 |
1731 |     function openPlotsModal() {
1732 |       addLog('Generating scientific evaluation plots from telemetry log...', 'success');
1733 |       fetch('/api/generate_plots', { method: 'POST' })
1734 |         .then(r => r.json())
1735 |         .then(() => {
1736 |           const t = Date.now();
1737 |           document.getElementById('imgTraj').src = `/plots/swarm_3d_flight_trajectories.png?t=${t}`;
1738 |           document.getElementById('imgFdi').src = `/plots/swarm_fdi_safety_timeline.png?t=${t}`;
1739 |           document.getElementById('imgMesh').src = `/plots/swarm_mesh_performance.png?t=${t}`;
1740 |           document.getElementById('plotsModal').classList.add('active');
1741 |         });
1742 |     }
1743 |
1744 |     function closePlotsModal() {
1745 |       document.getElementById('plotsModal').classList.remove('active');
1746 |     }
1747 |
1748 |     // Initialize 3D scene on window load
1749 |     window.addEventListener('load', () => {
1750 |       initThreeScene();
1751 |     });
1752 |   </script>
1753 | </body>
1754 | </html>
```